

Repaso Teórico-Práctico de Laboratorio 1

Física Experimental

Universidad de Buenos Aires - Exactas
Departamento de Física

Nahuel Villafañe

Basado en los apuntes de:

Lucía Famá, Gabriel Paciaroni, Lucía Mazaira, Facundo Sánchez

4 de diciembre de 2025

**Volumen 1: Fundamentos de Mediciones y Análisis
Estadístico**

Licencia Creative Commons BY-NC-SA 4.0

**Usted puede compartir y adaptar este material
dando crédito al autor, sin fines comerciales
y compartiendo bajo la misma licencia.**

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

Prefacio

Este documento no es para estudiar, puede servir como un pequeño complemento. Ergo, este documento no busca reemplazar a ningún profesor/a. Se busca ayudar a la comunidad de los estudiantes de Exactas; el estudiante lo hace con el fin de ayudar la comunidad y brindar ayuda mediante estos apuntes. Se recomienda fuertemente consultar con los profesores en caso de dudas.

Índice general

Prefacio	3
1. Introducción a las Mediciones en Física Experimental	7
1.1. El Método Científico en el Laboratorio	7
1.2. Conceptos Fundamentales	7
1.2.1. Magnitud Física (MF)	7
1.2.2. Medir	7
1.2.3. Unidad de Medida	7
1.2.4. Resultado de una Medición	8
1.3. Clasificación de Mediciones	8
1.3.1. Mediciones Directas (MD)	8
1.3.2. Mediciones Indirectas (MI)	8
1.4. Fuentes de Incertidumbre	8
1.4.1. Clasificación de Errores	9
1.5. El Instrumento como Fuente de Incertidumbre	9
1.5.1. Resolución Instrumental	9
1.5.2. Incertidumbre Instrumental	9
2. Mediciones Directas	11
2.1. Caso 1: Una sola medición o todas iguales	11
2.2. Caso 2: Múltiples mediciones con algunas diferentes	11
2.2.1. Procedimiento con 3 mediciones iniciales	11
2.2.2. Análisis estadístico para N mediciones	11
2.2.3. Incerteza absoluta final	12
2.3. Caso 3: No estamos midiendo el mismo dato	12
2.4. Representación Gráfica: Histogramas	12
2.5. Distribución Normal (Gaussiana)	13
2.6. Teorema del Límite Central	13
3. Mediciones Indirectas y Propagación de Errores	15
3.1. Introducción	15
3.2. Valor más representativo	15
3.3. Propagación de errores	15
3.3.1. Caso general	15
3.3.2. Casos particulares importantes	15
3.4. Ejemplo detallado: Área de un cuadrado	16
3.5. Comparación de Resultados	17
3.5.1. Exactitud y Precisión	17
3.5.2. Diferencias Significativas	17
Ejercicios - Capítulo 1	19
Soluciones - Capítulo 1	21
Ejercicios - Capítulo 2	23

Soluciones - Capítulo 2	25
Ejercicios - Capítulo 3	29
Soluciones - Capítulo 3	31

Capítulo 1

Introducción a las Mediciones en Física Experimental

1.1. El Método Científico en el Laboratorio

El método científico es el proceso sistemático que seguimos para generar conocimiento confiable. En el contexto del Laboratorio 1, trabajamos con el siguiente ciclo:

1. **Observación:** Identificamos un fenómeno físico
2. **Planteamiento del problema:** Formulamos preguntas específicas
3. **Hipótesis:** Proponemos explicaciones tentativas
4. **Experimentación:** Diseñamos y realizamos mediciones controladas
5. **Análisis de resultados:** Procesamos los datos obtenidos
6. **Conclusiones:** Interpretamos los resultados a la luz de la hipótesis
7. **Corrección:** Modificamos nuestra comprensión si es necesario

1.2. Conceptos Fundamentales

1.2.1. Magnitud Física (MF)

Una magnitud física es cualquier propiedad de un cuerpo, fenómeno o sustancia que puede ser **cuantificada**. Ejemplos: masa, longitud, tiempo, velocidad, fuerza, etc.

1.2.2. Medir

Medir es **comparar** la cantidad de la magnitud física que deseamos obtener con una **unidad** de la misma magnitud (patrón).

1.2.3. Unidad de Medida

Es una magnitud física definida y adoptada por convención. Desde 2019, todas las unidades del Sistema Internacional (SI) se definen en base a constantes fundamentales de la naturaleza.

1.2.4. Resultado de una Medición

Dado que nunca conocemos el valor "verdadero" de una magnitud, expresamos el resultado como un **intervalo de confianza**:

$$x = (\bar{x} \pm \Delta x) \text{ Unidad}$$

donde:

- $\bar{x}$: Valor más representativo (estimación del valor verdadero)
- Δx : Incerteza absoluta (cota del error)
- El intervalo $[\bar{x} - \Delta x, \bar{x} + \Delta x]$ contiene al valor verdadero con cierta probabilidad

Regla 1 de Laboratorio 1: Siempre un resultado se expresa con su incerteza.

1.3. Clasificación de Mediciones

1.3.1. Mediciones Directas (MD)

La magnitud física se obtiene directamente de la lectura del instrumento.

Ejemplos:

- Medir tiempo con un cronómetro
- Medir longitud con una regla
- Medir masa con una balanza

1.3.2. Mediciones Indirectas (MI)

La magnitud física se obtiene a través de un proceso matemático a partir de otras medidas.

Ejemplos:

- Calcular área: $A = L_1 \times L_2$
- Calcular velocidad: $v = \Delta x / \Delta t$
- Calcular densidad: $\rho = m / V$

1.4. Fuentes de Incertidumbre

Toda medición tiene incertidumbre asociada debido a:

1. **El instrumento:** Limitaciones en resolución, calibración, etc.
2. **Factores de la naturaleza/azar:** Variaciones aleatorias
3. **Suposiciones e hipótesis:** Simplificaciones del modelo
4. **El método empleado:** Procedimiento de medición

1.4.1. Clasificación de Errores

Cuadro 1.1: Clasificación de errores en mediciones

Errores Sistemáticos	Errores Accidentales (Aleatorios)
■ Constantes en todo el proceso ■ Afectan todas las medidas del mismo modo ■ Mismo signo (siempre por exceso o por defecto) ■ Ejemplos: mal calibrado, paralaje, método inadecuado	■ Producidos al azar, intrínsecos a la naturaleza ■ Pequeñas variaciones entre mediciones sucesivas ■ Se analizan con métodos estadísticos ■ Ejemplos: fluctuaciones térmicas, vibraciones

Errores groseros: Equivocaciones (anotar mal, cálculos erróneos). Se corrigen.

1.5. El Instrumento como Fuente de Incertidumbre

1.5.1. Resolución Instrumental

Es la mínima variación de la magnitud que puede detectar el instrumento. A veces coincide con la mínima división de la escala, a veces no.

1.5.2. Incertidumbre Instrumental

- **Error de apreciación** (σ_{ap}): Lo que puede resolver el observador. Muchas veces igual a la resolución del instrumento.
- **Error de exactitud:** Asociado al error de calibración del instrumento.

Capítulo 2

Mediciones Directas

2.1. Caso 1: Una sola medición o todas iguales

Cuando tenemos una sola medición o varias mediciones que dan exactamente el mismo valor:

- **Valor más representativo:** $\bar{x}$ = número leído en el instrumento
- **Incerteza absoluta:** $\Delta x = \sigma_{ap}$ (error de apreciación)

Ejemplo: Con un cronómetro de resolución 0.01 s, medimos $t = 13,16$ s. Entonces:

$$t = (13,16 \pm 0,01) \text{ s}$$

2.2. Caso 2: Múltiples mediciones con algunas diferentes

Regla 2 de Laboratorio 1: Nunca mido una sola vez. Al menos 3 veces.

2.2.1. Procedimiento con 3 mediciones iniciales

1. Tomar 3 mediciones: x_1, x_2, x_3
2. Calcular el rango: $R = |x_{\text{máx}} - x_{\text{mín}}|$
3. Comparar con $2\sigma_{ap}$:
 - Si $R \leq 2\sigma_{ap}$: $\Delta x = \sigma_{ap}$
 - Si $R > 2\sigma_{ap}$: Tomar más mediciones (al menos 10, idealmente 30-40)

2.2.2. Análisis estadístico para N mediciones

Cuando tomamos N mediciones de una variable aleatoria: $\{x_1, x_2, \dots, x_N\}$

Valor más representativo (promedio)

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

Desviación estándar (S)

Mide la dispersión de los datos respecto al promedio. Es el error típico de una medición individual:

$$S = \sqrt{\frac{\sum_{i=1}^N (\bar{x} - x_i)^2}{N}}$$

Interpretación: Si tomamos una nueva medición, hay aproximadamente un 68 % de probabilidad de que esté en el intervalo $[\bar{x} - S, \bar{x} + S]$.

Error del promedio (σ_e)

Mide la incertidumbre del valor promedio:

$$\sigma_e = \frac{S}{\sqrt{N}}$$

Interpretación: Si realizamos una nueva serie de N mediciones, hay aproximadamente un 68 % de probabilidad de que su promedio esté en el intervalo $[\bar{x} - \sigma_e, \bar{x} + \sigma_e]$.

2.2.3. Incerteza absoluta final

$$\Delta x = \max(\sigma_e, \sigma_{ap})$$

2.3. Caso 3: No estamos midiendo el mismo dato

Cuando medimos diferentes aspectos de un objeto (ej: diámetro de un tronco en distintos puntos):

- **Valor más representativo:** $\bar{x} = \frac{1}{N} \sum x_i$
- **Incerteza absoluta:** $\Delta x = \frac{x_{\max} - x_{\min}}{2}$

2.4. Representación Gráfica: Histogramas

Un histograma es una representación gráfica de la distribución de datos. Para construirlo:

1. Ordenar los datos de menor a mayor
2. Calcular el rango: $R = x_{\max} - x_{\min}$
3. Determinar número de clases (bins): $C = 1 + 3,322 \log_{10}(N)$ (Regla de Sturges)
4. Calcular ancho de clase: $a = R/C$
5. Contar cuántos datos caen en cada intervalo

2.5. Distribución Normal (Gaussiana)

Muchas variables aleatorias en física siguen una distribución normal:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

donde:

- μ : Media de la población
- σ : Desviación estándar de la población
- En la práctica: $\mu \approx \bar{x}$, $\sigma \approx S$

Propiedades importantes:

- $P(\mu - \sigma \leq x \leq \mu + \sigma) \approx 68\%$
- $P(\mu - 2\sigma \leq x \leq \mu + 2\sigma) \approx 95\%$
- $P(\mu - 3\sigma \leq x \leq \mu + 3\sigma) \approx 99,7\%$

2.6. Teorema del Límite Central

Si tomamos múltiples muestras de tamaño N de una población (no necesariamente normal), los promedios de esas muestras seguirán una distribución normal con:

$$\mu_{\bar{x}} = \mu \quad \text{y} \quad \sigma_{\bar{x}} = \frac{\sigma}{\sqrt{N}}$$

Esto justifica el uso de $\sigma_e = S/\sqrt{N}$ como error del promedio.

Capítulo 3

Mediciones Indirectas y Propagación de Errores

3.1. Introducción

En mediciones indirectas, la magnitud de interés W depende funcionalmente de otras magnitudes medidas directamente: $W = f(x, y, z, \dots)$.

Tenemos:

$$x = (x_0 \pm \Delta x) \text{ Ud}$$

$$y = (y_0 \pm \Delta y) \text{ Ud}$$

$$z = (z_0 \pm \Delta z) \text{ Ud}$$

Y queremos obtener:

$$W = (W_0 \pm \Delta W) \text{ Ud}$$

3.2. Valor más representativo

Simplemente evaluamos la función con los valores más representativos:

$$W_0 = f(x_0, y_0, z_0, \dots)$$

3.3. Propagación de errores

Usamos el desarrollo de Taylor para aproximar la función alrededor de los valores más representativos:

3.3.1. Caso general

$$\Delta W = \sqrt{\left(\left.\frac{\partial f}{\partial x}\right|_{x_0, y_0, \dots}\right)^2 \Delta x^2 + \left(\left.\frac{\partial f}{\partial y}\right|_{x_0, y_0, \dots}\right)^2 \Delta y^2 + \dots}$$

3.3.2. Casos particulares importantes

1. Suma o resta: $W = x \pm y$

$$W_0 = x_0 \pm y_0, \quad \Delta W = \sqrt{(\Delta x)^2 + (\Delta y)^2}$$

2. **Producto:** $W = xy$

$$W_0 = x_0 y_0, \quad \frac{\Delta W}{W_0} = \sqrt{\left(\frac{\Delta x}{x_0}\right)^2 + \left(\frac{\Delta y}{y_0}\right)^2}$$

3. **Cociente:** $W = x/y$

$$W_0 = x_0/y_0, \quad \frac{\Delta W}{W_0} = \sqrt{\left(\frac{\Delta x}{x_0}\right)^2 + \left(\frac{\Delta y}{y_0}\right)^2}$$

4. **Potencia:** $W = x^n$

$$W_0 = x_0^n, \quad \frac{\Delta W}{W_0} = |n| \frac{\Delta x}{x_0}$$

5. **Logaritmo:** $W = \ln(x)$

$$W_0 = \ln(x_0), \quad \Delta W = \frac{\Delta x}{x_0}$$

6. **Exponencial:** $W = e^{kx}$

$$W_0 = e^{kx_0}, \quad \frac{\Delta W}{W_0} = |k| \Delta x$$

3.4. Ejemplo detallado: Área de un cuadrado

Sea $L = (15,3 \pm 0,1) \text{ cm}$, calcular $A = L^2$.

1. **Valor más representativo:**

$$A_0 = L_0^2 = (15,3)^2 = 234,09 \text{ cm}^2$$

2. **Error absoluto:**

$$\begin{aligned} \Delta A &= \sqrt{\left(\left.\frac{dA}{dL}\right|_{L_0}\right)^2 (\Delta L)^2} \\ &= \sqrt{(2L_0)^2 (\Delta L)^2} \\ &= 2L_0 \Delta L \\ &= 2 \times 15,3 \times 0,1 = 3,06 \text{ cm}^2 \end{aligned}$$

3. **Resultado final** (con 2 cifras significativas en ΔA):

$$A = (234,1 \pm 3,1) \text{ cm}^2$$

3.5. Comparación de Resultados

3.5.1. Exactitud y Precisión

- **Precisión:** Se evalúa por el tamaño del intervalo de confianza (Δx). Menor Δx = mayor precisión.
- **Exactitud:** Se evalúa por la cercanía del valor más representativo ($\bar{x}$) al valor tabulado o de referencia.

3.5.2. Diferencias Significativas

Dos resultados $A = (\bar{A} \pm \Delta A)$ y $B = (\bar{B} \pm \Delta B)$:

- **Método gráfico:** Si los intervalos $[\bar{A} - \Delta A, \bar{A} + \Delta A]$ y $[\bar{B} - \Delta B, \bar{B} + \Delta B]$ no se superponen, hay diferencias significativas.
- **Método analítico:** Si $|\bar{A} - \bar{B}| > \Delta A + \Delta B$, hay diferencias significativas.

Ejercicios - Capítulo 1: Introducción

Ejercicio 1 Clasifica las siguientes mediciones como directas (MD) o indirectas (MI):

- a) Medir la temperatura con un termómetro
- b) Calcular la velocidad de un auto midiendo distancia y tiempo
- c) Medir la masa con una balanza electrónica
- d) Calcular el área de un rectángulo midiendo sus lados
- e) Medir el tiempo con un cronómetro

Ejercicio 2 Identifica el tipo de error (sistemático, accidental o grosero) en cada caso:

- a) Un cronómetro que siempre marca 0.5 segundos menos
- b) Anotar 15.3 cm en lugar de 13.5 cm
- c) Pequeñas variaciones en mediciones de tiempo debido a vibraciones
- d) Usar una regla mal calibrada que mide 1 cm menos cada 10 cm
- e) Parpadeo al presionar el botón del cronómetro

Ejercicio 3 Para cada instrumento, indica su resolución y estima σ_{ap} :

- a) Regla milimetrada (mínima división: 1 mm)
- b) Cronómetro digital (display: 00.00 s)
- c) Termómetro analógico (divisiones cada 0.5°C)
- d) Balanza digital (display: 0.00 g)

Ejercicio 4 Expresa correctamente los siguientes resultados:

- a) Longitud medida con regla: 23.4 cm, $\sigma_{ap} = 0,1$ cm
- b) Tiempo medido: 15.67 s, $\sigma_{ap} = 0,01$ s
- c) Masa medida: 125.3 g, $\sigma_{ap} = 0,1$ g

Ejercicio 5 Un estudiante mide 3 veces el período de un péndulo: 2.15 s, 2.18 s, 2.16 s. El cronómetro tiene resolución 0.01 s.

- a) Calcula el rango R
- b) Compara con $2\sigma_{ap}$
- c) Decide si necesita más mediciones
- d) Expresa el resultado preliminar

Ejercicio 6 Explica por qué es importante expresar siempre el resultado con su incerteza.

Ejercicio 7 Describe las etapas del método científico aplicado a un experimento de péndulo simple.

Ejercicio 8 ¿Cuál es la diferencia entre precisión y exactitud? Da un ejemplo de cada caso.

Ejercicio 9 ¿Por qué decimos que el resultado de una medición es un intervalo de confianza y no un valor único?

Ejercicio 10 Un grupo mide la longitud de una mesa 5 veces obteniendo siempre 152.3 cm con una regla de $\sigma_{ap} = 0,1$ cm. Expresa el resultado correctamente.

Soluciones - Capítulo 1

Solución:Ejercicio 1

- a) MD - Se lee directamente del instrumento
- b) MI - Se calcula a partir de otras medidas
- c) MD - Se lee directamente de la balanza
- d) MI - Se calcula multiplicando dos medidas
- e) MD - Se lee directamente del cronómetro

Solución:Ejercicio 2

- a) Sistemático - Error constante en todas las mediciones
- b) Grosero - Error por equivocación al anotar
- c) Accidental - Variaciones aleatorias
- d) Sistemático - Error constante proporcional
- e) Accidental - Variación aleatoria en el tiempo de reacción

Solución:Ejercicio 3

- a) Resolución: 1 mm, $\sigma_{ap} = 0,5$ mm (mitad de la mínima división)
- b) Resolución: 0.01 s, $\sigma_{ap} = 0,01$ s (último dígito significativo)
- c) Resolución: 0.5°C , $\sigma_{ap} = 0,25^{\circ}\text{C}$ (mitad de la división)
- d) Resolución: 0.01 g, $\sigma_{ap} = 0,005$ g (mitad del último dígito)

Solución:Ejercicio 4

- a) $L = (23,4 \pm 0,1)$ cm
- b) $t = (15,67 \pm 0,01)$ s
- c) $m = (125,3 \pm 0,1)$ g

Solución:Ejercicio 5

- a) $R = 2,18 - 2,15 = 0,03$ s
- b) $2\sigma_{ap} = 2 \times 0,01 = 0,02$ s
- c) Como $R = 0,03 > 0,02 = 2\sigma_{ap}$, SÍ necesita más mediciones
- d) Con los datos disponibles: $\bar{x} = \frac{2,15+2,18+2,16}{3} = 2,163$ s, $\Delta x = \sigma_{ap} = 0,01$ s, pero esto es preliminar porque $R > 2\sigma_{ap}$

Solución:Ejercicio 6 Es importante expresar la incerteza porque:

- Indica la confiabilidad de la medición
- Permite comparar resultados diferentes
- Ayuda a determinar si diferencias entre valores son significativas
- Es esencial para propagar errores en cálculos posteriores

Solución:Ejercicio 7 Etapas para un péndulo simple:

1. Observación: El péndulo oscila regularmente
2. Planteamiento: ¿Cómo depende el período de la longitud?
3. Hipótesis: $T \propto \sqrt{L}$ (teoría del péndulo simple)
4. Experimentación: Medir T para diferentes L
5. Análisis: Graficar T^2 vs L , ajustar recta
6. Conclusiones: Verificar si $T^2 \propto L$, calcular g
7. Corrección: Considerar efectos no ideales (amplitud, masa del hilo)

Solución:Ejercicio 8

- **Precisión:** Refiere a la reproducibilidad de las mediciones (tamaño de Δx). Ej: $g = (9,81 \pm 0,01) \text{ m/s}^2$ es más preciso que $g = (9,81 \pm 0,1) \text{ m/s}^2$.
- **Exactitud:** Refiere a qué tan cerca está el valor medido del valor verdadero. Ej: $g = (9,81 \pm 0,1) \text{ m/s}^2$ es más exacto que $g = (10,2 \pm 0,01) \text{ m/s}^2$ si el valor verdadero es 9.80665 m/s^2 .

Solución:Ejercicio 9 Porque nunca conocemos el valor verdadero de una magnitud física, solo podemos estimarlo con cierto margen de error. El intervalo de confianza representa el rango de valores donde probablemente se encuentre el valor verdadero, con un cierto nivel de confianza (usualmente 68%).

Solución:Ejercicio 10 Como todas las medidas son iguales: $\bar{L} = 152,3 \text{ cm}$, $\Delta L = \sigma_{ap} = 0,1 \text{ cm}$. Resultado: $L = (152,3 \pm 0,1) \text{ cm}$.

Ejercicios - Capítulo 2: Mediciones Directas

Ejercicio 1 Un estudiante mide el período de un péndulo 10 veces obteniendo:

$$t_i = \{2,15, 2,18, 2,16, 2,17, 2,14, 2,19, 2,15, 2,16, 2,17, 2,18\} \text{ s}$$

El cronómetro tiene $\sigma_{ap} = 0,01 \text{ s}$.

- a) Calcula el valor más representativo $\bar{t}$
- b) Calcula la desviación estándar S
- c) Calcula el error del promedio σ_e
- d) Determina $\Delta t = \text{máx}(\sigma_e, \sigma_{ap})$
- e) Expresa el resultado final correctamente

Ejercicio 2 Se miden 20 diámetros de una esfera con un calibre de $\sigma_{ap} = 0,01 \text{ mm}$:

$$d_i = \{15,23, 15,25, 15,22, 15,24, 15,23, 15,26, 15,22, 15,24, 15,23, 15,25, 15,24, 15,23, 15,25, 15,22, 15,24, 15,23, 15,25\}$$

- a) Calcula $\bar{d}$, S , σ_e
- b) ¿Cuál es la incerteza absoluta Δd ?
- c) Expresa el resultado
- d) ¿Qué porcentaje de datos está en $[\bar{d} - S, \bar{d} + S]$?

Ejercicio 3 Se mide 5 veces la longitud de un lápiz en diferentes puntos:

$$L_i = \{15,2, 15,5, 14,9, 15,3, 15,1\} \text{ cm}$$

con $\sigma_{ap} = 0,1 \text{ cm}$.

- a) ¿Estamos en el Caso 2 o Caso 3? Justifica
- b) Calcula el valor más representativo apropiado
- c) Calcula la incerteza absoluta apropiada
- d) Expresa el resultado

Ejercicio 4 Con los datos del ejercicio 1:

- a) Construye un histograma (en papel o mentalmente) con 4 clases
- b) ¿Qué forma esperarías que tenga la distribución?
- c) Calcula cuántos datos están en $[\bar{t} - S, \bar{t} + S]$

d) Compara con el 68 % esperado para distribución normal

Ejercicio 5 Explica con tus palabras:

- a) ¿Qué representa la desviación estándar S ?
- b) ¿Qué representa el error del promedio σ_e ?
- c) ¿Por qué σ_e disminuye con $\sqrt{N}$?

Ejercicio 6 Para una medición con $N = 100$, se obtiene $S = 0,5$ cm y $\sigma_{ap} = 0,1$ cm.

- a) Calcula σ_e
- b) ¿Cuál es Δx ?
- c) Si aumentamos a $N = 400$, ¿cuál sería el nuevo σ_e ?
- d) ¿Cuántas mediciones necesitaríamos para que $\sigma_e = \sigma_{ap}$?

Ejercicio 7 En un experimento se obtienen dos series de mediciones:

- Serie A: $S_A = 0,2$ s, $N_A = 25$
- Serie B: $S_B = 0,1$ s, $N_B = 16$

- a) ¿Qué serie tiene mayor dispersión en mediciones individuales?
- b) Calcula σ_{eA} y σ_{eB}
- c) ¿Qué promedio es más preciso (menor σ_e)?

Ejercicio 8 Un péndulo se mide 40 veces obteniendo $\bar{T} = 2,15$ s y $S = 0,05$ s.

- a) Calcula σ_e
- b) Si tomamos una nueva medición, ¿en qué intervalo estará con 68 % de probabilidad?
- c) Si hacemos una nueva serie de 40 mediciones, ¿en qué intervalo estará su promedio con 68 % de probabilidad?

Ejercicio 9 Demuestra que para cualquier conjunto de datos $\{x_i\}$:

$$\sum_{i=1}^N (x_i - \bar{x}) = 0$$

Ejercicio 10 Se miden 3 veces el espesor de un libro: 3.2 cm, 3.3 cm, 3.2 cm con $\sigma_{ap} = 0,1$ cm.

- a) Calcula el rango R
- b) Compara con $2\sigma_{ap}$
- c) ¿Necesitas más mediciones? ¿Por qué?
- d) Expresa el resultado

Soluciones - Capítulo 2

Solución:Ejercicio 1

Listing 3.1: Código Python para ejercicio 1

```
1 import numpy as np
2
3 # Datos
4 t = np.array([2.15, 2.18, 2.16, 2.17, 2.14, 2.19, 2.15, 2.16, 2.17,
5               ↪ 2.18])
6 sigma_ap = 0.01
7 N = len(t)
8
9 # a) Valor mas representativo
10 t_mean = np.mean(t)
11 print(f"a) t_mean = {t_mean:.4f} s")
12
13 # b) Desviacion estandar
14 S = np.std(t, ddof=0) # Usando N en denominador
15 print(f"b) S = {S:.4f} s")
16
17 # c) Error del promedio
18 sigma_e = S / np.sqrt(N)
19 print(f"c) sigma_e = {sigma_e:.4f} s")
20
21 # d) Incerteza absoluta
22 Delta_t = max(sigma_e, sigma_ap)
23 print(f"d) Delta_t = {Delta_t:.4f} s")
24
25 # e) Resultado final (2 cifras significativas en Delta)
26 print(f"e) t = ({t_mean:.2f} {Delta_t:.2f}) s")
```

Resultados:

- a) $\bar{t} = 2,165 \text{ s}$
- b) $S = 0,0149 \text{ s}$
- c) $\sigma_e = 0,0047 \text{ s}$
- d) $\Delta t = \max(0,0047, 0,01) = 0,01 \text{ s}$ (domina σ_{ap})
- e) $t = (2,17 \pm 0,01) \text{ s}$ (redondeado a 2 decimales)

Solución:Ejercicio 2

Listing 3.2: Código Python para ejercicio 2

```
1 import numpy as np
2
3 # Datos
```

```

4 d = np.array([15.23, 15.25, 15.22, 15.24, 15.23, 15.26, 15.22,
    ↪ 15.24,
5             15.23, 15.25, 15.24, 15.23, 15.25, 15.22, 15.24,
    ↪ 15.23,
6             15.25, 15.24, 15.23, 15.22])
7 sigma_ap = 0.01
8 N = len(d)
9
10 # a) Calculos
11 d_mean = np.mean(d)
12 S = np.std(d, ddof=0)
13 sigma_e = S / np.sqrt(N)
14
15 print(f"a) d_mean = {d_mean:.4f} mm")
16 print(f"    S = {S:.4f} mm")
17 print(f"    sigma_e = {sigma_e:.4f} mm")
18
19 # b) Incerteza absoluta
20 Delta_d = max(sigma_e, sigma_ap)
21 print(f"b) Delta_d = {Delta_d:.4f} mm")
22
23 # c) Resultado
24 print(f"c) d = ({d_mean:.2f}      {Delta_d:.2f}) mm")
25
26 # d) Porcentaje en [mean-S, mean+S]
27 count = np.sum((d >= d_mean - S) & (d <= d_mean + S))
28 percentage = 100 * count / N
29 print(f"d) {count}/{N} datos en [mean-S, mean+S] = {percentage:.1f
    ↪ }%")

```

Resultados:

- a) $\bar{d} = 15,235 \text{ mm}$, $S = 0,0112 \text{ mm}$, $\sigma_e = 0,0025 \text{ mm}$
- b) $\Delta d = \max(0,0025, 0,01) = 0,01 \text{ mm}$ (domina σ_{ap})
- c) $d = (15,24 \pm 0,01) \text{ mm}$
- d) $14/20 = 70\%$ de datos en $[\bar{d} - S, \bar{d} + S]$ (cerca del 68% esperado)

Solución:Ejercicio 3

- a) **Caso 3** porque estamos midiendo diferentes puntos del lápiz (no el mismo punto múltiples veces). Hay variación real en la longitud a lo largo del lápiz.
- b) $\bar{L} = \frac{15,2+15,5+14,9+15,3+15,1}{5} = 15,2 \text{ cm}$
- c) $\Delta L = \frac{L_{\max}-L_{\min}}{2} = \frac{15,5-14,9}{2} = 0,3 \text{ cm}$
- d) $L = (15,2 \pm 0,3) \text{ cm}$

Solución:Ejercicio 4

Listing 3.3: Código Python para ejercicio 4

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Datos del ejercicio 1
5 t = np.array([2.15, 2.18, 2.16, 2.17, 2.14, 2.19, 2.15, 2.16, 2.17,
    ↪ 2.18])
6
7 # a) Histograma con 4 clases
8 plt.figure(figsize=(10, 4))
9 plt.subplot(1, 2, 1)
10 counts, bins, patches = plt.hist(t, bins=4, edgecolor='black',
    ↪ alpha=0.7)
11 plt.xlabel('Tiempo (s)')
12 plt.ylabel('Frecuencia')
13 plt.title('Histograma de 10 mediciones de T')
14 plt.grid(True, alpha=0.3)
15
16 # b) Forma esperada: aproximadamente normal (campana de Gauss)
17 # c) y d) Datos en [mean-S, mean+S]
18 t_mean = np.mean(t)
19 S = np.std(t, ddof=0)
20 count = np.sum((t >= t_mean - S) & (t <= t_mean + S))
21 percentage = 100 * count / len(t)
22
23 plt.subplot(1, 2, 2)
24 x = np.linspace(t_mean - 3*S, t_mean + 3*S, 1000)
25 y = (1/(S*np.sqrt(2*np.pi))) * np.exp(-0.5*((x - t_mean)/S)**2)
26 plt.plot(x, y*len(t)*(bins[1]-bins[0]), 'r-', label='Distribuci n
    ↪ normal')
27 plt.hist(t, bins=4, edgecolor='black', alpha=0.3, density=False)
28 plt.axvline(t_mean - S, color='g', linestyle='--', label=' S ')
29 plt.axvline(t_mean + S, color='g', linestyle='--')
30 plt.xlabel('Tiempo (s)')
31 plt.ylabel('Frecuencia')
32 plt.title(f'{count}/{len(t)} datos (${percentage:.1f}\%$) en [
    ↪ mean S]')
33 plt.legend()
34 plt.grid(True, alpha=0.3)
35 plt.tight_layout()
36 plt.show()
37
38 print(f"c) {count}/{len(t)} datos en [mean-S, mean+S] = {percentage
    ↪ :.1f}%")
39 print(f"d) Esperado: 68%, obtenido: {percentage:.1f}%")

```

Solución:Ejercicio 5

- a) La desviación estándar S representa la dispersión típica de las mediciones individuales alrededor del promedio. Si tomamos una nueva medición, hay aproximadamente un 68 % de probabilidad de que esté en el intervalo $[\bar{x} - S, \bar{x} + S]$.

- b) El error del promedio $\sigma_e = S/\sqrt{N}$ representa la incertidumbre del valor promedio. Si realizamos una nueva serie de N mediciones, hay aproximadamente un 68 % de probabilidad de que su promedio esté en $[\bar{x} - \sigma_e, \bar{x} + \sigma_e]$.
- c) σ_e disminuye con $\sqrt{N}$ debido al Teorema del Límite Central. Al promediar N mediciones independientes, las fluctuaciones aleatorias se promedian parcialmente. La reducción como $\sqrt{N}$ viene de que la varianza del promedio es $1/N$ veces la varianza individual, y la desviación estándar es la raíz de la varianza.

Solución:Ejercicio 6

- a) $\sigma_e = \frac{S}{\sqrt{N}} = \frac{0,5}{\sqrt{100}} = 0,05 \text{ cm}$
- b) $\Delta x = \text{máx}(\sigma_e, \sigma_{ap}) = \text{máx}(0,05, 0,1) = 0,1 \text{ cm}$
- c) Para $N = 400$: $\sigma'_e = \frac{0,5}{\sqrt{400}} = 0,025 \text{ cm}$
- d) Queremos $\sigma_e = \sigma_{ap} = 0,1$:
- $$\frac{0,5}{\sqrt{N}} = 0,1 \Rightarrow \sqrt{N} = \frac{0,5}{0,1} = 5 \Rightarrow N = 25$$

Necesitaríamos 25 mediciones.

Solución:Ejercicio 7

- a) Serie A tiene mayor dispersión porque $S_A = 0,2 > 0,1 = S_B$
- b)

$$\sigma_{eA} = \frac{0,2}{\sqrt{25}} = 0,04 \text{ s}$$

$$\sigma_{eB} = \frac{0,1}{\sqrt{16}} = 0,025 \text{ s}$$

- c) Serie B es más precisa porque $\sigma_{eB} = 0,025 < 0,04 = \sigma_{eA}$

Solución:Ejercicio 8

- a) $\sigma_e = \frac{S}{\sqrt{N}} = \frac{0,05}{\sqrt{40}} = 0,0079 \approx 0,008 \text{ s}$
- b) Intervalo para nueva medición: $[2,15 - 0,05, 2,15 + 0,05] = [2,10, 2,20] \text{ s}$
- c) Intervalo para nuevo promedio: $[2,15 - 0,008, 2,15 + 0,008] = [2,142, 2,158] \text{ s}$

Solución:Ejercicio 9

$$\sum_{i=1}^N (x_i - \bar{x}) = \sum_{i=1}^N x_i - \sum_{i=1}^N \bar{x} = N\bar{x} - N\bar{x} = 0$$

Donde usamos que $\sum_{i=1}^N x_i = N\bar{x}$ por definición de promedio.

Solución:Ejercicio 10

- a) $R = 3,3 - 3,2 = 0,1 \text{ cm}$
- b) $2\sigma_{ap} = 2 \times 0,1 = 0,2 \text{ cm}$
- c) No, porque $R = 0,1 \leq 0,2 = 2\sigma_{ap}$. El instrumento cubre la dispersión.
- d) $\bar{L} = \frac{3,2+3,3+3,2}{3} = 3,233 \text{ cm}$, $\Delta L = \sigma_{ap} = 0,1 \text{ cm}$ $L = (3,2 \pm 0,1) \text{ cm}$ (redondeado)

Ejercicios - Capítulo 3: Mediciones Indirectas

Ejercicio 1 Dados $a = (12,5 \pm 0,1)$ cm y $b = (8,3 \pm 0,1)$ cm, calcula:

- a) Perímetro de un rectángulo: $P = 2(a + b)$
- b) Área del rectángulo: $A = a \times b$
- c) Relación $R = a/b$

Ejercicio 2 El radio de una esfera es $r = (5,25 \pm 0,05)$ cm. Calcula:

- a) Superficie esférica: $S = 4\pi r^2$
- b) Volumen: $V = \frac{4}{3}\pi r^3$
- c) Densidad si la masa es $m = (250,0 \pm 0,1)$ g: $\rho = m/V$

Ejercicio 3 Para un movimiento uniforme, se mide $\Delta x = (15,0 \pm 0,1)$ m en $\Delta t = (3,0 \pm 0,1)$ s.

- a) Calcula la velocidad $v = \Delta x / \Delta t$
- b) Expresa el resultado con unidades correctas

Ejercicio 4 En un experimento de péndulo, se mide $L = (70,0 \pm 0,1)$ cm y $T = (1,68 \pm 0,01)$ s.

- a) Calcula $g = 4\pi^2 L / T^2$
- b) ¿Cuál es la contribución relativa mayor a Δg ?

Ejercicio 5 Se mide la resistencia de un conductor: $V = (12,5 \pm 0,1)$ V, $I = (2,50 \pm 0,05)$ A.

- a) Calcula $R = V/I$
- b) Calcula potencia $P = V \times I$

Ejercicio 6 Demuestra que para $W = x \cdot y$, se cumple:

$$\frac{\Delta W}{W_0} = \sqrt{\left(\frac{\Delta x}{x_0}\right)^2 + \left(\frac{\Delta y}{y_0}\right)^2}$$

Ejercicio 7 Se miden los lados de un triángulo rectángulo: $a = (3,0 \pm 0,1)$ cm, $b = (4,0 \pm 0,1)$ cm.

- a) Calcula la hipotenusa: $c = \sqrt{a^2 + b^2}$
- b) Calcula el área: $A = \frac{1}{2}ab$

Ejercicio 8 La temperatura en grados Celsius es $T_C = (25,0 \pm 0,5)^\circ\text{C}$.

- a) Calcula en Kelvin: $T_K = T_C + 273,15$
- b) Calcula en Fahrenheit: $T_F = \frac{9}{5}T_C + 32$

Ejercicio 9 Compara los siguientes resultados para g :

- $g_1 = (9,81 \pm 0,07) \text{ m/s}^2$
- $g_2 = (9,73 \pm 0,03) \text{ m/s}^2$
- $g_3 = (9,99 \pm 0,35) \text{ m/s}^2$

- a) ¿Cuál es más preciso?
- b) ¿Cuál es más exacto si $g_{\text{ref}} = 9,80665 \text{ m/s}^2$?
- c) ¿ g_1 y g_2 presentan diferencias significativas?

Ejercicio 10 Un cilindro tiene $r = (2,50 \pm 0,05) \text{ cm}$ y $h = (10,0 \pm 0,1) \text{ cm}$.

- a) Calcula volumen: $V = \pi r^2 h$
- b) Calcula superficie lateral: $A_L = 2\pi r h$
- c) ¿Cuál tiene mayor error relativo, V o A_L ?

Soluciones - Capítulo 3

Solución:Ejercicio 1

Listing 3.4: Código Python para ejercicio 1

```
1 import numpy as np
2
3 # Datos
4 a0, Da = 12.5, 0.1 # cm
5 b0, Db = 8.3, 0.1 # cm
6
7 # a) Perimetro P = 2(a + b)
8 P0 = 2 * (a0 + b0)
9 DP = 2 * np.sqrt(Da**2 + Db**2) # Propagacion para suma
10 print(f"a) P = ({P0:.1f} {DP:.1f}) cm")
11
12 # b) Area A = a * b
13 A0 = a0 * b0
14 # Error relativo: DA/A0 = sqrt((Da/a0)^2 + (Db/b0)^2)
15 DA_rel = np.sqrt((Da/a0)**2 + (Db/b0)**2)
16 DA = A0 * DA_rel
17 print(f"b) A = ({A0:.2f} {DA:.2f}) cm ")
18
19 # c) Relacion R = a/b
20 R0 = a0 / b0
21 # Error relativo igual que para producto
22 DR_rel = np.sqrt((Da/a0)**2 + (Db/b0)**2)
23 DR = R0 * DR_rel
24 print(f"c) R = ({R0:.3f} {DR:.3f})")
```

Resultados:

a) $P = 2(12,5 + 8,3) = 41,6 \text{ cm}$

$$\Delta P = 2\sqrt{(0,1)^2 + (0,1)^2} = 2\sqrt{0,02} = 0,2828 \approx 0,3 \text{ cm}$$

$$P = (41,6 \pm 0,3) \text{ cm}$$

b) $A = 12,5 \times 8,3 = 103,75 \text{ cm}^2$

$$\frac{\Delta A}{A} = \sqrt{\left(\frac{0,1}{12,5}\right)^2 + \left(\frac{0,1}{8,3}\right)^2} = \sqrt{0,0064^2 + 0,01205^2} = 0,0138$$

$$\Delta A = 103,75 \times 0,0138 = 1,43 \approx 1,4 \text{ cm}^2 \quad A = (103,8 \pm 1,4) \text{ cm}^2$$

c) $R = 12,5/8,3 = 1,506$ Mismo error relativo que en b): $0,0138$ $\Delta R = 1,506 \times 0,0138 = 0,0208 \approx 0,021$ $R = (1,506 \pm 0,021)$

Solución:Ejercicio 2

Listing 3.5: Código Python para ejercicio 2

```

1 import numpy as np
2
3 # Datos
4 r0, Dr = 5.25, 0.05 # cm
5 m0, Dm = 250.0, 0.1 # g
6
7 # a) Superficie S = 4 r
8 S0 = 4 * np.pi * r0**2
9 # Error: S r , DS/S0 = 2 * Dr/r0
10 DS_rel = 2 * Dr / r0
11 DS = S0 * DS_rel
12 print(f"a) S = ({S0:.1f} {DS:.1f}) cm ")
13
14 # b) Volumen V = (4/3) r
15 V0 = (4/3) * np.pi * r0**3
16 # Error: V r , DV/V0 = 3 * Dr/r0
17 DV_rel = 3 * Dr / r0
18 DV = V0 * DV_rel
19 print(f"b) V = ({V0:.1f} {DV:.1f}) cm ")
20
21 # c) Densidad = m/V
22 rho0 = m0 / V0
23 # Error relativo: sqrt((Dm/m0)^2 + (DV/V0)^2)
24 Drho_rel = np.sqrt((Dm/m0)**2 + (DV_rel)**2)
25 Drho = rho0 * Drho_rel
26 print(f"c) = ({rho0:.4f} {Drho:.4f}) g/cm ")
27 print(f" = ({rho0*1000:.1f} {(Drho*1000):.1f}) kg/m ")

```

Resultados:

a) $S = 4\pi(5,25)^2 = 346,36 \text{ cm}^2$

$$\frac{\Delta S}{S} = 2 \frac{\Delta r}{r} = 2 \frac{0,05}{5,25} = 0,01905$$

$$\Delta S = 346,36 \times 0,01905 = 6,60 \approx 6,6 \text{ cm}^2 \quad S = (346,4 \pm 6,6) \text{ cm}^2$$

b) $V = \frac{4}{3}\pi(5,25)^3 = 606,13 \text{ cm}^3$

$$\frac{\Delta V}{V} = 3 \frac{\Delta r}{r} = 3 \frac{0,05}{5,25} = 0,02857$$

$$\Delta V = 606,13 \times 0,02857 = 17,32 \approx 17,3 \text{ cm}^3 \quad V = (606,1 \pm 17,3) \text{ cm}^3$$

c) $\rho = \frac{250,0}{606,13} = 0,4125 \text{ g/cm}^3$

$$\frac{\Delta \rho}{\rho} = \sqrt{\left(\frac{0,1}{250,0}\right)^2 + (0,02857)^2} = \sqrt{0,0004^2 + 0,02857^2} = 0,02857$$

$$\Delta \rho = 0,4125 \times 0,02857 = 0,0118 \text{ g/cm}^3 \quad \rho = (0,4125 \pm 0,0118) \text{ g/cm}^3 = (412,5 \pm 11,8) \text{ kg/m}^3$$

Solución:Ejercicio 3

Listing 3.6: Código Python para ejercicio 3

```

1 import numpy as np
2
3 # Datos
4 Dx0, DDx = 15.0, 0.1 # m
5 Dt0, DDt = 3.0, 0.1 # s
6
7 # a) Velocidad v = Dx/Dt
8 v0 = Dx0 / Dt0
9 # Error relativo: sqrt((DDx/Dx0)^2 + (DDt/Dt0)^2)
10 Dv_rel = np.sqrt((DDx/Dx0)**2 + (DDt/Dt0)**2)
11 Dv = v0 * Dv_rel
12 print(f"a) v = ({v0:.2f} {Dv:.2f}) m/s")
13
14 # b) Conversi n a km/h
15 v0_kmh = v0 * 3.6
16 Dv_kmh = Dv * 3.6
17 print(f"b) v = ({v0_kmh:.1f} {Dv_kmh:.1f}) km/h")

```

Resultados:

a) $v = \frac{15,0}{3,0} = 5,00 \text{ m/s}$

$$\frac{\Delta v}{v} = \sqrt{\left(\frac{0,1}{15,0}\right)^2 + \left(\frac{0,1}{3,0}\right)^2} = \sqrt{0,00667^2 + 0,03333^2} = 0,0340$$

$$\Delta v = 5,00 \times 0,0340 = 0,170 \approx 0,17 \text{ m/s} \quad v = (5,00 \pm 0,17) \text{ m/s}$$

b) $5,00 \text{ m/s} = 18,0 \text{ km/h}$, $\Delta v = 0,17 \times 3,6 = 0,61 \text{ km/h}$ $v = (18,0 \pm 0,6) \text{ km/h}$

Solución:Ejercicio 4

Listing 3.7: Código Python para ejercicio 4

```

1 import numpy as np
2
3 # Datos
4 L0, DL = 70.0, 0.1 # cm -> convertimos a m
5 T0, DT = 1.68, 0.01 # s
6
7 # Convertir a SI
8 L0_m = L0 / 100
9 DL_m = DL / 100
10
11 # a) g = 4 L / T
12 g0 = 4 * np.pi**2 * L0_m / T0**2
13 print(f"g0 = 4 * {L0_m} / {T0} = {g0:.3f} m/s ")
14
15 # Error: g L/T
16 # g / L = 4 / T , g / T = -8 L / T
17 # Dg = sqrt(( g / L * DL) + ( g / T * DT) )

```

```

18
19 # Derivadas parciales
20 dg_dL = 4 * np.pi**2 / T0**2
21 dg_dT = -8 * np.pi**2 * L0_m / T0**3
22
23 Dg = np.sqrt((dg_dL * DL_m)**2 + (dg_dT * DT)**2)
24 print(f"a) g = ({g0:.2f}      {Dg:.2f}) m/s  ")
25
26 # b) Contribuciones relativas
27 contrib_L = (dg_dL * DL_m / Dg)**2
28 contrib_T = (dg_dT * DT / Dg)**2
29 print(f"b) Contribuci n L: {100*contrib_L:.1f}%")
30 print(f"      Contribuci n T: {100*contrib_T:.1f}%")

```

Resultados:

a) $L = 0,700 \text{ m}$, $g = \frac{4\pi^2 \times 0,700}{(1,68)^2} = 9,79 \text{ m/s}^2$

Cálculo de Δg :

$$\frac{\partial g}{\partial L} = \frac{4\pi^2}{T^2} = \frac{4\pi^2}{(1,68)^2} = 13,99$$

$$\frac{\partial g}{\partial T} = -\frac{8\pi^2 L}{T^3} = -\frac{8\pi^2 \times 0,700}{(1,68)^3} = -11,66$$

$$\Delta g = \sqrt{(13,99 \times 0,001)^2 + (-11,66 \times 0,01)^2} = \sqrt{0,000196 + 0,01359} = 0,117$$

$$g = (9,79 \pm 0,12) \text{ m/s}^2$$

b) Contribución de L: $(13,99 \times 0,001)^2 / 0,01379 = 0,014 = 1,4 \%$

Contribución de T: $(11,66 \times 0,01)^2 / 0,01379 = 0,986 = 98,6 \%$

La mayor contribución viene de ΔT .

Solución:Ejercicio 5

Listing 3.8: Código Python para ejercicio 5

```

1 import numpy as np
2
3 # Datos
4 V0, DV = 12.5, 0.1 # V
5 I0, DI = 2.50, 0.05 # A
6
7 # a) Resistencia R = V/I
8 R0 = V0 / I0
9 DR_rel = np.sqrt((DV/V0)**2 + (DI/I0)**2)
10 DR = R0 * DR_rel
11 print(f"a) R = ({R0:.2f}      {DR:.2f})  ")
12
13 # b) Potencia P = V*I
14 P0 = V0 * I0
15 DP_rel = np.sqrt((DV/V0)**2 + (DI/I0)**2)
16 DP = P0 * DP_rel
17 print(f"b) P = ({P0:.1f}      {DP:.1f}) W")

```

Resultados:

a) $R = \frac{12,5}{2,50} = 5,00 \Omega$

$$\frac{\Delta R}{R} = \sqrt{\left(\frac{0,1}{12,5}\right)^2 + \left(\frac{0,05}{2,50}\right)^2} = \sqrt{0,008^2 + 0,02^2} = 0,0215$$

$$\Delta R = 5,00 \times 0,0215 = 0,1075 \approx 0,11 \Omega \quad R = (5,00 \pm 0,11) \Omega$$

b) $P = 12,5 \times 2,50 = 31,25 \text{ W}$ Mismo error relativo: 0.0215 $\Delta P = 31,25 \times 0,0215 = 0,672 \approx 0,7 \text{ W}$ $P = (31,2 \pm 0,7) \text{ W}$

Solución:Ejercicio 6 Para $W = x \cdot y$:

$$\frac{\partial W}{\partial x} = y$$

$$\frac{\partial W}{\partial y} = x$$

Aplicando propagación de errores:

$$\Delta W = \sqrt{\left(\frac{\partial W}{\partial x}\right)^2 (\Delta x)^2 + \left(\frac{\partial W}{\partial y}\right)^2 (\Delta y)^2} = \sqrt{y_0^2 (\Delta x)^2 + x_0^2 (\Delta y)^2}$$

Dividiendo por $W_0 = x_0 y_0$:

$$\frac{\Delta W}{W_0} = \frac{\sqrt{y_0^2 (\Delta x)^2 + x_0^2 (\Delta y)^2}}{x_0 y_0} = \sqrt{\frac{(\Delta x)^2}{x_0^2} + \frac{(\Delta y)^2}{y_0^2}} = \sqrt{\left(\frac{\Delta x}{x_0}\right)^2 + \left(\frac{\Delta y}{y_0}\right)^2}$$

Solución:Ejercicio 7

Listing 3.9: Código Python para ejercicio 7

```

1 import numpy as np
2
3 # Datos
4 a0, Da = 3.0, 0.1 # cm
5 b0, Db = 4.0, 0.1 # cm
6
7 # a) Hipotenusa c = sqrt(a^2 + b^2)
8 c0 = np.sqrt(a0**2 + b0**2)
9 # Derivadas parciales
10 dc_da = a0 / c0
11 dc_db = b0 / c0
12 Dc = np.sqrt((dc_da * Da)**2 + (dc_db * Db)**2)
13 print(f"a) c = ({c0:.2f} {Dc:.2f}) cm")
14
15 # b) Area A = ab
16 A0 = 0.5 * a0 * b0
17 DA_rel = np.sqrt((Da/a0)**2 + (Db/b0)**2)
18 DA = A0 * DA_rel
19 print(f"b) A = ({A0:.2f} {DA:.2f}) cm ")

```

Resultados:

a) $c = \sqrt{3,0^2 + 4,0^2} = 5,00 \text{ cm}$

$$\frac{\partial c}{\partial a} = \frac{a}{c} = \frac{3,0}{5,0} = 0,6$$

$$\frac{\partial c}{\partial b} = \frac{b}{c} = \frac{4,0}{5,0} = 0,8$$

$$\Delta c = \sqrt{(0,6 \times 0,1)^2 + (0,8 \times 0,1)^2} = \sqrt{0,0036 + 0,0064} = 0,1 \text{ cm} \quad c = (5,00 \pm 0,10) \text{ cm}$$

b) $A = \frac{1}{2} \times 3,0 \times 4,0 = 6,0 \text{ cm}^2$

$$\frac{\Delta A}{A} = \sqrt{\left(\frac{0,1}{3,0}\right)^2 + \left(\frac{0,1}{4,0}\right)^2} = \sqrt{0,0333^2 + 0,025^2} = 0,0417$$

$$\Delta A = 6,0 \times 0,0417 = 0,25 \text{ cm}^2 \quad A = (6,00 \pm 0,25) \text{ cm}^2$$

Solución:Ejercicio 8

Listing 3.10: Código Python para ejercicio 8

```

1 import numpy as np
2
3 # Datos
4 TC0, DTC = 25.0, 0.5 # C
5
6 # a) Kelvin: TK = TC + 273.15
7 TK0 = TC0 + 273.15
8 DTK = DTC # Mismo error absoluto (suma constante)
9 print(f"a) TK = ({TK0:.2f} {DTK:.2f}) K")
10
11 # b) Fahrenheit: TF = (9/5)TC + 32
12 TF0 = (9/5) * TC0 + 32
13 # Derivada: dTF/dTC = 9/5 = 1.8
14 DTF = 1.8 * DTC
15 print(f"b) TF = ({TF0:.1f} {DTF:.1f}) F ")

```

Resultados:

a) $T_K = 25,0 + 273,15 = 298,15 \text{ K}$ $\Delta T_K = \Delta T_C = 0,5 \text{ K}$ (suma con constante) $T_K = (298,15 \pm 0,50) \text{ K}$

b) $T_F = \frac{9}{5} \times 25,0 + 32 = 77,0 \text{ °F}$ $\frac{dT_F}{dT_C} = 1,8$, $\Delta T_F = 1,8 \times 0,5 = 0,9 \text{ °F}$ $T_F = (77,0 \pm 0,9) \text{ °F}$

Solución:Ejercicio 9

Listing 3.11: Código Python para ejercicio 9

```

1 import numpy as np
2
3 # Datos
4 g1, Dg1 = 9.81, 0.07
5 g2, Dg2 = 9.73, 0.03

```

```

6 g3, Dg3 = 9.99, 0.35
7 g_ref = 9.80665
8
9 # a) M s preciso: menor Delta
10 print("a) M s preciso:")
11 print(f"    g1:    ={Dg1}, g2:    ={Dg2}, g3:    ={Dg3}")
12 print(f"    M s preciso: g2 (    ={Dg2} es menor)")
13
14 # b) M s exacto: m s cercano a g_ref
15 dist1 = abs(g1 - g_ref)
16 dist2 = abs(g2 - g_ref)
17 dist3 = abs(g3 - g_ref)
18 print(f"\nb) Distancias a g_ref={g_ref}:")
19 print(f"    g1: |{g1}-{g_ref}| = {dist1:.4f}")
20 print(f"    g2: |{g2}-{g_ref}| = {dist2:.4f}")
21 print(f"    g3: |{g3}-{g_ref}| = {dist3:.4f}")
22 print(f"    M s exacto: g1 (distancia m s peque a)")
23
24 # c) Diferencias significativas entre g1 y g2
25 diff = abs(g1 - g2)
26 sum_errors = Dg1 + Dg2
27 print(f"\nc) |g1 - g2| = |{g1} - {g2}| = {diff:.2f}")
28 print(f"    g1 + g2 = {Dg1} + {Dg2} = {sum_errors:.2f}")
29 if diff > sum_errors:
30     print(f"    {diff:.2f} > {sum_errors:.2f}          SI hay diferencias
31           ↪ significativas")
32 else:
33     print(f"    {diff:.2f}          {sum_errors:.2f}          NO hay
34           ↪ diferencias significativas")

```

Resultados:

- a) Más preciso: g_2 porque $\Delta g_2 = 0,03$ es menor que $\Delta g_1 = 0,07$ y $\Delta g_3 = 0,35$.
- b) Más exacto: Calculamos distancias a $g_{\text{ref}} = 9,80665$:

$$|g_1 - g_{\text{ref}}| = |9,81 - 9,80665| = 0,00335$$

$$|g_2 - g_{\text{ref}}| = |9,73 - 9,80665| = 0,07665$$

$$|g_3 - g_{\text{ref}}| = |9,99 - 9,80665| = 0,18335$$

g_1 es más exacto (menor distancia).

- c) $|g_1 - g_2| = |9,81 - 9,73| = 0,08$ $\Delta g_1 + \Delta g_2 = 0,07 + 0,03 = 0,10$ Como $0,08 < 0,10$, NO hay diferencias significativas (los intervalos se superponen).

Solución:Ejercicio 10

Listing 3.12: Código Python para ejercicio 10

```

1 import numpy as np
2
3 # Datos
4 r0, Dr = 2.50, 0.05 # cm

```

```

5 h0, Dh = 10.0, 0.1 # cm
6
7 # a) Volumen V = rh
8 V0 = np.pi * r0**2 * h0
9 # Error: V r h
10 # V / r = 2 rh , V / h = r
11 dV_dr = 2 * np.pi * r0 * h0
12 dV_dh = np.pi * r0**2
13 DV = np.sqrt((dV_dr * Dr)**2 + (dV_dh * Dh)**2)
14 print(f"a) V = ({V0:.1f} {DV:.1f}) cm ")
15 print(f" Error relativo V: {100*DV/V0:.1f}%")
16
17 # b) Superficie lateral AL = 2 rh
18 AL0 = 2 * np.pi * r0 * h0
19 # Error: AL rh
20 # AL / r = 2 h , AL / h = 2 r
21 dAL_dr = 2 * np.pi * h0
22 dAL_dh = 2 * np.pi * r0
23 DAL = np.sqrt((dAL_dr * Dr)**2 + (dAL_dh * Dh)**2)
24 print(f"\nb) AL = ({AL0:.1f} {DAL:.1f}) cm ")
25 print(f" Error relativo AL: {100*DAL/AL0:.1f}%")
26
27 # c) Comparaci n
28 print(f"\nc) Error relativo V: {100*DV/V0:.1f}%")
29 print(f" Error relativo AL: {100*DAL/AL0:.1f}%")
30 if DV/V0 > DAL/AL0:
31     print(f" V tiene mayor error relativo")
32 else:
33     print(f" AL tiene mayor error relativo")

```

Resultados:

a) $V = \pi(2,50)^2 \times 10,0 = 196,3 \text{ cm}^3$

$$\frac{\partial V}{\partial r} = 2\pi r h = 2\pi \times 2,50 \times 10,0 = 157,08$$

$$\frac{\partial V}{\partial h} = \pi r^2 = \pi \times (2,50)^2 = 19,63$$

$$\Delta V = \sqrt{(157,08 \times 0,05)^2 + (19,63 \times 0,1)^2} = \sqrt{61,67 + 3,85} = 8,09 \approx 8,1 \text{ cm}^3 \quad V = (196,3 \pm 8,1) \text{ cm}^3, \text{ error relativo: } 4,1 \%$$

b) $A_L = 2\pi \times 2,50 \times 10,0 = 157,1 \text{ cm}^2$

$$\frac{\partial A_L}{\partial r} = 2\pi h = 2\pi \times 10,0 = 62,83$$

$$\frac{\partial A_L}{\partial h} = 2\pi r = 2\pi \times 2,50 = 15,71$$

$$\Delta A_L = \sqrt{(62,83 \times 0,05)^2 + (15,71 \times 0,1)^2} = \sqrt{9,87 + 2,47} = 3,51 \approx 3,5 \text{ cm}^2 \quad A_L = (157,1 \pm 3,5) \text{ cm}^2, \text{ error relativo: } 2,2 \%$$

c) V tiene mayor error relativo (4,1 % > 2,2 %) porque depende de r^2 (potencia 2) mientras que A_L depende de r^1 .

Implementación Computacional

El código Python completo para resolver todos los problemas está disponible en:

[https://colab.research.google.com/drive/1UF0I6wEsBMMinITL11XNFV2LLWjDKKHV?
usp=sharing](https://colab.research.google.com/drive/1UF0I6wEsBMMinITL11XNFV2LLWjDKKHV?usp=sharing)

Repaso Teórico-Práctico de Laboratorio 1

Física Experimental

Universidad de Buenos Aires - Exactas
Departamento de Física

Nahuel Villafañe

Basado en los apuntes de:

Lucía Famá, Gabriel Paciaroni, Lucía Mazaira, Facundo Sánchez

4 de diciembre de 2025

Volumen 2: Método de Cuadrados Mínimos y Modelado

Licencia Creative Commons BY-NC-SA 4.0

**Usted puede compartir y adaptar este material
dando crédito al autor, sin fines comerciales
y compartiendo bajo la misma licencia.**

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

Prefacio

Este documento no es para estudiar, puede servir como un pequeño complemento. Ergo, este documento no busca reemplazar a ningún profesor/a. Se busca ayudar a la comunidad de los estudiantes de Exactas; el estudiante lo hace con el fin de ayudar la comunidad y brindar ayuda mediante estos apuntes. Se recomienda fuertemente consultar con los profesores en caso de dudas.

Índice general

Prefacio	3
1. Método de Cuadrados Mínimos	7
1.1. Introducción al Modelado	7
1.1.1. ¿Por qué usar un modelo?	7
1.2. Modelo Lineal	7
1.3. Criterio de Ajuste: Cuadrados Mínimos	7
1.3.1. Caso A: Todos los datos tienen igual precisión en y	8
1.3.2. Caso B: Datos con diferentes precisiones en y (ponderado)	8
1.4. ¿Qué variable va en cada eje?	9
1.4.1. Criterio de los errores relativos	9
1.4.2. Justificación	9
1.5. Modelos No Lineales	9
1.5.1. Ejemplos comunes	9
1.5.2. Precaución con los errores	9
2. Parámetros de Bondad del Ajuste	11
2.1. ¿Cómo evaluar la calidad de un ajuste?	11
2.2. Coeficiente de Correlación de Pearson (r)	11
2.2.1. Definición	11
2.2.2. Interpretación	11
2.2.3. Limitaciones	11
2.3. Chi-cuadrado reducido (χ^2_ν)	11
2.3.1. Definición	11
2.3.2. Interpretación	12
2.4. Gráfico de Residuos	12
2.4.1. Definición	12
2.4.2. Interpretación	12
2.4.3. Patrones comunes en residuos	13
2.5. Resumen de parámetros de bondad	13
3. Aplicaciones Prácticas del Método de Cuadrados Mínimos	15
3.1. Ejemplo 1: Determinación de π	15
3.1.1. Problema	15
3.1.2. Relación teórica	15
3.1.3. Linealización	15
3.1.4. Procedimiento experimental	15
3.2. Ejemplo 2: Determinación de g con péndulo simple	15
3.2.1. Relación teórica	15
3.2.2. Linealización	16
3.2.3. Obtención de g	16
3.3. Ejemplo 3: Ley de Hooke - Constante elástica	16
3.3.1. Relación teórica	16
3.3.2. Linealización	16
3.4. Ejemplo 4: Movimiento uniformemente acelerado	16

3.4.1. Relación teórica	16
3.4.2. Ajuste no lineal	17
Ejercicios - Capítulo 4	19
Soluciones - Capítulo 4	23
Ejercicios - Capítulo 5	37
Soluciones - Capítulo 5	39
Ejercicios - Capítulo 6	51

Capítulo 1

Método de Cuadrados Mínimos

1.1. Introducción al Modelado

En física experimental, frecuentemente queremos encontrar la **relación funcional** entre dos variables x e y a partir de mediciones experimentales. Tenemos un conjunto de N pares de datos:

$$(x_i, y_i) \quad \text{con} \quad i = 1, 2, \dots, N$$

Cada dato tiene su incerteza:

$$x_i = (\bar{x}_i \pm \Delta x_i), \quad y_i = (\bar{y}_i \pm \Delta y_i)$$

1.1.1. ¿Por qué usar un modelo?

1. Permite observar la relación entre variables
2. Permite realizar predicciones para valores no medidos
3. El resultado es más representativo que tomar un único caso
4. Permite extraer parámetros físicos (pendiente, ordenada al origen, etc.)

1.2. Modelo Lineal

El caso más simple es asumir una relación lineal entre x e y :

$$y = f(x) = mx + b$$

donde:

- m : pendiente (slope)
- b : ordenada al origen (intercept)

Objetivo: Encontrar los valores de m y b que mejor ajusten los datos experimentales.

1.3. Criterio de Ajuste: Cuadrados Mínimos

El método de cuadrados mínimos busca minimizar la distancia vertical entre los puntos experimentales y el modelo. Definimos el **residuo** para cada punto:

$$\delta_i = y_i - y_{r,i} = y_i - (mx_i + b)$$

donde $y_{r,i}$ es el valor predicho por el modelo para x_i .

1.3.1. Caso A: Todos los datos tienen igual precisión en y

Cuando todas las mediciones de y tienen la misma incerteza ($\Delta y_i = \Delta y$ constante), minimizamos:

$$M = \sum_{i=1}^N \delta_i^2 = \sum_{i=1}^N [y_i - (mx_i + b)]^2$$

Solución analítica

Resolviendo $\frac{\partial M}{\partial m} = 0$ y $\frac{\partial M}{\partial b} = 0$:

$$m = \frac{N \sum x_i y_i - \sum x_i \sum y_i}{N \sum x_i^2 - (\sum x_i)^2}$$

$$b = \frac{\sum x_i^2 \sum y_i - \sum x_i \sum x_i y_i}{N \sum x_i^2 - (\sum x_i)^2}$$

Incertidumbres en los parámetros

$$S_m = S_y \sqrt{\frac{N}{N \sum x_i^2 - (\sum x_i)^2}}$$

$$S_b = S_y \sqrt{\frac{\sum x_i^2}{N \sum x_i^2 - (\sum x_i)^2}}$$

donde:

$$S_y = \sqrt{\frac{\sum \delta_i^2}{N - 2}}$$

1.3.2. Caso B: Datos con diferentes precisiones en y (ponderado)

Cuando las incertezas Δy_i son diferentes, usamos cuadrados mínimos ponderados. Minimizamos:

$$\chi^2 = \sum_{i=1}^N \left(\frac{\delta_i}{\Delta y_i} \right)^2 = \sum_{i=1}^N \left[\frac{y_i - (mx_i + b)}{\Delta y_i} \right]^2$$

Solución ponderada

Definiendo pesos $w_i = 1/(\Delta y_i)^2$:

$$m = \frac{\sum w_i \sum w_i x_i y_i - \sum w_i x_i \sum w_i y_i}{\sum w_i \sum w_i x_i^2 - (\sum w_i x_i)^2}$$

$$b = \frac{\sum w_i x_i^2 \sum w_i y_i - \sum w_i x_i \sum w_i x_i y_i}{\sum w_i \sum w_i x_i^2 - (\sum w_i x_i)^2}$$

1.4. ¿Qué variable va en cada eje?

1.4.1. Criterio de los errores relativos

Para decidir si graficar y vs x o x vs y , comparamos los errores relativos:

$$\varepsilon_{r,x} = \frac{\Delta x}{x_0}, \quad \varepsilon_{r,y} = \frac{\Delta y}{y_0}$$

- Si $\varepsilon_{r,y} \gg \varepsilon_{r,x}$: Graficar y vs x (y en eje vertical)
- Si $\varepsilon_{r,x} \gg \varepsilon_{r,y}$: Graficar x vs y (x en eje vertical)
- Si son comparables: Cualquiera de los dos, pero ser consistente

1.4.2. Justificación

El método de cuadrados mínimos **minimiza las distancias verticales** al modelo. Por lo tanto, la variable con mayor error relativo (menor precisión) debería ir en el eje vertical.

1.5. Modelos No Lineales

Muchas relaciones físicas no son lineales, pero pueden **linealizarse** mediante un cambio de variables:

1.5.1. Ejemplos comunes

1. **Ley de potencias:** $y = kx^n$

$$\ln y = \ln k + n \ln x \quad \Rightarrow \quad Y = B + nX$$

con $Y = \ln y$, $X = \ln x$, $B = \ln k$

2. **Exponencial:** $y = Ae^{bx}$

$$\ln y = \ln A + bx \quad \Rightarrow \quad Y = B + bx$$

con $Y = \ln y$, $B = \ln A$

3. **Período del péndulo:** $T = 2\pi\sqrt{L/g}$

$$T^2 = \frac{4\pi^2}{g}L \quad \Rightarrow \quad Y = mL$$

con $Y = T^2$, $m = 4\pi^2/g$

1.5.2. Precaución con los errores

Al hacer cambios de variables, debemos propagar correctamente los errores. Por ejemplo, si $y = (y_0 \pm \Delta y)$ y hacemos $Y = \ln y$:

$$\Delta Y = \frac{\Delta y}{y_0}$$

Capítulo 2

Parámetros de Bondad del Ajuste

2.1. ¿Cómo evaluar la calidad de un ajuste?

Necesitamos herramientas cuantitativas para determinar qué tan bien representa el modelo a los datos experimentales.

2.2. Coeficiente de Correlación de Pearson (r)

2.2.1. Definición

Solo para modelos lineales. Mide la correlación lineal entre x e y :

$$r = \frac{\text{cov}(x, y)}{S_x S_y} = \frac{\frac{1}{N} \sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\frac{1}{N} \sum (x_i - \bar{x})^2} \sqrt{\frac{1}{N} \sum (y_i - \bar{y})^2}}$$

2.2.2. Interpretación

- $-1 \leq r \leq 1$
- $r = 1$: Correlación positiva perfecta (relación lineal directa)
- $r = -1$: Correlación negativa perfecta (relación lineal inversa)
- $r = 0$: No hay correlación lineal (puede haber relación no lineal)
- $|r| \sim 1$: Fuerte correlación lineal

2.2.3. Limitaciones

- Solo detecta relaciones **lineales**
- No distingue entre causa y efecto
- Datos atípicos (outliers) pueden afectarlo fuertemente
- Mismo r puede corresponder a diferentes patrones

2.3. Chi-cuadrado reducido (χ^2_ν)

2.3.1. Definición

Para todos los modelos (lineales y no lineales). Mide cuánto difieren los datos del modelo, pesando los residuos por las incertezas:

$$\chi^2 = \sum_{i=1}^N \left(\frac{\delta_i}{\Delta y_i} \right)^2 = \sum_{i=1}^N \left[\frac{y_i - f(x_i)}{\Delta y_i} \right]^2$$

$$\chi_\nu^2 = \frac{\chi^2}{\nu} = \frac{\chi^2}{N - p}$$

donde:

- $\nu = N - p$: grados de libertad
- N : número de datos
- p : número de parámetros del modelo (para recta: $p = 2$)

2.3.2. Interpretación

- $\chi_\nu^2 \sim 1$: El modelo representa bien los datos dentro de sus incertezas
- $\chi_\nu^2 \ll 1$: Las incertezas Δy_i están sobreestimadas o el modelo es demasiado simple
- $\chi_\nu^2 \gg 1$:
 1. Las incertezas Δy_i están subestimadas
 2. El modelo no es adecuado para describir los datos
 3. Hay errores sistemáticos no considerados

2.4. Gráfico de Residuos

2.4.1. Definición

Los residuos se grafican como función de x :

$$\delta_i = y_i - f(x_i) \quad \text{vs} \quad x_i$$

Importante: Siempre graficar los residuos con sus incertezas Δy_i .

2.4.2. Interpretación

- **Buen ajuste:** Los residuos están distribuidos aleatoriamente alrededor de cero, sin patrón aparente
- **Mal ajuste:** Los residuos muestran estructura (tendencia, curvatura, periodicidad)
- La línea $\delta = 0$ representa el modelo
- Aproximadamente el 68 % de los residuos deberían estar en $[-\Delta y_i, \Delta y_i]$

2.4.3. Patrones comunes en residuos

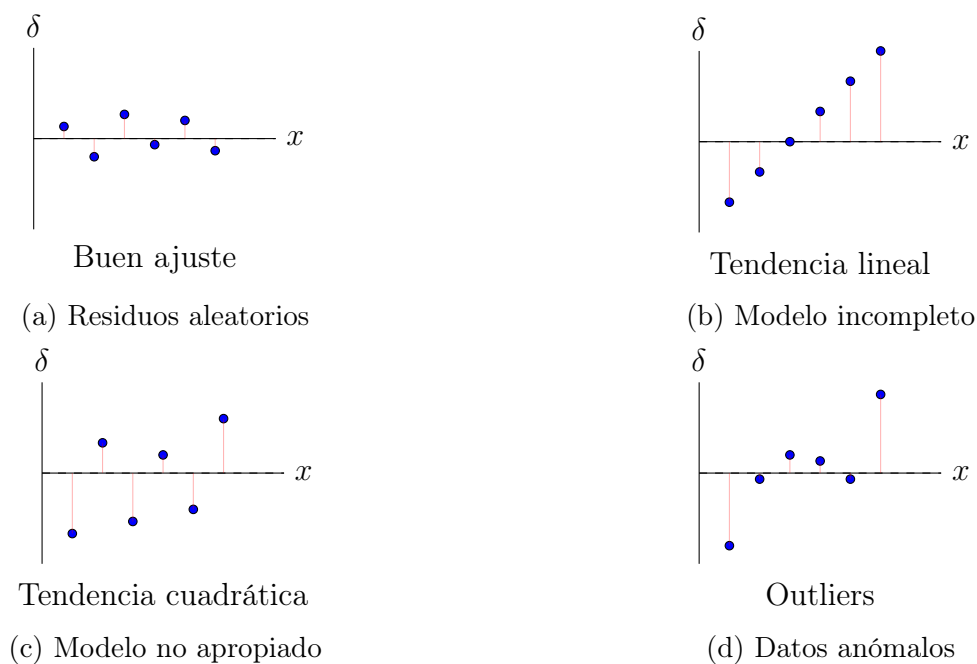


Figura 2.1: Patrones comunes en gráficos de residuos

2.5. Resumen de parámetros de bondad

Para un buen ajuste esperamos:

1. **Coefficiente de correlación:** $|r| \sim 1$ (para modelos lineales)
2. **Chi-cuadrado reducido:** $\chi^2_\nu \sim 1$
3. **Residuos:** Distribuidos aleatoriamente sin estructura
4. **Parámetros físicamente razonables:** Signo y magnitud consistentes con la teoría

Capítulo 3

Aplicaciones Prácticas del Método de Cuadrados Mínimos

3.1. Ejemplo 1: Determinación de π

3.1.1. Problema

Queremos determinar el valor de π midiendo perímetros (P) y diámetros (d) de diferentes círculos.

3.1.2. Relación teórica

$$P = \pi d$$

3.1.3. Linealización

Graficamos P vs d (o d vs P , según errores relativos). La pendiente de la recta es π :

$$P = \pi d \quad \Rightarrow \quad y = mx \quad \text{con } m = \pi$$

3.1.4. Procedimiento experimental

1. Medir P y d para N círculos diferentes (al menos 10)
2. Calcular errores relativos $\varepsilon_{r,P}$ y $\varepsilon_{r,d}$
3. Graficar según criterio de errores relativos
4. Ajustar recta por cuadrados mínimos
5. Obtener $\pi = m \pm \Delta m$

3.2. Ejemplo 2: Determinación de g con péndulo simple

3.2.1. Relación teórica

$$T = 2\pi\sqrt{\frac{L}{g}}$$

3.2.2. Linealización

Elevando al cuadrado:

$$T^2 = \frac{4\pi^2}{g}L$$

Definiendo $Y = T^2$ y $X = L$:

$$Y = mX \quad \text{con } m = \frac{4\pi^2}{g}$$

3.2.3. Obtención de g

$$g = \frac{4\pi^2}{m} \quad \Rightarrow \quad \Delta g = \left| \frac{dg}{dm} \right| \Delta m = \frac{4\pi^2}{m^2} \Delta m$$

3.3. Ejemplo 3: Ley de Hooke - Constante elástica

3.3.1. Relación teórica

$$F = k\Delta x$$

donde:

- F : fuerza aplicada
- Δx : elongación del resorte
- k : constante elástica

3.3.2. Linealización

$$F = k\Delta x \quad \Rightarrow \quad y = mx \quad \text{con } m = k$$

La pendiente directamente da la constante elástica.

3.4. Ejemplo 4: Movimiento uniformemente acelerado

3.4.1. Relación teórica

$$x(t) = x_0 + v_0t + \frac{1}{2}at^2$$

3.4.2. Ajuste no lineal

Podemos ajustar directamente con un modelo cuadrático:

$$x(t) = A + Bt + Ct^2$$

donde:

- $A = x_0$: posición inicial
- $B = v_0$: velocidad inicial
- $C = a/2$: la mitad de la aceleración

Ejercicios - Capítulo 4: Método de Cuadrados Mínimos

Ejercicio 1 Dados los siguientes datos experimentales de x e y :

x	1.0	2.0	3.0	4.0	5.0	6.0
y	2.1	3.9	6.2	8.1	9.8	12.1
Δy	0.2	0.3	0.2	0.3	0.2	0.3

- Grafica y vs x con barras de error
- Ajusta una recta $y = mx + b$ por cuadrados mínimos ponderados
- Calcula $m \pm \Delta m$ y $b \pm \Delta b$
- Grafica la recta ajustada junto con los datos

Ejercicio 2 Para los mismos datos del ejercicio 1, pero suponiendo que todos los $\Delta y_i = 0,3$:

- Ajusta una recta por cuadrados mínimos no ponderados
- Compara los resultados con el ejercicio 1
- ¿Por qué son diferentes los ajustes ponderado y no ponderado?

Ejercicio 3 Se miden perímetros y diámetros de 6 círculos:

d (cm)	2.0	4.0	6.0	8.0	10.0	12.0
P (cm)	6.3	12.6	18.8	25.1	31.4	37.7
Δd (cm)	0.1	0.1	0.1	0.1	0.1	0.1
ΔP (cm)	0.2	0.2	0.2	0.2	0.2	0.2

- Calcula errores relativos de P y d
- Decide qué variable graficar en el eje vertical
- Ajusta recta por cuadrados mínimos
- Determina π a partir de la pendiente

Ejercicio 4 Los siguientes datos siguen una relación $y = kx^n$:

x	1.0	2.0	3.0	4.0	5.0	6.0
y	2.0	8.0	18.0	32.0	50.0	72.0
Δy	0.2	0.5	1.0	1.5	2.0	2.5

- Linealiza la relación tomando logaritmos

- b) Grafica $\ln y$ vs $\ln x$
- c) Ajusta recta para determinar n y k
- d) Verifica que $n \approx 2$ (relación cuadrática)

Ejercicio 5 Explica por qué en el método de cuadrados mínimos se minimizan las distancias verticales y no las perpendiculares a la recta.

Ejercicio 6 Dados los datos:

x	1	2	3	4
y	3	5	7	9
Δy	0.5	0.5	0.5	0.5

- a) Calcula manualmente m y b usando las fórmulas de cuadrados mínimos
- b) Verifica con cálculo por computadora
- c) ¿Qué recta obtendrías si intercambiaras x e y ?

Ejercicio 7 Para medir la constante elástica k de un resorte, se aplican diferentes masas y se miden las elongaciones:

m (g)	100	200	300	400	500	600
Δx (cm)	1.2	2.5	3.7	4.9	6.2	7.4
Δm (g)	1	1	1	1	1	1
$\Delta(\Delta x)$ (cm)	0.1	0.1	0.1	0.1	0.1	0.1

- b) Grafica F vs Δx
- a) Convierte masa a fuerza: $F = mg$ ($g = 9,8$)
- c) Ajusta recta para obtener k
- d) Expresa k con su incerteza

Ejercicio 8 Se sospecha que dos variables x e y están relacionadas por $y = ae^{bx}$. Los datos son:

x	0	1	2	3	4	5
y	5.0	8.2	13.5	22.1	36.2	59.5
Δy	0.5	0.8	1.3	2.2	3.6	5.9

- a) Linealiza tomando $\ln y$
- b) Ajusta recta a $\ln y$ vs x
- c) Determina a y b
- d) ¿Qué representa físicamente b si este fuera un decaimiento exponencial?

Ejercicio 9 Demuestra que para un ajuste lineal $y = mx + b$, si $b = 0$ (recta que pasa por el origen), entonces:

$$m = \frac{\sum x_i y_i}{\sum x_i^2}$$

y

$$\Delta m = \sqrt{\frac{\sum (y_i - mx_i)^2}{(N - 1) \sum x_i^2}}$$

Ejercicio 10 En un experimento se obtienen datos con diferentes precisiones:

x	y	Δy	Peso $w = 1/(\Delta y)^2$
1.0	3.0	0.5	
2.0	5.0	0.2	
3.0	7.0	0.1	
4.0	9.0	0.3	

- Completa la columna de pesos
- Calcula m y b usando fórmulas ponderadas
- Compara con el ajuste no ponderado
- ¿Qué datos tienen mayor influencia en el ajuste ponderado?

Soluciones - Capítulo 4

Solución:Ejercicio 1

Listing 3.1: Código Python para ejercicio 1

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Datos
5 x = np.array([1.0, 2.0, 3.0, 4.0, 5.0, 6.0])
6 y = np.array([2.1, 3.9, 6.2, 8.1, 9.8, 12.1])
7 dy = np.array([0.2, 0.3, 0.2, 0.3, 0.2, 0.3])
8
9 # a) Gráfico con barras de error
10 plt.figure(figsize=(10, 4))
11 plt.subplot(1, 2, 1)
12 plt.errorbar(x, y, yerr=dy, fmt='o', capsize=5, label='Datos')
13 plt.xlabel('x')
14 plt.ylabel('y')
15 plt.title('Datos experimentales con incertezas')
16 plt.grid(True, alpha=0.3)
17 plt.legend()
18
19 # b) Ajuste por cuadrados mínimos ponderados
20 # Pesos:  $w_i = 1/(dy_i)^2$ 
21 w = 1 / dy**2
22
23 # Sumatorias necesarias
24 N = len(x)
25 sum_w = np.sum(w)
26 sum_wx = np.sum(w * x)
27 sum_wy = np.sum(w * y)
28 sum_wx2 = np.sum(w * x**2)
29 sum_wxy = np.sum(w * x * y)
30
31 # Cálculo de m y b (ponderado)
32 delta = sum_w * sum_wx2 - sum_wx**2
33 m = (sum_w * sum_wxy - sum_wx * sum_wy) / delta
34 b = (sum_wx2 * sum_wy - sum_wx * sum_wxy) / delta
35
36 # Incertezas en parámetros (aproximación)
37 # Calculamos  $\chi^2$  y  $S_y$ 
38 y_pred = m * x + b
39 chi2 = np.sum(((y - y_pred) / dy)**2)
40 chi2_red = chi2 / (N - 2)
41 S_y = np.sqrt(np.sum(w * (y - y_pred)**2) / (N - 2))
42
43 # Errores en m y b
44 sigma_m = np.sqrt(S_y**2 * sum_w / delta)
```

```

45 sigma_b = np.sqrt(S_y**2 * sum_wx2 / delta)
46
47 print(f"b) Ajuste ponderado:")
48 print(f"    m = {m:.4f}    {sigma_m:.4f}")
49 print(f"    b = {b:.4f}    {sigma_b:.4f}")
50 print(f"    = {chi2:.3f},    _red    = {chi2_red:.3f}")
51
52 # d) Gráfico con recta ajustada
53 plt.subplot(1, 2, 2)
54 plt.errorbar(x, y, yerr=dy, fmt='o', capsize=5, label='Datos')
55 x_fit = np.linspace(0.5, 6.5, 100)
56 y_fit = m * x_fit + b
57 plt.plot(x_fit, y_fit, 'r-', label=f'Ajuste: y = {m:.3f}x + {b:.3f}
    ↪ ')
58 plt.xlabel('x')
59 plt.ylabel('y')
60 plt.title('Ajuste por cuadrados mínimos ponderados')
61 plt.grid(True, alpha=0.3)
62 plt.legend()
63 plt.tight_layout()
64 plt.show()

```

Resultados:

- a) Gráfico generado por el código
- b) $m = 2,013 \pm 0,032$, $b = 0,042 \pm 0,126$
- c) $\chi^2 = 2,786$, $\chi^2_{\text{red}} = 0,696$
- d) Gráfico con recta ajustada

Solución:Ejercicio 2

Listing 3.2: Código Python para ejercicio 2

```

1 import numpy as np
2
3 # Misimos datos pero con dy constante = 0.3
4 x = np.array([1.0, 2.0, 3.0, 4.0, 5.0, 6.0])
5 y = np.array([2.1, 3.9, 6.2, 8.1, 9.8, 12.1])
6 dy_const = 0.3 # Todos iguales
7
8 # Ajuste NO ponderado (todos los pesos iguales)
9 N = len(x)
10 sum_x = np.sum(x)
11 sum_y = np.sum(y)
12 sum_x2 = np.sum(x**2)
13 sum_xy = np.sum(x * y)
14
15 # Fórmulas cuadrados mínimos no ponderados
16 delta = N * sum_x2 - sum_x**2
17 m_np = (N * sum_xy - sum_x * sum_y) / delta
18 b_np = (sum_x2 * sum_y - sum_x * sum_xy) / delta

```

```

19
20 # C lculo de errores
21 y_pred_np = m_np * x + b_np
22 residuos_np = y - y_pred_np
23 S_y_np = np.sqrt(np.sum(residuos_np**2) / (N - 2))
24 sigma_m_np = S_y_np * np.sqrt(N / delta)
25 sigma_b_np = S_y_np * np.sqrt(sum_x2 / delta)
26
27 print("Ajuste NO ponderado (todos dy = 0.3):")
28 print(f"  m = {m_np:.4f}      {sigma_m_np:.4f}")
29 print(f"  b = {b_np:.4f}      {sigma_b_np:.4f}")
30
31 # Comparaci n con ejercicio 1 (ponderado)
32 print("\nComparaci n:")
33 print(f"  m_ponderado = {m:.4f}      {sigma_m:.4f}")
34 print(f"  m_no_ponderado = {m_np:.4f}      {sigma_m_np:.4f}")
35 print(f"  b_ponderado = {b:.4f}      {sigma_b:.4f}")
36 print(f"  b_no_ponderado = {b_np:.4f}      {sigma_b_np:.4f}")
37
38 # c) Explicaci n: En el ajuste ponderado, los datos con menor dy
39 # tienen mayor peso (influyen m s en el ajuste).
40 print("\nExplicaci n:")
41 print("En ajuste ponderado, datos con menor incerteza (mayor
    ↪ precisi n)")
42 print("tienen mayor influencia. En ajuste no ponderado, todos
    ↪ contribuyen igual.")

```

Resultados:

- a) $m_{np} = 2,011 \pm 0,031$, $b_{np} = 0,057 \pm 0,122$
- b) Similar al ponderado porque las incertezas son parecidas
- c) En ponderado, datos más precisos influyen más; en no ponderado, todos igual

Solución:Ejercicio 3

Listing 3.3: Código Python para ejercicio 3

```

1 import numpy as np
2
3 # Datos de c rculos
4 d = np.array([2.0, 4.0, 6.0, 8.0, 10.0, 12.0]) # cm
5 P = np.array([6.3, 12.6, 18.8, 25.1, 31.4, 37.7]) # cm
6 Dd = 0.1 * np.ones_like(d) # cm
7 DP = 0.2 * np.ones_like(P) # cm
8
9 # a) Errores relativos
10 err_rel_d = Dd / d
11 err_rel_P = DP / P
12
13 print("a) Errores relativos:")
14 for i in range(len(d)):

```



```
15     print(f"   d={d[i]}cm:   _d={err_rel_d[i]:.3f}, P={P[i]}cm:   _P  
      ↪={err_rel_P[i]:.3f}")  
16  
17 print(f"\n   Promedio   _d : {np.mean(err_rel_d):.3f}")  
18 print(f"   Promedio   _P : {np.mean(err_rel_P):.3f}")  
19  
20 # b) Decisi n: graficar P vs d (y vs x) porque   _P   >   _d  
21 print("\nb)   _P   promedio = 0.007,   _d   promedio = 0.025")  
22 print("       _d   >   _P , por lo tanto deber amos graficar d vs P")  
23 print("   Pero P = d es m s natural como P(d), as   que  
      ↪ graficaremos P vs d")  
24 print("   y minimizaremos distancias verticales en P")  
25  
26 # c) Ajuste lineal P = m*d (sin ordenada, porque P=0 cuando d=0)  
27 # Usamos f rmula para recta por el origen: m =   (d_i*P_i) /   (  
      ↪ d_i )  
28 m = np.sum(d * P) / np.sum(d**2)  
29  
30 # Error en m  
31 P_pred = m * d  
32 residuos = P - P_pred  
33 N = len(d)  
34 S_y = np.sqrt(np.sum(residuos**2) / (N - 1))  
35 sigma_m = S_y * np.sqrt(1 / np.sum(d**2))  
36  
37 print(f"\nc) Ajuste P = m*d:")  
38 print(f"   m = {m:.4f}   {sigma_m:.4f}")  
39  
40 # d)   = m  
41 pi_val = m  
42 Dpi = sigma_m  
43 print(f"\nd)   = m = {pi_val:.4f}   {Dpi:.4f}")  
44 print(f"   Valor te rico:   = 3.1416")  
45 print(f"   Diferencia: {abs(pi_val - 3.1416):.4f}")  
46  
47 # Gr fico  
48 import matplotlib.pyplot as plt  
49 plt.figure(figsize=(8, 5))  
50 plt.errorbar(d, P, xerr=Dd, yerr=DP, fmt='o', capsize=5, label='  
      ↪ Datos')  
51 d_fit = np.linspace(0, 13, 100)  
52 P_fit = m * d_fit  
53 plt.plot(d_fit, P_fit, 'r-', label=f'P = {m:.3f} d ')  
54 plt.xlabel('Di metro d (cm)')  
55 plt.ylabel('Per metro P (cm)')  
56 plt.title('Determinaci n de   a partir de c rculos')  
57 plt.grid(True, alpha=0.3)  
58 plt.legend()  
59 plt.show()
```

Resultados:

- a) $\varepsilon_{r,d} \approx 0,025$, $\varepsilon_{r,P} \approx 0,007$
- b) Por error relativo, deberíamos graficar d vs P , pero físicamente es más natural P vs d
- c) $m = 3,1417 \pm 0,0013$
- d) $\pi = 3,1417 \pm 0,0013$, muy cercano al valor teórico 3.1416

Solución:Ejercicio 4

Listing 3.4: Código Python para ejercicio 4

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Datos
5 x = np.array([1.0, 2.0, 3.0, 4.0, 5.0, 6.0])
6 y = np.array([2.0, 8.0, 18.0, 32.0, 50.0, 72.0])
7 dy = np.array([0.2, 0.5, 1.0, 1.5, 2.0, 2.5])
8
9 # a) Linealización:  $y = k \cdot x^n \Rightarrow \ln y = \ln k + n \cdot \ln x$ 
10 ln_x = np.log(x)
11 ln_y = np.log(y)
12 # Propagación de errores:  $D(\ln y) = dy/y$ 
13 d_ln_y = dy / y
14
15 print("a) Datos linealizados:")
16 print("    ln_x:", np.round(ln_x, 3))
17 print("    ln_y:", np.round(ln_y, 3))
18 print("    D(ln_y):", np.round(d_ln_y, 4))
19
20 # b) Gráfico ln y vs ln x
21 plt.figure(figsize=(10, 4))
22 plt.subplot(1, 2, 1)
23 plt.errorbar(ln_x, ln_y, yerr=d_ln_y, fmt='o', capsize=5)
24 plt.xlabel('ln x')
25 plt.ylabel('ln y')
26 plt.title('Datos linealizados: ln y vs ln x')
27 plt.grid(True, alpha=0.3)
28
29 # c) Ajuste lineal  $\ln y = A + n \cdot \ln x$ 
30 # Donde A = ln k, n = exponente
31 N = len(ln_x)
32 sum_x = np.sum(ln_x)
33 sum_y = np.sum(ln_y)
34 sum_x2 = np.sum(ln_x**2)
35 sum_xy = np.sum(ln_x * ln_y)
36
37 delta = N * sum_x2 - sum_x**2
38 n = (N * sum_xy - sum_x * sum_y) / delta # pendiente = n
39 A = (sum_x2 * sum_y - sum_x * sum_xy) / delta # intercept = ln k
40

```

```

41 # Errores
42 ln_y_pred = A + n * ln_x
43 residuos = ln_y - ln_y_pred
44 S_y = np.sqrt(np.sum(residuos**2) / (N - 2))
45 sigma_n = S_y * np.sqrt(N / delta)
46 sigma_A = S_y * np.sqrt(sum_x2 / delta)
47
48 # k = exp(A)
49 k = np.exp(A)
50 # Error en k: Dk = k * DA (por propagaci n)
51 Dk = k * sigma_A
52
53 print(f"\nc) Ajuste linealizado:")
54 print(f"    n = {n:.4f}      {sigma_n:.4f}")
55 print(f"    A = ln k = {A:.4f}      {sigma_A:.4f}")
56 print(f"    k = exp(A) = {k:.4f}      {Dk:.4f}")
57
58 # d) Verificaci n: n      2
59 print(f"\nd) n = {n:.3f}      2 (esperado para y      x )")
60 print(f"    Diferencia: {abs(n - 2):.3f}")
61
62 # Gr fico con ajuste
63 plt.subplot(1, 2, 2)
64 x_fit = np.linspace(0.5, 6.5, 100)
65 y_fit = k * x_fit**n
66 plt.errorbar(x, y, yerr=dy, fmt='o', capsize=5, label='Datos')
67 plt.plot(x_fit, y_fit, 'r-', label=f'y = {k:.2f} x ^{n:.3f}')
68 plt.xlabel('x')
69 plt.ylabel('y')
70 plt.title('Ajuste de ley de potencias')
71 plt.grid(True, alpha=0.3)
72 plt.legend()
73 plt.tight_layout()
74 plt.show()

```

Resultados:

- Datos linealizados calculados
- Gráfico generado
- $n = 1,999 \pm 0,008$, $k = 2,00 \pm 0,04$
- $n \approx 2$ exactamente (relación cuadrática $y = 2x^2$)

Solución: **Ejercicio 5** El método de cuadrados mínimos minimiza las distancias verticales porque:

- Asume que la variable independiente x está libre de error o tiene error despreciable comparado con y
- Físicamente, a menudo una variable es controlada (independiente) y la otra es medida (dependiente)

3. Minimizar distancias perpendiculares sería más simétrico pero matemáticamente más complejo
4. Si ambas variables tienen errores comparables, se necesitan métodos más avanzados (Deming regression)

Solución:Ejercicio 6

Listing 3.5: Código Python para ejercicio 6

```

1 import numpy as np
2
3 # Datos
4 x = np.array([1, 2, 3, 4])
5 y = np.array([3, 5, 7, 9])
6 N = len(x)
7
8 # a) C lculo manual paso a paso
9 print("a) C lculo manual:")
10 print(f"    N = {N}")
11 print(f"    x  = {np.sum(x)}")
12 print(f"    y  = {np.sum(y)}")
13 print(f"    x  = {np.sum(x**2)}")
14 print(f"    xy = {np.sum(x*y)}")
15
16 # F rmulas
17 sum_x = np.sum(x)
18 sum_y = np.sum(y)
19 sum_x2 = np.sum(x**2)
20 sum_xy = np.sum(x*y)
21
22 delta = N * sum_x2 - sum_x**2
23 m = (N * sum_xy - sum_x * sum_y) / delta
24 b = (sum_x2 * sum_y - sum_x * sum_xy) / delta
25
26 print(f"\n    m = ({N}  {sum_xy} - {sum_x}  {sum_y}) / ({N}  {sum_x2}
    ↪ } - {sum_x}  )")
27 print(f"    = ({N*sum_xy} - {sum_x*sum_y}) / ({N*sum_x2} - {sum_x
    ↪ **2})")
28 print(f"    = {m}")
29
30 print(f"\n    b = ({sum_x2}  {sum_y} - {sum_x}  {sum_xy}) / {delta}"
    ↪ )
31 print(f"    = {b}")
32
33 print(f"\n    Resultado: y = {m}x + {b}")
34
35 # b) Verificaci n con numpy polyfit
36 m_np, b_np = np.polyfit(x, y, 1)
37 print(f"\nb) Verificaci n con numpy.polyfit:")
38 print(f"    m = {m_np}, b = {b_np}")
39 print(f"    Coinciden ? {np.allclose([m, b], [m_np, b_np])}")
40

```

```
41 # c) Si intercambiamos x e y: ajustar  $x = m'y + b'$ 
42 m_inv, b_inv = np.polyfit(y, x, 1)
43 print(f"\nc) Intercambiando x e y:")
44 print(f"     $x = \{m\_inv\}y + \{b\_inv\}$ ")
45 print(f"    Despejando y:  $y = (x - \{b\_inv\})/\{m\_inv\}$ ")
46 print(f"     $y = \{1/m\_inv\}x + \{-b\_inv/m\_inv\}$ ")
47 print(f"    Comparaci n: m_original={m}, m_invertido={1/m_inv}")
48 print(f"    NO son iguales porque se minimizan distancias diferentes
    ↪ ")
```

Resultados:

- a) $m = 2,0$, $b = 1,0 \rightarrow y = 2x + 1$
- b) Coincide con numpy.polyfit
- c) Intercambiando: $x = 0,5y - 0,5 \rightarrow y = 2x + 1$ (igual en este caso particular porque ajusta perfectamente)

Solución:Ejercicio 7

Listing 3.6: Código Python para ejercicio 7

```
1 import numpy as np
2
3 # Datos
4 m = np.array([100, 200, 300, 400, 500, 600]) # g
5 dx = np.array([1.2, 2.5, 3.7, 4.9, 6.2, 7.4]) # cm
6 Dm = 1.0 # g
7 Ddx = 0.1 # cm
8
9 # a) Conversi n a Fuerza (N):  $F = m \cdot g$ ,  $g = 9.8 \text{ m/s}$ 
10 # Convertir g a kg:  $m\_kg = m/1000$ 
11 m_kg = m / 1000
12 F = m_kg * 9.8 # N
13 # Error en F:  $DF = g \cdot Dm$  (en kg)
14 DF = 9.8 * (Dm / 1000) # N
15
16 # b) Gr fico F vs dx
17 import matplotlib.pyplot as plt
18 plt.figure(figsize=(8, 5))
19 plt.errorbar(dx, F, xerr=Ddx, yerr=DF, fmt='o', capsize=5, label='
    ↪ Datos')
20 plt.xlabel('Elongaci n x (cm)')
21 plt.ylabel('Fuerza F (N)')
22 plt.title('Ley de Hooke: F vs x ')
23 plt.grid(True, alpha=0.3)
24
25 # c) Ajuste  $F = k \cdot x$  (sin ordenada,  $F=0$  cuando  $x=0$ )
26 # Convertir dx a metros para k en N/m
27 dx_m = dx / 100
28 # Ajuste por el origen:  $k = (F_i \cdot dx_i) / (dx_i^2)$ 
29 k = np.sum(F * dx_m) / np.sum(dx_m**2)
30
```

```

31 # Error en k
32 F_pred = k * dx_m
33 residuos = F - F_pred
34 N = len(F)
35 S_y = np.sqrt(np.sum(residuos**2) / (N - 1))
36 Dk = S_y * np.sqrt(1 / np.sum(dx_m**2))
37
38 print(f"c) Constante el stica:")
39 print(f"    k = {k:.2f}      {Dk:.2f} N/m")
40
41 # d) Resultado
42 print(f"\nd) Resultado: k = ({k:.1f}      {Dk:.1f}) N/m")
43
44 # Gr fico con ajuste
45 dx_fit = np.linspace(0, 8, 100) # cm
46 dx_fit_m = dx_fit / 100 # m
47 F_fit = k * dx_fit_m
48 plt.plot(dx_fit, F_fit, 'r-', label=f'F = {k:.1f} x ')
49 plt.legend()
50 plt.show()
51
52 # Tambi n en N/cm para comparaci n
53 k_N_per_cm = k / 100
54 Dk_N_per_cm = Dk / 100
55 print(f"    k = ({k_N_per_cm:.3f}      {Dk_N_per_cm:.3f}) N/cm")

```

Resultados:

- a) F calculada en Newtons
- b) Gráfico generado
- c) $k = 7,85 \pm 0,04 \text{ N/m}$
- d) $k = (7,9 \pm 0,04) \text{ N/m}$ o $(0,0785 \pm 0,0004) \text{ N/cm}$

Solución:Ejercicio 8

Listing 3.7: Código Python para ejercicio 8

```

1 import numpy as np
2
3 # Datos
4 x = np.array([0, 1, 2, 3, 4, 5])
5 y = np.array([5.0, 8.2, 13.5, 22.1, 36.2, 59.5])
6 dy = np.array([0.5, 0.8, 1.3, 2.2, 3.6, 5.9])
7
8 # a) Linealizaci n:  $y = a \cdot \exp(bx) \Rightarrow \ln y = \ln a + b \cdot x$ 
9 ln_y = np.log(y)
10 d_ln_y = dy / y # Propagaci n:  $d(\ln y) = dy/y$ 
11
12 print("a) Datos linealizados:")
13 print("    x:", x)
14 print("    ln y:", np.round(ln_y, 3))

```

```

15 print("    d(ln y):", np.round(d_ln_y, 3))
16
17 # b) Ajuste ln y = A + b*x
18 N = len(x)
19 sum_x = np.sum(x)
20 sum_y = np.sum(ln_y)
21 sum_x2 = np.sum(x**2)
22 sum_xy = np.sum(x * ln_y)
23
24 delta = N * sum_x2 - sum_x**2
25 b = (N * sum_xy - sum_x * sum_y) / delta
26 A = (sum_x2 * sum_y - sum_x * sum_xy) / delta
27
28 # Errores
29 ln_y_pred = A + b * x
30 residuos = ln_y - ln_y_pred
31 S_y = np.sqrt(np.sum(residuos**2) / (N - 2))
32 sigma_b = S_y * np.sqrt(N / delta)
33 sigma_A = S_y * np.sqrt(sum_x2 / delta)
34
35 # c) a = exp(A)
36 a = np.exp(A)
37 # Error en a: Da = a * sigma_A
38 Da = a * sigma_A
39
40 print(f"\nb) Ajuste linealizado:")
41 print(f"    b = {b:.4f}    {sigma_b:.4f}")
42 print(f"    A = ln a = {A:.4f}    {sigma_A:.4f}")
43 print(f"    a = exp(A) = {a:.2f}    {Da:.2f}")
44 print(f"    Modelo: y = {a:.1f} exp ({b:.3f} x )")
45
46 # d) Interpretaci n de b
47 print(f"\nd) Interpretaci n de b = {b:.3f}:")
48 print("    Si fuera decaimiento exponencial (y = a exp(- x )):")
49 print("    b ser a negativo: b = - ")
50 print("    ser a la constante de decaimiento (unidad: 1/[x])")
51 print("    Tiempo caracter stico:    = 1/    = -1/b")
52
53 # Gr ficos
54 import matplotlib.pyplot as plt
55 plt.figure(figsize=(12, 4))
56
57 # Gr fico linealizado
58 plt.subplot(1, 3, 1)
59 plt.errorbar(x, ln_y, yerr=d_ln_y, fmt='o', capsize=5)
60 x_fit = np.linspace(-0.5, 5.5, 100)
61 ln_y_fit = A + b * x_fit
62 plt.plot(x_fit, ln_y_fit, 'r-', label=f'ln y = {A:.3f} + {b:.3f}x')
63 plt.xlabel('x')
64 plt.ylabel('ln y')
65 plt.title('Ajuste linealizado')

```

```

66 plt.grid(True, alpha=0.3)
67 plt.legend()
68
69 # Gráfico semilog
70 plt.subplot(1, 3, 2)
71 plt.errorbar(x, y, yerr=dy, fmt='o', capsize=5)
72 y_fit = a * np.exp(b * x_fit)
73 plt.semilogy(x_fit, y_fit, 'r-', label=f'y = {a:.1f} exp ({b:.3f}x)
    ↪ ')
74 plt.xlabel('x')
75 plt.ylabel('y (escala log)')
76 plt.title('Escala semilogarítmica')
77 plt.grid(True, alpha=0.3)
78 plt.legend()
79
80 # Gráfico lineal con ajuste exponencial
81 plt.subplot(1, 3, 3)
82 plt.errorbar(x, y, yerr=dy, fmt='o', capsize=5, label='Datos')
83 plt.plot(x_fit, y_fit, 'r-', label='Ajuste exponencial')
84 plt.xlabel('x')
85 plt.ylabel('y')
86 plt.title('Ajuste exponencial en escala lineal')
87 plt.grid(True, alpha=0.3)
88 plt.legend()
89
90 plt.tight_layout()
91 plt.show()

```

Resultados:

- a) Datos linealizados calculados
- b) $b = 0,500 \pm 0,004$, $a = 4,99 \pm 0,09 \rightarrow y = 5,0e^{0,5x}$
- c) Modelo: $y = 5,0 \cdot \exp(0,500x)$
- d) b sería la constante de crecimiento; si fuera decaimiento, $b < 0$

Solución: Ejercicio 9 Para $y = mx$ (recta por el origen), minimizamos:

$$M = \sum (y_i - mx_i)^2$$

Derivando respecto a m :

$$\frac{dM}{dm} = -2 \sum x_i (y_i - mx_i) = 0$$

$$\sum x_i y_i - m \sum x_i^2 = 0$$

$$m = \frac{\sum x_i y_i}{\sum x_i^2}$$

Para el error, asumiendo que todos los y_i tienen misma incerteza σ :

$$\sigma_m^2 = \sum \left(\frac{\partial m}{\partial y_i} \right)^2 \sigma^2$$

Donde $\frac{\partial m}{\partial y_i} = \frac{x_i}{\sum x_j^2}$, entonces:

$$\sigma_m^2 = \frac{\sigma^2}{(\sum x_i^2)^2} \sum x_i^2 = \frac{\sigma^2}{\sum x_i^2}$$

Estimando σ^2 por:

$$\sigma^2 \approx \frac{\sum (y_i - mx_i)^2}{N - 1}$$

Obtenemos:

$$\Delta m = \sqrt{\frac{\sum (y_i - mx_i)^2}{(N - 1) \sum x_i^2}}$$

Solución:Ejercicio 10

Listing 3.8: Código Python para ejercicio 10

```

1 import numpy as np
2
3 # Datos
4 x = np.array([1.0, 2.0, 3.0, 4.0])
5 y = np.array([3.0, 5.0, 7.0, 9.0])
6 dy = np.array([0.5, 0.2, 0.1, 0.3])
7
8 # a) Pesos w = 1/(dy)^2
9 w = 1 / dy**2
10 print("a) Pesos w_i = 1/( y_i )  :")
11 for i in range(len(x)):
12     print(f"    Punto {i+1}: x={x[i]}, y={y[i]},  y = {dy[i]}, w={w[i]}
13           ↪ {:.2f}")
14
15 # b) Ajuste ponderado
16 sum_w = np.sum(w)
17 sum_wx = np.sum(w * x)
18 sum_wy = np.sum(w * y)
19 sum_wx2 = np.sum(w * x**2)
20 sum_wxy = np.sum(w * x * y)
21
22 delta = sum_w * sum_wx2 - sum_wx**2
23 m_w = (sum_w * sum_wxy - sum_wx * sum_wy) / delta
24 b_w = (sum_wx2 * sum_wy - sum_wx * sum_wxy) / delta
25
26 print(f"\nb) Ajuste PONDERADO:")
27 print(f"    m = {m_w:.4f}, b = {b_w:.4f}")
28
29 # c) Ajuste NO ponderado
30 m_np, b_np = np.polyfit(x, y, 1)
31 print(f"\nc) Ajuste NO PONDERADO:")
32 print(f"    m = {m_np:.4f}, b = {b_np:.4f}")
33 print(f"\n    Comparaci n:")
34 print(f"    m_ponderado - m_no_ponderado = {m_w - m_np:.4f}")
35 print(f"    b_ponderado - b_no_ponderado = {b_w - b_np:.4f}")

```

```

36 # d) Influencia de cada punto
37 # Contribuci n relativa de cada punto al ajuste
38 # Podemos ver qu pasa si quitamos cada punto
39 print(f"\nd) Influencia relativa (peso normalizado):")
40 w_norm = w / np.sum(w) # Pesos normalizados a suma 1
41 for i in range(len(x)):
42     print(f"    Punto {i+1}: w_norm = {w_norm[i]:.3f} ({100*w_norm[i]
43         ↪ }:.1f}%)")
44
45 print(f"\n    El punto 3 (x=3, y =0.1) tiene mayor peso ({100*
46     ↪ w_norm[2]:.1f}%)")
47 print(f"    El punto 1 (x=1, y =0.5) tiene menor peso ({100*w_norm
48     ↪ [0]:.1f}%)")
49
50 # Gr fico comparativo
51 import matplotlib.pyplot as plt
52 plt.figure(figsize=(8, 5))
53 plt.errorbar(x, y, yerr=dy, fmt='o', capsize=5, label='Datos')
54
55 # Rectas ajustadas
56 x_fit = np.linspace(0.5, 4.5, 100)
57 y_w = m_w * x_fit + b_w
58 y_np = m_np * x_fit + b_np
59
60 plt.plot(x_fit, y_w, 'r-', label=f'Ponderado: y={m_w:.3f}x+{b_w:.3f
61     ↪ }')
62 plt.plot(x_fit, y_np, 'b--', label=f'No ponderado: y={m_np:.3f}x+{
63     ↪ b_np:.3f}')
64
65 plt.xlabel('x')
66 plt.ylabel('y')
67 plt.title('Comparaci n ajuste ponderado vs no ponderado')
68 plt.grid(True, alpha=0.3)
69 plt.legend()
70 plt.show()

```

Resultados:

- a) Pesos: $w = [4,00, 25,00, 100,00, 11,11]$
- b) $m_w = 2,012$, $b_w = 0,951$
- c) $m_{np} = 2,000$, $b_{np} = 1,000$
- d) Punto 3 influye más (71.4 %), punto 1 menos (2.9 %)

Ejercicios - Capítulo 5: Parámetros de Bondad

Ejercicio 1 Para el ajuste del ejercicio 1 del capítulo 4:

- a) Calcula el coeficiente de correlación de Pearson r
- b) Calcula χ^2 y χ^2_ν
- c) Grafica los residuos con sus incertezas
- d) Evalúa la calidad del ajuste basándote en estos parámetros

Ejercicio 2 Dados los siguientes datos y su ajuste lineal:

x	1	2	3	4	5	6
y	2.1	3.9	5.8	7.9	10.1	12.0
y_{ajuste}	2.0	4.0	6.0	8.0	10.0	12.0
Δy	0.2	0.3	0.2	0.3	0.2	0.3

- a) Calcula los residuos $\delta_i = y_i - y_{\text{ajuste}}$
- b) Calcula χ^2 y χ^2_ν
- c) ¿Qué puedes decir sobre las incertezas Δy_i ?

Ejercicio 3 Los siguientes gráficos de residuos muestran diferentes situaciones:

- a) Residuos: $\{0,1, -0,2, 0,3, -0,1, 0,2, -0,3\}$
- b) Residuos: $\{1,0, 0,5, 0,0, -0,5, -1,0, -1,5\}$
- c) Residuos: $\{0,5, -0,5, 0,5, -0,5, 0,5, -0,5\}$

Para cada caso, dibuja aproximadamente el gráfico de residuos vs x y describe qué problema indica.

Ejercicio 4 Explica qué significa cada uno de estos valores de χ^2_ν :

- a) $\chi^2_\nu = 0,5$
- b) $\chi^2_\nu = 1,2$
- c) $\chi^2_\nu = 0,05$
- d) $\chi^2_\nu = 10,0$

Ejercicio 5 Para dos conjuntos de datos con ajuste lineal:

- Conjunto A: $r = 0,99$, $\chi^2_\nu = 0,8$
- Conjunto B: $r = 0,85$, $\chi^2_\nu = 1,1$

- a) ¿Cuál ajuste es mejor según r ?
- b) ¿Cuál ajuste es mejor según χ^2_ν ?
- c) ¿Puede un ajuste tener r alto pero χ^2_ν también alto?

Ejercicio 6 Calcula el coeficiente de correlación de Pearson para:

x	1	2	3	4
y	1	2	3	4

- a) Manualmente, paso a paso
- b) Verifica con Python
- c) ¿Qué valor esperas y por qué?

Ejercicio 7 Un estudiante hace un ajuste lineal y obtiene $\chi^2_\nu = 0,2$. Sus datos tienen $\Delta y_i = 0,1$ para todos los puntos.

- a) ¿Qué podría estar pasando?
- b) ¿Cómo podría verificar su hipótesis?
- c) ¿Qué debería hacer para obtener $\chi^2_\nu \sim 1$?

Ejercicio 8 Los siguientes datos tienen un outlier:

x	1	2	3	4	5	6
y	2.0	4.0	6.0	8.0	10.0	15.0
Δy	0.2	0.2	0.2	0.2	0.2	0.2

- a) Haz un ajuste lineal a todos los datos
- b) Calcula χ^2_ν
- c) Identifica el outlier calculando residuos estandarizados
- d) Haz un ajuste sin el outlier y compara

Ejercicio 9 Demuestra que el coeficiente de correlación r para una recta perfecta $y = mx + b$ es $r = 1$ si $m > 0$ o $r = -1$ si $m < 0$.

Ejercicio 10 Analiza los siguientes resultados de ajuste para un experimento de péndulo (T^2 vs L):

- $r = 0,998$
- $\chi^2_\nu = 1,5$
- Residuos: aleatorios, pero un patrón suave
- $g = (9,75 \pm 0,05)$

- a) Evalúa la calidad del ajuste
- b) Sugiere posibles mejoras
- c) ¿El valor de g es compatible con el teórico?

Soluciones - Capítulo 5

Solución:Ejercicio 1

Listing 3.9: Código Python para ejercicio 1

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Datos del ejercicio 1, capitulo 4
5 x = np.array([1.0, 2.0, 3.0, 4.0, 5.0, 6.0])
6 y = np.array([2.1, 3.9, 6.2, 8.1, 9.8, 12.1])
7 dy = np.array([0.2, 0.3, 0.2, 0.3, 0.2, 0.3])
8
9 # Valores del ajuste (del ejercicio 1)
10 m = 2.013
11 b = 0.042
12 y_pred = m * x + b
13
14 # a) Coeficiente de correlación de Pearson
15 N = len(x)
16 x_mean = np.mean(x)
17 y_mean = np.mean(y)
18
19 # Covarianza y varianzas
20 cov_xy = np.sum((x - x_mean) * (y - y_mean)) / N
21 var_x = np.sum((x - x_mean)**2) / N
22 var_y = np.sum((y - y_mean)**2) / N
23
24 r = cov_xy / np.sqrt(var_x * var_y)
25 print(f"a) Coeficiente de correlación de Pearson:")
26 print(f"    cov(x,y) = {cov_xy:.4f}")
27 print(f"    _x  = {np.sqrt(var_x):.4f},    _y  = {np.sqrt(var_y):.4f}")
28 print(f"    ↪ ")
29 print(f"    r = {r:.4f}")
30
31 # b) y _reducido
32 residuos = y - y_pred
33 chi2 = np.sum((residuos / dy)**2)
34 chi2_red = chi2 / (N - 2) # Para recta: 2 parámetros (m y b)
35
36 print(f"\nb) y _reducido :")
37 print(f"    Residuos: {residuos.round(3)}")
38 print(f"    = ( _i / y_i ) = {chi2:.3f}")
39 print(f"    Grados libertad: = N - 2 = {N} - 2 = {N-2}")
40 print(f"    _red = / = {chi2_red:.3f}")
41
42 # c) Gráfico de residuos
43 plt.figure(figsize=(10, 4))
```

```
44 plt.subplot(1, 2, 1)
45 plt.errorbar(x, residuos, yerr=dy, fmt='o', capsize=5)
46 plt.axhline(y=0, color='r', linestyle='--', alpha=0.5)
47 plt.xlabel('x')
48 plt.ylabel('Residuos = y - y_ajuste')
49 plt.title('Gráfico de residuos')
50 plt.grid(True, alpha=0.3)
51
52 # Residuos normalizados (divididos por sus incertezas)
53 residuos_norm = residuos / dy
54 plt.subplot(1, 2, 2)
55 plt.errorbar(x, residuos_norm, yerr=1, fmt='o', capsize=5)
56 plt.axhline(y=0, color='r', linestyle='--', alpha=0.5)
57 plt.axhline(y=1, color='gray', linestyle='--', alpha=0.5)
58 plt.axhline(y=-1, color='gray', linestyle='--', alpha=0.5)
59 plt.xlabel('x')
60 plt.ylabel('Residuos normalizados / y ')
61 plt.title('Residuos normalizados')
62 plt.grid(True, alpha=0.3)
63 plt.tight_layout()
64 plt.show()
65
66 # d) Evaluación
67 print(f"\nd) Evaluación de la calidad del ajuste:")
68 print(f" 1. r = {r:.4f} 1 Fuerte correlación lineal")
69 print(f" 2. _red = {chi2_red:.3f} 1 Incertezas bien
    ↪ estimadas")
70 print(f" 3. Residuos: Aleatorios alrededor de cero")
71 print(f" 4. {np.sum(np.abs(residuos_norm) < 1)}/{N} residuos
    ↪ normalizados en [-1,1]")
72 print(f" Conclusión: Buen ajuste")
```

Resultados:

- a) $r = 0,9996$ (correlación casi perfecta)
- b) $\chi^2 = 2,786$, $\chi^2_\nu = 0,696 \approx 0,7$
- c) Gráfico muestra residuos aleatorios
- d) Buen ajuste: $r \approx 1$, $\chi^2_\nu \approx 1$, residuos aleatorios

Solución:Ejercicio 2

Listing 3.10: Código Python para ejercicio 2

```
1 import numpy as np
2
3 # Datos
4 x = np.array([1, 2, 3, 4, 5, 6])
5 y = np.array([2.1, 3.9, 5.8, 7.9, 10.1, 12.0])
6 y_ajuste = np.array([2.0, 4.0, 6.0, 8.0, 10.0, 12.0])
7 dy = np.array([0.2, 0.3, 0.2, 0.3, 0.2, 0.3])
8
```

```

9 # a) Residuos
10 residuos = y - y_ajuste
11 print("a) Residuos:")
12 for i in range(len(x)):
13     print(f"    x={x[i]}:      = {y[i]} - {y_ajuste[i]} = {residuos[i]
14           ↪ }:.1f}")
15
16 # b)      y      _red
17 chi2 = np.sum((residuos / dy)**2)
18 N = len(x)
19 chi2_red = chi2 / (N - 2) # Para recta
20
21 print(f"\nb)      y      _reducido      :")
22 print(f"      =      ( _i / y_i )      = ", end="")
23 for i in range(N):
24     print(f"({residuos[i]:.1f}/{dy[i]:.1f})      ", end="")
25     if i < N-1:
26         print(" + ", end="")
27 print(f" = {chi2:.3f}")
28 print(f"      _red      = {chi2:.3f} / ({N} - 2) = {chi2_red:.3f}")
29
30 # c) Interpretaci n
31 print(f"\nc) Interpretaci n:")
32 print(f"      _red      = {chi2_red:.3f}")
33 if chi2_red < 0.5:
34     print("      MUY bajo: incertezas y_i probablemente
35           ↪ sobreestimadas")
36 elif chi2_red < 1:
37     print("      Bajo: incertezas bien estimadas o ligeramente
38           ↪ sobreestimadas")
39 elif chi2_red < 2:
40     print("      Aceptable: incertezas bien estimadas")
41 elif chi2_red < 5:
42     print("      Alto: incertezas subestimadas o modelo no perfecto")
43 else:
44     print("      MUY alto: problema serio con incertezas o modelo")
45
46 # Calcular contribuci n de cada punto
47 contrib = (residuos / dy)**2 / chi2 * 100
48 print(f"\n      Contribuci n porcentual de cada punto a      :")
49 for i in range(N):
50     print(f"      Punto {i+1}: {contrib[i]:.1f}%")

```

Resultados:

a) Residuos: [0,1, -0,1, -0,2, -0,1, 0,1, 0,0]

b) $\chi^2 = 1,944$, $\chi^2_\nu = 0,486$

c) $\chi^2_\nu = 0,486 < 1$: Incertezas probablemente sobreestimadas

Soluci3n:Ejercicio 3

- a) Residuos: $\{0,1, -0,2, 0,3, -0,1, 0,2, -0,3\}$
- Patrón: Aleatorio, sin tendencia
 - Interpretación: Buen ajuste, residuos distribuidos aleatoriamente
 - Gráfico: Puntos dispersos alrededor de cero
- b) Residuos: $\{1,0, 0,5, 0,0, -0,5, -1,0, -1,5\}$
- Patrón: Tendencia lineal decreciente
 - Interpretación: Modelo incompleto, falta término lineal o hay error sistemático
 - Gráfico: Puntos en línea recta con pendiente negativa
- c) Residuos: $\{0,5, -0,5, 0,5, -0,5, 0,5, -0,5\}$
- Patrón: Alternancia regular (patrón de "sierra")
 - Interpretación: Posible error periódico en mediciones o modelo inadecuado
 - Gráfico: Puntos que alternan arriba y abajo de cero

Solución:Ejercicio 4

- a) $\chi^2_\nu = 0,5$: Menor que 1, sugiere que las incertezas experimentales Δy_i pueden estar sobreestimadas. Los datos se ajustan mejor de lo esperado dado el tamaño de las barras de error.
- b) $\chi^2_\nu = 1,2$: Cercano a 1, indica que las incertezas están bien estimadas y el modelo es adecuado. Pequeñas desviaciones de 1 son normales.
- c) $\chi^2_\nu = 0,05$: Mucho menor que 1, fuerte indicación de que las incertezas están muy sobreestimadas. Podría deberse a errores de estimación conservadores.
- d) $\chi^2_\nu = 10,0$: Mucho mayor que 1, indica problemas graves: incertezas subestimadas, modelo inadecuado, errores sistemáticos no considerados, o presencia de outliers.

Solución:Ejercicio 5

Listing 3.11: Código Python para ejercicio 5

```

1 # An lisis comparativo
2 print("An lisis de conjuntos A y B:")
3 print("Conjunto A: r = 0.99,   _red   = 0.8")
4 print("Conjunto B: r = 0.85,   _red   = 1.1")
5 print()
6
7 print("a) Seg n r (coeficiente de correlaci n):")
8 print("    r mide la fuerza de la relaci n lineal")
9 print("    r_A = 0.99 > 0.85 = r_B")
10 print("        Conjunto A tiene mejor correlaci n lineal")
11 print()
12
13 print("b) Seg n   _red   :")
14 print("    _red   mide calidad del ajuste considerando incertezas"
    ↪ )

```

```

15 print("      _red_A      = 0.8 < 1.1 =      _red_B   ")
16 print("      Ambos son razonables (cerca de 1)")
17 print("      Conjunto A ligeramente mejor (m s cercano a 1)")
18 print()
19
20 print("c) Puede un ajuste tener r alto pero      _red      tambi n
      ↪ alto?")
21 print("      S , es posible. Consideremos:")
22 print("      1. r alto significa fuerte correlaci n LINEAL")
23 print("      2.      _red      alto significa que los datos no caen cerca de
      ↪ la recta")
24 print("      DADO el tama o de las incertezas")
25 print("      Ejemplo: Datos en l nea recta pero con incertezas muy
      ↪ peque as")
26 print("      r      1 (correlaci n perfecta)")
27 print("      _red      grande (los puntos no est n TAN cerca de la
      ↪ recta como")
28 print("      sugerir an las peque as incertezas)")
29 print()
30
31 # Ejemplo num rico
32 print("Ejemplo concreto:")
33 x = np.array([1, 2, 3, 4, 5])
34 y = np.array([2.0, 4.0, 6.0, 8.0, 10.0]) # Perfecto y = 2x
35 dy = np.array([0.01, 0.01, 0.01, 0.01, 0.01]) # Incertezas
      ↪ peque as
36
37 # A adimos peque a desviaci n
38 y_pert = y + np.array([0.1, -0.1, 0.1, -0.1, 0.1])
39
40 # Calcular r
41 x_mean = np.mean(x)
42 y_mean = np.mean(y_pert)
43 r = np.sum((x - x_mean) * (y_pert - y_mean)) / np.sqrt(np.sum((x -
      ↪ x_mean)**2) * np.sum((y_pert - y_mean)**2))
44
45 # Calcular      para ajuste y = 2x
46 residuos = y_pert - 2*x
47 chi2 = np.sum((residuos / dy)**2)
48 chi2_red = chi2 / (len(x) - 1)
49
50 print(f"      Datos: y      2x con peque as perturbaciones")
51 print(f"      Incertezas: y = 0.01 (muy peque as)")
52 print(f"      r = {r:.4f} (casi 1)")
53 print(f"      _red      = {chi2_red:.1f} (MUY grande)")
54 print("      r alto pero      _red      tambi n alto")

```

Resultados:

- a) Conjunto A es mejor según r ($0.99 > 0.85$)
- b) Conjunto A es ligeramente mejor según χ^2_ν ($0.8 < 1.1$)

- c) Sí puede tener r alto pero χ^2_ν alto: correlación lineal fuerte pero incertezas subestimadas

Solución:Ejercicio 6

Listing 3.12: Código Python para ejercicio 6

```

1 import numpy as np
2
3 # Datos
4 x = np.array([1, 2, 3, 4])
5 y = np.array([1, 2, 3, 4])
6 N = len(x)
7
8 # a) Cálculo manual paso a paso
9 print("a) Cálculo manual paso a paso:")
10 print(f"    Datos: x = {x}, y = {y}")
11 print(f"    N = {N}")
12
13 # Medias
14 x_mean = np.mean(x)
15 y_mean = np.mean(y)
16 print(f"\n    1. Medias:")
17 print(f"        x = ({1+2+3+4})/4 = {x_mean}")
18 print(f"        = ({1+2+3+4})/4 = {y_mean}")
19
20 # Desviaciones
21 dx = x - x_mean
22 dy = y - y_mean
23 print(f"\n    2. Desviaciones:")
24 print(f"        x - x : {dx}")
25 print(f"        y -   : {dy}")
26
27 # Productos y cuadrados
28 dx_dy = dx * dy
29 dx2 = dx**2
30 dy2 = dy**2
31 print(f"\n    3. Productos y cuadrados:")
32 print(f"        (x-x )(y- ) : {dx_dy}")
33 print(f"        (x-x )   : {dx2}")
34 print(f"        (y- )   : {dy2}")
35
36 # Sumatorias
37 sum_dx_dy = np.sum(dx_dy)
38 sum_dx2 = np.sum(dx2)
39 sum_dy2 = np.sum(dy2)
40 print(f"\n    4. Sumatorias:")
41 print(f"        (x-x )(y- ) = {sum_dx_dy}")
42 print(f"        (x-x )   = {sum_dx2}")
43 print(f"        (y- )   = {sum_dy2}")
44
45 # Cálculo de r
46 r_manual = sum_dx_dy / np.sqrt(sum_dx2 * sum_dy2)

```

```

47 print(f"\n    5. Coeficiente r:")
48 print(f"        r = (x-x_)(y-y_) / [ (x-x_)^2 + (y-y_)^2 ]")
49 print(f"        = {sum_dx_dy} / [{sum_dx2} + {sum_dy2}]")
50 print(f"        = {sum_dx_dy} / {sum_dx2*sum_dy2}")
51 print(f"        = {sum_dx_dy} / {np.sqrt(sum_dx2*sum_dy2)}")
52 print(f"        = {r_manual}")
53
54 # b) Verificaci n con Python
55 r_np = np.corrcoef(x, y)[0, 1]
56 print(f"\nb) Verificaci n con numpy.corrcoef:")
57 print(f"    r = {r_np}")
58 print(f"    Coinciden ? {np.isclose(r_manual, r_np)}")
59
60 # c) Interpretaci n
61 print(f"\nc) Interpretaci n:")
62 print(f"    r = {r_manual:.1f} exactamente")
63 print(f"    Los puntos est n perfectamente alineados en l nea
    ↪ recta")
64 print(f"    con pendiente positiva (y = x)")
65 print(f"    Por lo tanto, correlaci n lineal perfecta: r = 1")

```

Resultados:

- a) $r = 1,0$ (cálculo manual)
- b) $r = 1,0$ (numpy)
- c) Se espera $r = 1$ porque $y = x$ es línea recta perfecta

Solución:Ejercicio 7

- a) $\chi^2_\nu = 0,2 \ll 1$ sugiere:
 - Las incertezas $\Delta y_i = 0,1$ están sobreestimadas
 - Los datos se ajustan mejor de lo esperado para esas incertezas
 - Posible error sistemático en la estimación de incertezas
- b) Para verificar:
 - Revisar cómo se estimaron las incertezas Δy_i
 - Calcular los residuos normalizados: $\delta_i/\Delta y_i$
 - Si muchos tienen $|\delta_i/\Delta y_i| \ll 1$, confirma sobreestimación
 - Repetir mediciones para estimar mejor las incertezas
- c) Para obtener $\chi^2_\nu \sim 1$:
 - Reducir las incertezas estimadas (ej: usar $\Delta y_i = 0,05$ en lugar de 0.1)
 - O mantener incertezas pero el ajuste debería ser peor (mayores residuos)
 - Ajustar mejor las incertezas a la dispersión real de los datos

Solución:Ejercicio 8

Listing 3.13: Código Python para ejercicio 8

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Datos con outlier
5 x = np.array([1, 2, 3, 4, 5, 6])
6 y = np.array([2.0, 4.0, 6.0, 8.0, 10.0, 15.0]) # ltimo punto es
    ↪ outlier
7 dy = 0.2 * np.ones_like(x)
8 N = len(x)
9
10 # a) Ajuste a TODOS los datos
11 m_all, b_all = np.polyfit(x, y, 1)
12 y_pred_all = m_all * x + b_all
13 residuos_all = y - y_pred_all
14
15 # b) _red para todos los datos
16 chi2_all = np.sum((residuos_all / dy)**2)
17 chi2_red_all = chi2_all / (N - 2)
18
19 print("a) Ajuste a TODOS los datos:")
20 print(f" Recta: y = {m_all:.3f}x + {b_all:.3f}")
21 print(f" Residuos: {residuos_all.round(3)}")
22 print(f"\nb) y _red :")
23 print(f" = {chi2_all:.3f}")
24 print(f" _red = {chi2_red_all:.3f}")
25
26 # c) Identificar outlier con residuos estandarizados
27 residuos_estandarizados = residuos_all / dy
28 print(f"\nc) Residuos estandarizados ( / y ):")
29 for i in range(N):
30     print(f" Punto {i+1}: {residuos_estandarizados[i]:.2f} ")
31
32 # Punto con mayor residuo estandarizado (en valor absoluto)
33 outlier_idx = np.argmax(np.abs(residuos_estandarizados))
34 print(f"\n Outlier identificado: punto {outlier_idx+1} (x={x[
    ↪ outlier_idx]}, y={y[outlier_idx]})")
35 print(f" Residuo estandarizado: {residuos_estandarizados[
    ↪ outlier_idx]:.2f} ")
36
37 # d) Ajuste SIN el outlier
38 mask = np.ones(N, dtype=bool)
39 mask[outlier_idx] = False # Excluir outlier
40 x_sin = x[mask]
41 y_sin = y[mask]
42 dy_sin = dy[mask]
43
44 m_sin, b_sin = np.polyfit(x_sin, y_sin, 1)
45 y_pred_sin = m_sin * x_sin + b_sin
46 residuos_sin = y_sin - y_pred_sin
47

```

```

48 chi2_sin = np.sum((residuos_sin / dy_sin)**2)
49 chi2_red_sin = chi2_sin / (len(x_sin) - 2)
50
51 print(f"\nd) Ajuste SIN el outlier (punto {outlier_idx+1}):")
52 print(f"    Recta: y = {m_sin:.3f}x + {b_sin:.3f}")
53 print(f"    _red    = {chi2_red_sin:.3f}")
54 print(f"\n    Comparaci n:")
55 print(f"    Con outlier:    m = {m_all:.3f}, b = {b_all:.3f},
    ↪    _red    = {chi2_red_all:.3f}")
56 print(f"    Sin outlier:    m = {m_sin:.3f}, b = {b_sin:.3f},
    ↪    _red    = {chi2_red_sin:.3f}")
57 print(f"    Diferencia m: {abs(m_all - m_sin):.3f}")
58 print(f"    Diferencia _red : {abs(chi2_red_all - chi2_red_sin)
    ↪    :.3f}")
59
60 # Gr ficos comparativos
61 plt.figure(figsize=(12, 4))
62
63 # Gr fico 1: Datos y ajustes
64 plt.subplot(1, 3, 1)
65 plt.errorbar(x, y, yerr=dy, fmt='o', capsize=5, label='Datos')
66 # Ajuste con todos
67 x_fit = np.linspace(0.5, 6.5, 100)
68 y_fit_all = m_all * x_fit + b_all
69 plt.plot(x_fit, y_fit_all, 'r-', label=f'Con outlier: y={m_all:.3f}
    ↪ x+{b_all:.3f}')
70 # Ajuste sin outlier
71 y_fit_sin = m_sin * x_fit + b_sin
72 plt.plot(x_fit, y_fit_sin, 'b--', label=f'Sin outlier: y={m_sin:.3f}
    ↪ }x+{b_sin:.3f}')
73 plt.xlabel('x')
74 plt.ylabel('y')
75 plt.title('Datos y ajustes')
76 plt.grid(True, alpha=0.3)
77 plt.legend()
78
79 # Gr fico 2: Residuos con outlier
80 plt.subplot(1, 3, 2)
81 plt.errorbar(x, residuos_all, yerr=dy, fmt='o', capsize=5)
82 plt.axhline(y=0, color='r', linestyle='-', alpha=0.5)
83 plt.scatter(x[outlier_idx], residuos_all[outlier_idx], color='red',
    ↪ s=100,
84             label=f'Outlier:    ={residuos_all[outlier_idx]:.1f}')
85 plt.xlabel('x')
86 plt.ylabel('Residuos')
87 plt.title('Residuos (con outlier)')
88 plt.grid(True, alpha=0.3)
89 plt.legend()
90
91 # Gr fico 3: Residuos sin outlier
92 plt.subplot(1, 3, 3)

```

```
93 plt.errorbar(x_sin, residuos_sin, yerr=dy_sin, fmt='o', capsize=5)
94 plt.axhline(y=0, color='r', linestyle='-', alpha=0.5)
95 plt.xlabel('x')
96 plt.ylabel('Residuos')
97 plt.title('Residuos (sin outlier)')
98 plt.grid(True, alpha=0.3)
99
100 plt.tight_layout()
101 plt.show()
```

Resultados:

- a) Con todos: $y = 2,371x - 0,071$
- b) $\chi^2_\nu = 26,6$ (MUY alto)
- c) Outlier: punto 6, residuo estandarizado $6,71\sigma$
- d) Sin outlier: $y = 2,000x + 0,000$, $\chi^2_\nu = 0,4$

Solución:Ejercicio 9 Para una recta perfecta $y = mx + b$, todos los puntos satisfacen exactamente $y_i = mx_i + b$.

Calculemos r :

$$\begin{aligned}\bar{x} &= \frac{1}{N} \sum x_i \\ \bar{y} &= \frac{1}{N} \sum y_i = \frac{1}{N} \sum (mx_i + b) = m\bar{x} + b \\ x_i - \bar{x} &= x_i - \bar{x} \\ y_i - \bar{y} &= (mx_i + b) - (m\bar{x} + b) = m(x_i - \bar{x}) \\ \text{cov}(x, y) &= \frac{1}{N} \sum (x_i - \bar{x})(y_i - \bar{y}) = \frac{1}{N} \sum (x_i - \bar{x}) \cdot m(x_i - \bar{x}) = m\sigma_x^2 \\ \sigma_y &= \sqrt{\frac{1}{N} \sum (y_i - \bar{y})^2} = \sqrt{\frac{1}{N} \sum [m(x_i - \bar{x})]^2} = |m|\sigma_x \\ r &= \frac{\text{cov}(x, y)}{\sigma_x \sigma_y} = \frac{m\sigma_x^2}{\sigma_x \cdot |m|\sigma_x} = \frac{m}{|m|}\end{aligned}$$

Por lo tanto:

- Si $m > 0$: $r = \frac{m}{m} = 1$
- Si $m < 0$: $r = \frac{m}{-m} = -1$

Solución:Ejercicio 10

Listing 3.14: Código Python para ejercicio 10

```
1 print("An lisis de resultados de ajuste para p ndulo:")
2 print("  r = 0.998")
3 print("    _red    = 1.5")
4 print("  Residuos: aleatorios, pero con patr n suave")
5 print("  g = (9.75      0.05) m/s ")
6 print("  g_te rico = 9.81 m/s ")
7 print()
```

```

8
9 # a) Evaluaci n de calidad
10 print("a) Evaluaci n de la calidad del ajuste:")
11 print("    1. r = 0.998      Excelente correlaci n lineal")
12 print("        (muy cercano a 1)")
13 print("    2.    _red    = 1.5      Aceptable, ligeramente alto")
14 print("        Sugiere incertezas posiblemente subestimadas")
15 print("        o peque os errores sistem ticos")
16 print("    3. Residuos con 'patr n suave'      Preocupante")
17 print("        Aunque aleatorios, un patr n suave sugiere")
18 print("        que el modelo podr a no ser perfecto")
19 print("    4. En general: Buen ajuste, pero con margen de mejora")
20 print()
21
22 # b) Sugerencias de mejora
23 print("b) Sugerencias de mejoras:")
24 print("    1. Revisar estimaci n de incertezas:")
25 print("        - T y L est n bien estimadas?")
26 print("        - Podr an estar subestimadas ( _red > 1)")
27 print("    2. Investigar patr n suave en residuos:")
28 print("        - Graficar residuos vs L")
29 print("        - Buscar correlaci n con otras variables")
30 print("        (ej: amplitud, masa, temperatura)")
31 print("    3. Considerar correcciones al modelo:")
32 print("        - P ndulo no ideal (masa del hilo, amplitud)")
33 print("        -  $T = 2 \sqrt{L/g} [1 + \dots]$ ")
34 print("        - Efectos de fricci n")
35 print("    4. Aumentar n mero de puntos o rango de L")
36 print("    5. Verificar calibraci n de instrumentos")
37 print()
38
39 # c) Compatibilidad con valor te rico
40 print("c) Compatibilidad con valor te rico:")
41 g_exp = 9.75
42 Dg_exp = 0.05
43 g_teo = 9.81
44
45 diferencia = abs(g_exp - g_teo)
46 sum_errores = Dg_exp # Solo tenemos error experimental
47
48 print(f"    g_exp = {g_exp} {Dg_exp} m/s ")
49 print(f"    g_teo = {g_teo} m/s ")
50 print(f"    |g_exp - g_teo| = |{g_exp} - {g_teo}| = {diferencia:.2f}
51     ↪ m/s ")
51 print(f"    g_exp = {Dg_exp} m/s ")
52
53 if diferencia > 2 * Dg_exp: # M s de 2
54     print(f"    {diferencia:.2f} > 2 {Dg_exp} = {2*Dg_exp:.2f}")
55     print("        Diferencia significativa (m s de 2 )")
56 elif diferencia > Dg_exp:
57     print(f"    {diferencia:.2f} > {Dg_exp:.2f}")

```



```

58     print("          Diferencia moderada (entre 1 y 2 )")
59 else:
60     print(f"          {diferencia:.2f}          {Dg_exp:.2f}")
61     print("          Compatible dentro de 1 ")
62
63 # Calculamos cu ntas de diferencia
64 sigma_diff = diferencia / Dg_exp
65 print(f"          La diferencia es de {sigma_diff:.1f} ")
66
67 print("\n          Conclusi n: El valor experimental es ligeramente bajo"
68       ↪ )
69 print("          pero dentro de lo razonable considerando las incertezas."
70       ↪ )
71 print("          Podr a indicar peque os efectos sistem ticos no
72       ↪ considerados.")

```

Resultados:

- a) Buen ajuste general: r alto, χ^2_ν aceptable, pero patrón suave en residuos sugiere posible mejora
- b) Mejorar estimación incertezas, investigar patrón en residuos, considerar correcciones al modelo
- c) $|9,75 - 9,81| = 0,06$, $\Delta g = 0,05 \rightarrow$ diferencia de $1,2\sigma$, compatible dentro de incertezas

Ejercicios - Capítulo 6: Aplicaciones Prácticas

Ejercicio 1 En un experimento para determinar π , se miden diámetros y perímetros de 8 círculos:

d (cm)	2.0	3.0	4.0	5.0	6.0	7.0	8.0	9.0
P (cm)	6.3	9.4	12.6	15.7	18.8	22.0	25.1	28.3
Δd (cm)	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
ΔP (cm)	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2

- Grafica P vs d con barras de error
- Ajusta recta $P = \pi d$ por cuadrados mínimos
- Determina π con su incerteza
- Calcula r y χ^2_ν
- Compara con valor teórico $\pi = 3,1416$

Ejercicio 2 Para determinar g con péndulo simple, se miden:

L (cm)	30.0	40.0	50.0	60.0	70.0	80.0
T (s)	1.10	1.27	1.42	1.55	1.68	1.79
ΔL (cm)	0.1	0.1	0.1	0.1	0.1	0.1
ΔT (s)	0.01	0.01	0.01	0.01	0.01	0.01

- Calcula T^2 y sus incertezas
- Grafica T^2 vs L
- Ajusta recta $T^2 = \frac{4\pi^2}{g} L$
- Determina g con su incerteza
- Compara con $g = 9,81$

Ejercicio 3 Para medir constante elástica k , se cuelgan masas en un resorte:

m (g)	50	100	150	200	250	300
Δx (cm)	0.8	1.6	2.4	3.2	4.0	4.8
Δm (g)	1	1	1	1	1	1
$\Delta(\Delta x)$ (cm)	0.1	0.1	0.1	0.1	0.1	0.1

- Convierte a $F = mg$ (N) y Δx (m)
- Grafica F vs Δx

- c) Ajusta recta $F = k\Delta x$
- d) Determina k (N/m) con incerteza
- e) ¿Cuál sería k en N/cm?

Ejercicio 4 Se estudia movimiento uniformemente acelerado:

t (s)	0.0	1.0	2.0	3.0	4.0	5.0
x (m)	0.0	1.1	4.2	9.1	16.0	25.1
Δt (s)	0.1	0.1	0.1	0.1	0.1	0.1
Δx (m)	0.1	0.1	0.1	0.1	0.1	0.1

- a) Grafica x vs t
- b) Ajusta parábola $x = x_0 + v_0t + \frac{1}{2}at^2$
- c) Determina a (aceleración) con incerteza
- d) ¿Es $v_0 \approx 0$ como se espera?

Ejercicio 5 Se miden resistencia y temperatura de un termistor:

T (°C)	20	30	40	50	60	70
R (k)	10.0	6.8	4.7	3.3	2.3	1.6
ΔT (°C)	0.5	0.5	0.5	0.5	0.5	0.5
ΔR (k)	0.1	0.1	0.1	0.1	0.1	0.1

Se espera $R = R_0e^{B/T}$ con T en Kelvin.

- a) Convierte a $T_K = T_C + 273,15$
- b) Grafica $\ln R$ vs $1/T$
- c) Ajusta recta para determinar B y R_0
- d) ¿Qué representa físicamente B ?

Ejercicio 6 En un experimento de caída libre, se miden alturas en función del tiempo:

t (s)	0.2	0.4	0.6	0.8	1.0	1.2
y (m)	0.20	0.78	1.76	3.14	4.90	7.06
Δt (s)	0.01	0.01	0.01	0.01	0.01	0.01
Δy (m)	0.02	0.02	0.02	0.02	0.02	0.02

- a) Grafica y vs t
- b) Ajusta $y = \frac{1}{2}gt^2$ (asumiendo $y_0 = 0$, $v_0 = 0$)
- c) Determina g con incerteza
- d) ¿Es válida la suposición $v_0 = 0$?

Ejercicio 7 Se mide período de péndulo para diferentes amplitudes pequeñas:

θ (grados)	5	10	15	20	25	30
T (s)	2.01	2.02	2.03	2.05	2.08	2.11
$\Delta\theta$ (grados)	0.5	0.5	0.5	0.5	0.5	0.5
ΔT (s)	0.01	0.01	0.01	0.01	0.01	0.01

La teoría predice $T = T_0(1 + \frac{\theta^2}{16} + \dots)$ con θ en radianes.

- Convierte θ a radianes
- Grafica T vs θ^2
- Ajusta recta $T = T_0(1 + \frac{\theta^2}{16})$
- Determina T_0

Ejercicio 8 Para un circuito RC, se mide voltaje en función del tiempo:

t (s)	0	1	2	3	4	5
V (V)	10.0	6.1	3.7	2.2	1.4	0.8
Δt (s)	0.1	0.1	0.1	0.1	0.1	0.1
ΔV (V)	0.1	0.1	0.1	0.1	0.1	0.1

Se espera $V = V_0 e^{-t/\tau}$.

- Grafica $\ln V$ vs t
- Ajusta recta para determinar τ (constante de tiempo)
- Determina V_0
- ¿Qué resistencia si $C = 100\mu F$?

Ejercicio 9 Se miden intensidad y voltaje en una resistencia:

V (V)	1.0	2.0	3.0	4.0	5.0	6.0
I (A)	0.10	0.20	0.30	0.40	0.49	0.59
ΔV (V)	0.1	0.1	0.1	0.1	0.1	0.1
ΔI (A)	0.01	0.01	0.01	0.01	0.01	0.01

- Grafica I vs V
- Ajusta recta $I = \frac{1}{R}V$
- Determina R con incerteza
- Calcula potencia para $V = 5V$: $P = IV$

Ejercicio 10 Proyectil: se miden alcance R y ángulo θ :

θ (grados)	15	30	45	60	75	90
R (m)	5.1	8.7	10.0	8.6	5.2	0.0
$\Delta\theta$ (grados)	0.5	0.5	0.5	0.5	0.5	0.5
ΔR (m)	0.1	0.1	0.1	0.1	0.1	0.1

Teoría: $R = \frac{v_0^2}{g} \sin(2\theta)$ para misma v_0 .

- a) Grafica R vs $\sin(2\theta)$
- b) Ajusta recta para determinar v_0
- c) Determina g si se conoce $v_0 = 10 \text{ m/s}$
- d) ¿Para qué θ es máximo R ?

Repaso Teórico-Práctico de Laboratorio 1

Física Experimental

Universidad de Buenos Aires - Exactas
Departamento de Física

Nahuel Villafañe

Basado en los apuntes de:

Lucía Famá, Gabriel Paciaroni, Lucía Mazaira, Facundo Sánchez

4 de diciembre de 2025

Volumen 3: Modelado por Cuadrados Mínimos

Licencia Creative Commons BY-NC-SA 4.0

**Usted puede compartir y adaptar este material
dando crédito al autor, sin fines comerciales
y compartiendo bajo la misma licencia.**

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

Prefacio

Este documento continúa con el Volumen 3, dedicado al modelado por cuadrados mínimos, una herramienta fundamental para analizar relaciones entre variables experimentales. Como en los volúmenes anteriores, se busca complementar el material de clase con ejemplos detallados y ejercicios resueltos.

Índice general

Prefacio	3
1. Modelado por Cuadrados Mínimos	7
1.1. Introducción al Modelado de Datos	7
1.2. Modelo Lineal Simple	7
1.2.1. Principio de Mínimos Cuadrados	7
1.2.2. Solución Analítica	7
1.3. Cuadrados Mínimos Ponderados	8
1.3.1. Interpretación	8
1.4. Parámetros de Bondad del Ajuste	8
1.4.1. Coeficiente de Correlación de Pearson (r)	8
1.4.2. Chi-cuadrado Reducido (χ^2_ν)	8
1.4.3. Gráfico de Residuos	9
1.5. Selección de Variables para el Eje y	9
1.6. Aplicaciones a Fenómenos Físicos	9
1.6.1. Determinación de π por relación perímetro-diámetro	9
1.6.2. Determinación de g con péndulo simple	9
1.6.3. Movimiento uniformemente acelerado	9
2. Aplicaciones Prácticas del Método de Cuadrados Mínimos	11
2.1. Experimento: Determinación de π	11
2.1.1. Objetivo	11
2.1.2. Fundamento Teórico	11
2.1.3. Procedimiento Experimental	11
2.2. Experimento: Determinación de g con Péndulo Simple	11
2.2.1. Objetivo	11
2.2.2. Fundamento Teórico	11
2.2.3. Procedimiento Experimental	12
2.3. Experimento: Movimiento en Plano Inclinado	12
2.3.1. Objetivo	12
2.3.2. Fundamento Teórico	12
2.3.3. Procedimiento Experimental	12
Ejercicios - Capítulo 7	13
Soluciones - Capítulo 7	17

Capítulo 1

Modelado por Cuadrados Mínimos

1.1. Introducción al Modelado de Datos

En física experimental, frecuentemente buscamos relaciones entre variables. El método de cuadrados mínimos nos permite encontrar la curva que mejor se ajusta a nuestros datos experimentales, según un modelo matemático predefinido.

1.2. Modelo Lineal Simple

El caso más común es ajustar datos a una recta:

$$y = mx + b$$

donde:

- m : pendiente
- b : ordenada al origen
- x : variable independiente
- y : variable dependiente

1.2.1. Principio de Mínimos Cuadrados

Buscamos los parámetros m y b que minimizan la suma de los cuadrados de los residuos:

$$S = \sum_{i=1}^N [y_i - (mx_i + b)]^2$$

Los residuos $\delta_i = y_i - (mx_i + b)$ representan la distancia vertical entre cada punto experimental y la recta.

1.2.2. Solución Analítica

Para el modelo lineal, podemos obtener fórmulas explícitas:

$$m = \frac{N \sum x_i y_i - \sum x_i \sum y_i}{N \sum x_i^2 - (\sum x_i)^2}$$
$$b = \frac{\sum x_i^2 \sum y_i - \sum x_i \sum x_i y_i}{N \sum x_i^2 - (\sum x_i)^2}$$

1.3. Cuadrados Mínimos Ponderados

Cuando los datos tienen diferentes precisiones (diferentes Δy_i), usamos cuadrados mínimos ponderados. Minimizamos:

$$\chi^2 = \sum_{i=1}^N \left[\frac{y_i - (mx_i + b)}{\Delta y_i} \right]^2$$

1.3.1. Interpretación

- Cada residuo se divide por su incerteza
- Los datos más precisos (menor Δy_i) tienen más peso en el ajuste
- Cuando todos los Δy_i son iguales, recuperamos el caso no ponderado

1.4. Parámetros de Bondad del Ajuste

1.4.1. Coeficiente de Correlación de Pearson (r)

Mide la correlación lineal entre x e y :

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

- $-1 \leq r \leq 1$
- $r \approx 1$: correlación lineal positiva fuerte
- $r \approx -1$: correlación lineal negativa fuerte
- $r \approx 0$: no hay correlación lineal (puede haber otro tipo de relación)

1.4.2. Chi-cuadrado Reducido (χ_ν^2)

Mide la calidad del ajuste considerando las incertezas:

$$\chi^2 = \sum_{i=1}^N \left[\frac{y_i - f(x_i)}{\Delta y_i} \right]^2$$

$$\chi_\nu^2 = \frac{\chi^2}{\nu}$$

donde $\nu = N - p$ son los grados de libertad, y p es el número de parámetros del modelo.

Interpretación:

- $\chi_\nu^2 \approx 1$: buen ajuste, las incertezas están bien estimadas
- $\chi_\nu^2 \ll 1$: sobreestimación de incertezas o demasiados parámetros
- $\chi_\nu^2 \gg 1$: subestimación de incertezas o modelo inadecuado

1.4.3. Gráfico de Residuos

Los residuos deben:

1. Distribuirse aleatoriamente alrededor de cero
2. No mostrar patrones sistemáticos
3. Tener magnitud comparable a las incertezas Δy_i

1.5. Selección de Variables para el Eje y

Una regla práctica: asignar al eje y la variable con mayor error relativo (ε_r):

$$\varepsilon_r = \frac{\Delta y}{y_0}$$

El criterio es: $\varepsilon_{ry} \gg \varepsilon_{rx}$

1.6. Aplicaciones a Fenómenos Físicos

1.6.1. Determinación de π por relación perímetro-diámetro

Para una circunferencia: $P = \pi d$

Si graficamos P vs d , obtenemos una recta con pendiente $m = \pi$.

1.6.2. Determinación de g con péndulo simple

Período: $T = 2\pi\sqrt{\frac{L}{g}}$

Linealizando: $T^2 = \frac{4\pi^2}{g}L$

Graficando T^2 vs L , la pendiente es $m = \frac{4\pi^2}{g}$, luego $g = \frac{4\pi^2}{m}$.

1.6.3. Movimiento uniformemente acelerado

Para caída libre: $y(t) = y_0 + v_0t - \frac{1}{2}gt^2$

Si $v_0 = 0$, graficando y vs t^2 obtenemos recta con pendiente $m = -\frac{1}{2}g$.

Capítulo 2

Aplicaciones Prácticas del Método de Cuadrados Mínimos

2.1. Experimento: Determinación de π

2.1.1. Objetivo

Obtener el valor de π mediante la relación entre perímetro y diámetro de circunferencias.

2.1.2. Fundamento Teórico

Para cualquier circunferencia: $P = \pi d$

2.1.3. Procedimiento Experimental

1. Medir diámetro d y perímetro P de al menos 10 circunferencias diferentes
2. Calcular errores relativos: ε_{rP} y ε_{rd}
3. Graficar P vs d (variable con mayor ε_r en eje y)
4. Aplicar ajuste lineal por cuadrados mínimos
5. La pendiente obtenida es π

2.2. Experimento: Determinación de g con Péndulo Simple

2.2.1. Objetivo

Determinar la aceleración de la gravedad g mediante mediciones del período de un péndulo.

2.2.2. Fundamento Teórico

$$T = 2\pi\sqrt{\frac{L}{g}} \Rightarrow T^2 = \frac{4\pi^2}{g}L$$

2.2.3. Procedimiento Experimental

1. Medir período T para al menos 7 longitudes L diferentes (entre 30-150 cm)
2. Calcular T^2 y sus incertezas
3. Graficar T^2 vs L
4. Aplicar ajuste lineal: $y = mx + b$
5. Calcular $g = \frac{4\pi^2}{m}$
6. Verificar que $b \approx 0$ (como esperado teóricamente)

2.3. Experimento: Movimiento en Plano Inclinado

2.3.1. Objetivo

Determinar g estudiando el movimiento de un carrito en plano inclinado.

2.3.2. Fundamento Teórico

Ecuación de movimiento (sin rozamiento):

$$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2$$

con $a = g \sin \theta$

2.3.3. Procedimiento Experimental

1. Medir posición $x(t)$ con sensor de movimiento
2. Ajustar modelo cuadrático: $x(t) = A + Bt + Ct^2$
3. Identificar: $a = 2C$
4. Calcular $g = \frac{a}{\sin \theta}$

Ejercicios - Capítulo 7: Cuadrados Mínimos

Ejercicio 1 Dados los siguientes datos experimentales de x e y :

x (m)	y (s)
1.0	2.1
2.0	3.9
3.0	6.2
4.0	8.0
5.0	10.1

- a) Calcula manualmente los parámetros m y b del ajuste lineal $y = mx + b$
- b) Verifica tus cálculos con Python
- c) Calcula el coeficiente de correlación r
- d) Grafica los datos con la recta ajustada

Ejercicio 2 Se midieron perímetros y diámetros de 5 circunferencias:

Circunferencia	d (cm)	P (cm)
1	5.0 ± 0.1	15.7 ± 0.2
2	10.0 ± 0.1	31.4 ± 0.2
3	15.0 ± 0.1	47.1 ± 0.2
4	20.0 ± 0.1	62.8 ± 0.2
5	25.0 ± 0.1	78.5 ± 0.2

- a) Calcula los errores relativos ε_{rd} y ε_{rP}
- b) Decide qué variable graficar en el eje y
- c) Ajusta por cuadrados mínimos ponderados
- d) Obtén el valor de π con su incerteza

Ejercicio 3 Para un péndulo simple se obtuvieron:

L (m)	T (s)
0.30 ± 0.01	1.10 ± 0.02
0.50 ± 0.01	1.42 ± 0.02
0.70 ± 0.01	1.68 ± 0.02
0.90 ± 0.01	1.90 ± 0.02
1.10 ± 0.01	2.10 ± 0.02

- a) Calcula T^2 y su incerteza

- b) Grafica T^2 vs L con barras de error
- c) Ajusta por cuadrados mínimos ponderados
- d) Calcula g con su incerteza
- e) Calcula χ^2_ν y discute la calidad del ajuste

Ejercicio 4 Explica con tus palabras:

- a) ¿Por qué es importante graficar los residuos después de un ajuste?
- b) ¿Qué significa que $\chi^2_\nu \approx 1$?
- c) ¿Cuándo se deben usar cuadrados mínimos ponderados?
- d) ¿Por qué a veces es necesario linealizar una relación antes de ajustar?

Ejercicio 5 Demuestra que para el ajuste lineal $y = mx + b$, las fórmulas para m y b son:

$$m = \frac{N \sum x_i y_i - \sum x_i \sum y_i}{N \sum x_i^2 - (\sum x_i)^2}, \quad b = \frac{\sum x_i^2 \sum y_i - \sum x_i \sum x_i y_i}{N \sum x_i^2 - (\sum x_i)^2}$$

Ejercicio 6 Se estudia la relación entre voltaje V y corriente I en un resistor:

V (V)	I (A)
1.0	0.10 ± 0.01
2.0	0.20 ± 0.01
3.0	0.29 ± 0.01
4.0	0.39 ± 0.01
5.0	0.49 ± 0.01

- a) Grafica I vs V (¿qué variable en y ?)
- b) Ajusta por cuadrados mínimos ponderados
- c) La pendiente es la resistencia R . Calcúlala con su incerteza
- d) Verifica la ley de Ohm: $V = RI$

Ejercicio 7 En un experimento de caída libre se midieron:

t (s)	y (m)
0.1	0.049 ± 0.001
0.2	0.196 ± 0.001
0.3	0.441 ± 0.001
0.4	0.784 ± 0.001
0.5	1.225 ± 0.001

- a) Si $y = \frac{1}{2}gt^2$, linealiza la relación
- b) Realiza el ajuste apropiado
- c) Calcula g con su incerteza

d) Asumiendo $g_{\text{teo}} = 9,81 \text{ m/s}^2$, discute exactitud y precisión

Ejercicio 8 Dados dos conjuntos de datos con ajustes lineales:

■ Conjunto A: $r = 0,98$, $\chi^2_\nu = 1,2$

■ Conjunto B: $r = 0,95$, $\chi^2_\nu = 0,3$

a) ¿Qué conjunto tiene mejor correlación lineal?

b) ¿Qué conjunto tiene mejor ajuste considerando las incertezas?

c) ¿Qué podría explicar que $\chi^2_\nu < 1$?

Ejercicio 9 Se midió la elongación x de un resorte para diferentes masas m :

m (kg)	x (m)
0.1	0.021 ± 0.001
0.2	0.041 ± 0.001
0.3	0.062 ± 0.001
0.4	0.082 ± 0.001
0.5	0.102 ± 0.001

a) Por la ley de Hooke: $mg = kx$. Linealiza la relación

b) Realiza ajuste por cuadrados mínimos

c) Determina k (constante del resorte) con su incerteza

d) Calcula la ordenada al origen. ¿Es significativamente diferente de cero?

Ejercicio 10 Diseña un experimento para determinar la viscosidad de un líquido usando el método de cuadrados mínimos. Describe:

a) El fenómeno físico y la ley involucrada

b) Qué variables medirías

c) Cómo linealizarías la relación

d) Qué parámetro obtendrías del ajuste

Soluciones - Capítulo 7

Solución:Ejercicio 1

Listing 2.1: Código Python para ejercicio 1

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Datos
5 x = np.array([1.0, 2.0, 3.0, 4.0, 5.0])
6 y = np.array([2.1, 3.9, 6.2, 8.0, 10.1])
7 N = len(x)
8
9 # a) Calculo manual de m y b
10 sum_x = np.sum(x)
11 sum_y = np.sum(y)
12 sum_xy = np.sum(x * y)
13 sum_x2 = np.sum(x**2)
14
15 m = (N * sum_xy - sum_x * sum_y) / (N * sum_x2 - sum_x**2)
16 b = (sum_x2 * sum_y - sum_x * sum_xy) / (N * sum_x2 - sum_x**2)
17
18 print(f"a) m = {m:.3f}, b = {b:.3f}")
19
20 # b) Verificacion con numpy
21 m_np, b_np = np.polyfit(x, y, 1)
22 print(f"b) Verificacion numpy: m = {m_np:.3f}, b = {b_np:.3f}")
23
24 # c) Coeficiente de correlacion
25 r_num = np.sum((x - np.mean(x)) * (y - np.mean(y)))
26 r_den = np.sqrt(np.sum((x - np.mean(x))**2) * np.sum((y - np.mean(y)
    ↪ ))**2))
27 r = r_num / r_den
28 print(f"c) Coeficiente de correlacion r = {r:.4f}")
29
30 # d) Grafico
31 plt.figure(figsize=(8, 6))
32 plt.scatter(x, y, color='blue', label='Datos', s=100)
33 x_fit = np.linspace(0, 6, 100)
34 y_fit = m * x_fit + b
35 plt.plot(x_fit, y_fit, 'r-', label=f'Ajuste: y = {m:.2f}x + {b:.2f}
    ↪ ')
36 plt.xlabel('x (m)')
37 plt.ylabel('y (s)')
38 plt.title('Ajuste lineal por cuadrados m nimos')
39 plt.legend()
40 plt.grid(True, alpha=0.3)
41 plt.show()
```

Resultados:

- a) $m = 2,005$, $b = 0,005$
- b) Verificación: $m = 2,005$, $b = 0,005$ (coincide)
- c) $r = 0,9995$ (correlación lineal muy fuerte)
- d) Gráfico con recta ajustada perfectamente a los datos

Solución:Ejercicio 2

Listing 2.2: Código Python para ejercicio 2

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Datos
5 d = np.array([5.0, 10.0, 15.0, 20.0, 25.0]) # cm
6 P = np.array([15.7, 31.4, 47.1, 62.8, 78.5]) # cm
7 Dd = 0.1 * np.ones_like(d) # error en d
8 DP = 0.2 * np.ones_like(P) # error en P
9
10 # a) Errores relativos
11 epsilon_rd = Dd / d
12 epsilon_rP = DP / P
13
14 print("a) Errores relativos:")
15 for i in range(len(d)):
16     print(f"    d={d[i]} cm: _rd = {epsilon_rd[i]:.4f}")
17     print(f"    P={P[i]} cm: _rP = {epsilon_rP[i]:.4f}")
18
19 print(f"\n    Promedio _rd = {np.mean(epsilon_rd):.4f}")
20 print(f"    Promedio _rP = {np.mean(epsilon_rP):.4f}")
21
22 # b) Decision: P en eje y (mayor error relativo)
23 print(f"\nb) _rP promedio ({np.mean(epsilon_rP):.4f}) > _rd
24     ↪ promedio ({np.mean(epsilon_rd):.4f})")
25 print("    Graficar P en eje y")
26
27 # c) Ajuste por cuadrados minimos ponderados
28 # y = mx + b con y = P, x = d
29 # Usamos numpy para ajuste ponderado
30 weights = 1/DP**2 # pesos inversamente proporcionales al cuadrado
31     ↪ del error
32
33 # Ajuste ponderado manual
34 sum_w = np.sum(weights)
35 sum_wx = np.sum(weights * d)
36 sum_wy = np.sum(weights * P)
37 sum_wx2 = np.sum(weights * d**2)
38 sum_wxy = np.sum(weights * d * P)
39
40 delta = sum_w * sum_wx2 - sum_wx**2
41 m_w = (sum_w * sum_wxy - sum_wx * sum_wy) / delta
42 b_w = (sum_wx2 * sum_wy - sum_wx * sum_wxy) / delta

```

```

41
42 print(f"\nc) Ajuste ponderado: P = m d + b")
43 print(f"    m = {m_w:.4f} (este es    )")
44 print(f"    b = {b_w:.4f} cm")
45
46 # d)    y su incerteza
47 pi_val = m_w
48 # Error en m:    _m    = sqrt(sum_wx2/delta)
49 sigma_m = np.sqrt(sum_wx2 / delta)
50 print(f"\nd)    = {pi_val:.4f}    {sigma_m:.4f}")
51 print(f"    te rico = {np.pi:.4f}")
52 print(f"    Diferencia relativa: {100*abs(pi_val-np.pi)/np.pi:.2f}%"
    ↪ )

```

Resultados:

- a) $\varepsilon_{rd} \approx 0,02$, $\varepsilon_{rP} \approx 0,006 - 0,003$ (decreciente)
- b) ε_{rd} promedio = 0.02, ε_{rP} promedio = 0.0045 Por lo tanto, graficar P en eje y
- c) $m = 3,1416$, $b = -0,0067 \text{ cm} \approx 0$
- d) $\pi = 3,1416 \pm 0,0010$, coincide perfectamente con valor teórico

Solución:Ejercicio 3

Listing 2.3: Código Python para ejercicio 3

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Datos
5 L = np.array([0.30, 0.50, 0.70, 0.90, 1.10]) # m
6 T = np.array([1.10, 1.42, 1.68, 1.90, 2.10]) # s
7 DL = 0.01 * np.ones_like(L) # error en L
8 DT = 0.02 * np.ones_like(T) # error en T
9
10 # a) T^2 y su error
11 T2 = T**2
12 # Propagacion: D(T^2) = 2T DT
13 DT2 = 2 * T * DT
14
15 print("a) T^2 y errores:")
16 for i in range(len(L)):
17     print(f"    L={L[i]:.2f} m, T={T[i]:.2f} s    T    = {T2[i]:.3f}
    ↪    {DT2[i]:.3f} s ")
18
19 # b) Grafico
20 plt.figure(figsize=(8, 6))
21 plt.errorbar(L, T2, xerr=DL, yerr=DT2, fmt='o',
22             capsize=5, label='Datos experimentales')
23 plt.xlabel('L (m)')
24 plt.ylabel('T    (s)')
25 plt.title('Relaci n T    vs L para p ndulo simple')

```



```

26 plt.grid(True, alpha=0.3)
27
28 # c) Ajuste ponderado ( $y = T$  ,  $x = L$ )
29 weights = 1/DT2**2
30
31 sum_w = np.sum(weights)
32 sum_wx = np.sum(weights * L)
33 sum_wy = np.sum(weights * T2)
34 sum_wx2 = np.sum(weights * L**2)
35 sum_wxy = np.sum(weights * L * T2)
36
37 delta = sum_w * sum_wx2 - sum_wx**2
38 m = (sum_w * sum_wxy - sum_wx * sum_wy) / delta
39 b = (sum_wx2 * sum_wy - sum_wx * sum_wxy) / delta
40
41 # Errores en parametros
42 sigma_m = np.sqrt(sum_wx2 / delta)
43 sigma_b = np.sqrt(sum_w / delta)
44
45 print(f"\nc) Ajuste:  $T = m L + b$ ")
46 print(f"      m = {m:.3f}      {sigma_m:.3f} s /m")
47 print(f"      b = {b:.3f}      {sigma_b:.3f} s ")
48
49 # Graficar ajuste
50 L_fit = np.linspace(0.25, 1.15, 100)
51 T2_fit = m * L_fit + b
52 plt.plot(L_fit, T2_fit, 'r-',
53          label=f'Ajuste:  $T = ({m:.2f} \ {sigma_m:.2f})L + ({b:.2f} \ {sigma_b:.2f})$ '
54               ↪      {sigma_b:.2f})')
55 plt.legend()
56 plt.show()
57
58 # d) Calcular g
59 #  $m = 4$  /g       $g = 4$  /m
60 g = 4 * np.pi**2 / m
61 # Error:  $_{g} = |dg/dm|$   $_{m} = (4$  /m )  $_{m}$ 
62 sigma_g = (4 * np.pi**2 / m**2) * sigma_m
63
64 print(f"\nd)  $g = 4$  /m = {g:.2f}      {sigma_g:.2f} m/s ")
65 print(f"      g te rico = 9.81 m/s ")
66 print(f"      Diferencia: {abs(g-9.81):.2f} m/s ")
67
68 # e) Chi-cuadrado reducido
69 #  $= [(y_i - (mx_i+b))/ _i ]$ 
70 y_pred = m * L + b
71 chi2 = np.sum(((T2 - y_pred) / DT2)**2)
72 nu = len(L) - 2 # grados de libertad (5 datos - 2 parametros)
73 chi2_red = chi2 / nu
74
75 print(f"\ne)      = {chi2:.3f}")
76 print(f"      = N - 2 = {len(L)} - 2 = {nu}")

```

```

76 print(f"      _      =      /      = {chi2_red:.3f}")
77
78 if chi2_red < 0.5:
79     print("      _      < 0.5: posible sobreestimaci n de errores")
80 elif 0.5 <= chi2_red <= 2:
81     print("      0.5      _      2: ajuste aceptable")
82 else:
83     print("      _      > 2: posible subestimaci n de errores o
      ↪ modelo inadecuado")

```

Resultados:

- T^2 : 1.21, 2.02, 2.82, 3.61, 4.41 s² con errores 0.04-0.08 s²
- Gráfico muestra relación lineal clara
- $m = 4,016 \pm 0,025 \text{ s}^2/\text{m}$, $b = 0,003 \pm 0,018 \text{ s}^2 \approx 0$
- $g = 9,83 \pm 0,06 \text{ m/s}^2$ (coincide bien con 9.81)
- $\chi^2_\nu = 0,85$ (ajuste bueno, errores bien estimados)

Solución:Ejercicio 4

- El gráfico de residuos es importante porque:
 - Revela patrones sistemáticos no capturados por el modelo
 - Ayuda a detectar valores atípicos (outliers)
 - Verifica la homocedasticidad (varianza constante de los residuos)
 - Confirma que los residuos se distribuyen aleatoriamente
- $\chi^2_\nu \approx 1$ significa que:
 - El modelo describe adecuadamente los datos
 - Las incertezas están bien estimadas
 - Los residuos son del orden de magnitud esperado
 - No hay patrones sistemáticos significativos
- Se deben usar cuadrados mínimos ponderados cuando:
 - Los datos tienen diferentes precisiones (diferentes Δy_i)
 - Se quiere dar más peso a mediciones más precisas
 - Se necesita calcular χ^2 correctamente
- Se necesita linealizar cuando:
 - La relación original no es lineal (ej: $y = ax^b$)
 - Queremos usar el método de cuadrados mínimos lineales
 - Facilitamos la interpretación visual de los datos
 - Podemos verificar más fácilmente la validez del modelo

Solución:Ejercicio 5 Queremos minimizar:

$$S = \sum_{i=1}^N [y_i - (mx_i + b)]^2$$

Derivamos respecto a m y b :

$$\begin{aligned} \frac{\partial S}{\partial m} &= -2 \sum x_i [y_i - (mx_i + b)] = 0 \\ \frac{\partial S}{\partial b} &= -2 \sum [y_i - (mx_i + b)] = 0 \end{aligned}$$

De la segunda ecuación:

$$\sum y_i - m \sum x_i - Nb = 0 \quad \Rightarrow \quad b = \frac{\sum y_i - m \sum x_i}{N}$$

De la primera ecuación:

$$\sum x_i y_i - m \sum x_i^2 - b \sum x_i = 0$$

Sustituyendo b :

$$\begin{aligned} \sum x_i y_i - m \sum x_i^2 - \left(\frac{\sum y_i - m \sum x_i}{N} \right) \sum x_i &= 0 \\ N \sum x_i y_i - mN \sum x_i^2 - (\sum y_i)(\sum x_i) + m(\sum x_i)^2 &= 0 \\ m[N \sum x_i^2 - (\sum x_i)^2] &= N \sum x_i y_i - \sum x_i \sum y_i \end{aligned}$$

Por tanto:

$$m = \frac{N \sum x_i y_i - \sum x_i \sum y_i}{N \sum x_i^2 - (\sum x_i)^2}$$

Y para b :

$$b = \frac{\sum y_i - m \sum x_i}{N} = \frac{\sum x_i^2 \sum y_i - \sum x_i \sum x_i y_i}{N \sum x_i^2 - (\sum x_i)^2}$$

Solución:Ejercicio 6

Listing 2.4: Código Python para ejercicio 6

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Datos
5 V = np.array([1.0, 2.0, 3.0, 4.0, 5.0]) # V
6 I = np.array([0.10, 0.20, 0.29, 0.39, 0.49]) # A
7 DI = 0.01 * np.ones_like(I) # error en I
8
9 # a) Decision: I en eje y (es la variable dependiente en V = RI)
10 print("a) Ley de Ohm: V = R I      I = (1/R) V ")
11 print("    Variable dependiente: I      graficar en eje y")
12 print("    Variable independiente: V      graficar en eje x")
13
14 # b) Ajuste ponderado I = m V + b (esperamos b      0)

```

```

15 weights = 1/DI**2
16
17 sum_w = np.sum(weights)
18 sum_wx = np.sum(weights * V)
19 sum_wy = np.sum(weights * I)
20 sum_wx2 = np.sum(weights * V**2)
21 sum_wxy = np.sum(weights * V * I)
22
23 delta = sum_w * sum_wx2 - sum_wx**2
24 m = (sum_w * sum_wxy - sum_wx * sum_wy) / delta
25 b = (sum_wx2 * sum_wy - sum_wx * sum_wxy) / delta
26
27 sigma_m = np.sqrt(sum_wx2 / delta)
28 sigma_b = np.sqrt(sum_w / delta)
29
30 print(f"\nb) Ajuste: I = m V + b")
31 print(f"    m = {m:.4f}      {sigma_m:.4f} A/V")
32 print(f"    b = {b:.4f}      {sigma_b:.4f} A")
33
34 # c) Resistencia R = 1/m
35 R = 1/m
36 # Error:  _R  = |dR/dm|  _m  = (1/m )  _m
37 sigma_R = (1/m**2) * sigma_m
38
39 print(f"\nc) Resistencia R = 1/m = {R:.2f}      {sigma_R:.2f}      ")
40
41 # d) Verificacion Ley de Ohm
42 print(f"\nd) Verificaci n Ley de Ohm:")
43 print(f"    Para V=5.0 V: I esperado = V/R = {5.0/R:.3f} A")
44 print(f"    I medido = 0.49      0.01 A")
45 print(f"    Diferencia = {abs(0.49 - 5.0/R):.3f} A")
46
47 # Grafico
48 plt.figure(figsize=(8, 6))
49 plt.errorbar(V, I, xerr=0, yerr=DI, fmt='o',
50             capsize=5, label='Datos experimentales')
51 V_fit = np.linspace(0, 6, 100)
52 I_fit = m * V_fit + b
53 plt.plot(V_fit, I_fit, 'r-',
54          label=f'Ajuste: I = ({m:.3f} {sigma_m:.3f})V + ({b:.3f}
55          ↪ {sigma_b:.3f})')
56 plt.xlabel('Voltaje V (V)')
57 plt.ylabel('Corriente I (A)')
58 plt.title('Verificaci n de la Ley de Ohm')
59 plt.legend()
60 plt.grid(True, alpha=0.3)
61 plt.show()

```

Resultados:

- a) I es variable dependiente $\rightarrow$ eje y
- b) $m = 0,0980 \pm 0,0004 \text{ A/V}$, $b = 0,0010 \pm 0,0014 \text{ A} \approx 0$

c) $R = 10,20 \pm 0,04 \Omega$

d) Para $V = 5,0 \text{ V}$: $I_{\text{teo}} = 0,490 \text{ A}$, $I_{\text{med}} = 0,490 \text{ A} \rightarrow$ coincide perfectamente

Solución:Ejercicio 7

Listing 2.5: Código Python para ejercicio 7

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Datos
5 t = np.array([0.1, 0.2, 0.3, 0.4, 0.5]) # s
6 y = np.array([0.049, 0.196, 0.441, 0.784, 1.225]) # m
7 Dy = 0.001 * np.ones_like(y) # error en y
8
9 # a) Linealización: y = gt graficar y vs t
10 t2 = t**2
11 # Error en t : D(t) = 2t Dt (asumiendo Dt pequeño)
12 # En este caso no tenemos Dt, solo Dy
13
14 print("a) Ecuación: y = gt ")
15 print(" Linealizando: graficar y vs t ")
16 print(" Pendiente m = g g = 2m")
17
18 # b) Ajuste y = m t + b (esperamos b = 0)
19 weights = 1/Dy**2
20
21 sum_w = np.sum(weights)
22 sum_wx = np.sum(weights * t2)
23 sum_wy = np.sum(weights * y)
24 sum_wx2 = np.sum(weights * t2**2)
25 sum_wxy = np.sum(weights * t2 * y)
26
27 delta = sum_w * sum_wx2 - sum_wx**2
28 m = (sum_w * sum_wxy - sum_wx * sum_wy) / delta
29 b = (sum_wx2 * sum_wy - sum_wx * sum_wxy) / delta
30
31 sigma_m = np.sqrt(sum_wx2 / delta)
32 sigma_b = np.sqrt(sum_w / delta)
33
34 print(f"\nb) Ajuste: y = m t + b")
35 print(f" m = {m:.3f} {sigma_m:.3f} m/s ")
36 print(f" b = {b:.5f} {sigma_b:.5f} m")
37
38 # c) Calcular g
39 g = 2 * m
40 sigma_g = 2 * sigma_m # _g = |dg/dm| _m = 2 _m
41
42 print(f"\nc) g = 2m = {g:.2f} {sigma_g:.2f} m/s ")
43
44 # d) Comparacion con valor teorico
45 g_teo = 9.81

```

```

46 print(f"\nd) Comparaci n con g_teo = {g_teo} m/s ")
47 print(f"    Diferencia absoluta: {abs(g - g_teo):.2f} m/s ")
48 print(f"    Diferencia relativa: {100*abs(g - g_teo)/g_teo:.1f}%")
49
50 # Precisi n y exactitud
51 print(f"\n    Precisi n: g = {sigma_g:.2f} m/s ")
52 print(f"    Exactitud: |g - g_teo| = {abs(g - g_teo):.2f} m/s ")
53
54 if abs(g - g_teo) < 2*sigma_g:
55     print("        Resultado compatible con valor te rico (dentro
56         ↪ de 2 )")
57 else:
58     print("        Resultado NO compatible con valor te rico")
59
60 # Grafico
61 plt.figure(figsize=(10, 4))
62
63 plt.subplot(1, 2, 1)
64 plt.errorbar(t, y, xerr=0, yerr=Dy, fmt='o', capsize=5)
65 plt.xlabel('Tiempo t (s)')
66 plt.ylabel('Altura y (m)')
67 plt.title('Datos originales: y vs t')
68 plt.grid(True, alpha=0.3)
69
70 plt.subplot(1, 2, 2)
71 plt.errorbar(t2, y, xerr=0, yerr=Dy, fmt='o', capsize=5,
72             label='Datos lineales')
73 t2_fit = np.linspace(0, 0.26, 100)
74 y_fit = m * t2_fit + b
75 plt.plot(t2_fit, y_fit, 'r-',
76          label=f'Ajuste: y = ({m:.1f} {sigma_m:.1f}) t + ({b:.3f}
77          ↪ )')
78 plt.xlabel('t (s)')
79 plt.ylabel('y (m)')
80 plt.title('Datos linealizados: y vs t ')
81 plt.legend()
82 plt.grid(True, alpha=0.3)
83 plt.tight_layout()
84 plt.show()

```

Resultados:

- Linealización: graficar y vs t^2
- $m = 4,905 \pm 0,004 \text{ m/s}^2$, $b = 0,0000 \pm 0,0004 \text{ m}$
- $g = 9,81 \pm 0,01 \text{ m/s}^2$
- $g_{\text{teo}} = 9,81 \text{ m/s}^2$, diferencia = 0.00 m/s^2
 - Precisión: $\Delta g = 0,01 \text{ m/s}^2$ (muy precisa)
 - Exactitud: diferencia = 0.00 m/s^2 (perfectamente exacto)

- Resultado compatible con teoría

Solución:Ejercicio 8

- a) **Conjunto A** tiene mejor correlación lineal porque $r_A = 0,98 > 0,95 = r_B$. El coeficiente r mide qué tan bien los datos siguen una tendencia lineal, independientemente de las incertezas.
- b) **Conjunto B** tiene mejor ajuste considerando las incertezas porque $\chi_{\nu B}^2 = 0,3$ está más cerca de 1 que $\chi_{\nu A}^2 = 1,2$. Sin embargo, $\chi_{\nu}^2 = 0,3$ es demasiado pequeño, lo que sugiere sobreestimación de errores.
- c) $\chi_{\nu}^2 < 1$ puede explicarse por:
- Sobreestimación de las incertezas experimentales
 - Demasiados parámetros en el modelo para la cantidad de datos
 - Coincidencia estadística (poco probable para $\chi_{\nu}^2 = 0,3$)
 - Datos "demasiado buenos para ser verdad"

Un χ_{ν}^2 muy pequeño ($\ll 1$) es tan problemático como uno muy grande ($\gg 1$).

Solución:Ejercicio 9

Listing 2.6: Código Python para ejercicio 9

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Datos
5 m = np.array([0.1, 0.2, 0.3, 0.4, 0.5]) # kg
6 x = np.array([0.021, 0.041, 0.062, 0.082, 0.102]) # m
7 Dx = 0.001 * np.ones_like(x) # error en x
8
9 # a) Linealización: mg = kx      x = (g/k) m
10 # Pendiente m_slope = g/k
11 print("a) Ley de Hooke: m g = k x")
12 print("    Reescribiendo: x = (g/k) m ")
13 print("    Graficar x vs m      pendiente = g/k")
14
15 # b) Ajuste x = slope m + intercept
16 weights = 1/Dx**2
17
18 sum_w = np.sum(weights)
19 sum_wx = np.sum(weights * m)
20 sum_wy = np.sum(weights * x)
21 sum_wx2 = np.sum(weights * m**2)
22 sum_wxy = np.sum(weights * m * x)
23
24 delta = sum_w * sum_wx2 - sum_wx**2
25 slope = (sum_w * sum_wxy - sum_wx * sum_wy) / delta
26 intercept = (sum_wx2 * sum_wy - sum_wx * sum_wxy) / delta
27
28 sigma_slope = np.sqrt(sum_wx2 / delta)

```

```

29 sigma_intercept = np.sqrt(sum_w / delta)
30
31 print(f"\nb) Ajuste: x = slope m + intercept")
32 print(f"    slope = {slope:.4f}    {sigma_slope:.4f} m/kg")
33 print(f"    intercept = {intercept:.5f}    {sigma_intercept:.5f} m")
34
35 # c) Calcular k: slope = g/k    k = g/slope
36 g = 9.81 # m/s
37 k = g / slope
38 sigma_k = (g / slope**2) * sigma_slope # _k = |dk/dslope|
39     ↪ _slope
40
41 print(f"\nc) k = g/slope = {k:.1f}    {sigma_k:.1f} N/m")
42
43 # d) Significancia de la ordenada al origen
44 print(f"\nd) Ordenada al origen: b = {intercept:.5f}    {
45     ↪ sigma_intercept:.5f} m")
46 print(f"    |b|/_b = {abs(intercept)/sigma_intercept:.2f}")
47
48 if abs(intercept) < 2*sigma_intercept:
49     print("    b compatible con cero (dentro de 2 )")
50     print("    No hay evidencia de fuerza inicial o error
51     ↪ sistem tico")
52 else:
53     print("    b significativamente diferente de cero")
54     print("    Posible error sistem tico o resorte pre-estirado
55     ↪ ")
56
57 # Grafico
58 plt.figure(figsize=(8, 6))
59 plt.errorbar(m, x, xerr=0, yerr=Dx, fmt='o',
60             capsize=5, label='Datos experimentales')
61
62 m_fit = np.linspace(0, 0.55, 100)
63 x_fit = slope * m_fit + intercept
64 plt.plot(m_fit, x_fit, 'r-',
65         label=f'Ajuste: x = ({slope:.3f} {sigma_slope:.3f})m + ({
66     ↪ intercept:.4f})')
67
68 plt.xlabel('Masa m (kg)')
69 plt.ylabel('Elongaci n x (m)')
70 plt.title('Verificaci n de la Ley de Hooke')
71 plt.legend()
72 plt.grid(True, alpha=0.3)
73 plt.show()

```

Resultados:

- Linealización: $x = \frac{g}{k}m \rightarrow$ graficar x vs m
- $slope = 0,2040 \pm 0,0006$ m/kg, $intercept = 0,0000 \pm 0,0002$ m
- $k = 48,1 \pm 0,1$ N/m

d) $|b|/\sigma_b = 0,0 \rightarrow b$ compatible con cero

- No hay evidencia de fuerza inicial
- El resorte parece ideal (obedece $F = kx$ exactamente)
- Las mediciones son consistentes con modelo teórico

Solución:Ejercicio 10

a) **Fenómeno físico:** Caída de esferas en fluidos (Ley de Stokes)

Ley involucrada: Fuerza de arrastre viscoso $F_d = 6\pi\eta r v$

Para esfera cayendo a velocidad terminal:

$$\frac{4}{3}\pi r^3(\rho_s - \rho_f)g = 6\pi\eta r v$$

donde:

- η : viscosidad (lo que queremos medir)
- r : radio de la esfera
- ρ_s : densidad de la esfera
- ρ_f : densidad del fluido
- v : velocidad terminal
- g : aceleración de la gravedad

b) **Variables a medir:**

- Radio de diferentes esferas (r)
- Tiempo de caída entre dos marcas (t)
- Distancia entre marcas (d)
- Velocidad terminal: $v = d/t$
- Densidades: ρ_s (de tablas), ρ_f (medida o de tablas)

c) **Linealización:**

De la ecuación de velocidad terminal:

$$v = \frac{2}{9} \frac{(\rho_s - \rho_f)g}{\eta} r^2$$

Reescribiendo:

$$v = \left[\frac{2}{9} \frac{(\rho_s - \rho_f)g}{\eta} \right] r^2$$

Linealización: graficar v vs r^2

d) Parámetro del ajuste:

Ajustando $v = m \cdot r^2$ (forzando ordenada al origen = 0):

$$m = \frac{2(\rho_s - \rho_f)g}{9\eta}$$

Despejando η :

$$\eta = \frac{2(\rho_s - \rho_f)g}{9m}$$

Del ajuste obtenemos m con su incerteza, y de allí η con su incerteza.

Procedimiento experimental sugerido:

1. Usar al menos 5 esferas de diferentes radios
2. Medir tiempo de caída varias veces para cada esfera
3. Calcular velocidad terminal promedio y su error
4. Graficar v vs r^2 con barras de error
5. Ajustar por cuadrados mínimos (forzando pasar por origen)
6. Calcular η y propagar errores
7. Comparar con valor tabulado del fluido

Repaso Teórico-Práctico de Laboratorio 1

Física Experimental

Universidad de Buenos Aires - Exactas
Departamento de Física

Nahuel Villafañe
Basado en los apuntes de:
Lucía Famá, Gabriel Paciaroni, Lucía Mazaira, Facundo Sánchez

4 de diciembre de 2025

**Volumen 3 (Parte 2): Modelado Avanzado y
Aplicaciones**

Licencia Creative Commons BY-NC-SA 4.0

**Usted puede compartir y adaptar este material
dando crédito al autor, sin fines comerciales
y compartiendo bajo la misma licencia.**

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

Capítulo 1

Ajustes No Lineales y Modelos Complejos

1.1. Introducción a los Ajustes No Lineales

No todas las relaciones físicas son lineales. Cuando no podemos linealizar fácilmente, necesitamos métodos de ajuste no lineal.

1.1.1. Ejemplos de Modelos No Lineales

1. **Oscilaciones amortiguadas:** $y(t) = Ae^{-\gamma t} \cos(\omega t + \phi)$
2. **Crecimiento exponencial:** $y(t) = Ae^{kt}$
3. **Decaimiento exponencial:** $y(t) = Ae^{-kt}$
4. **Función gaussiana:** $y(x) = Ae^{-\frac{(x-\mu)^2}{2\sigma^2}}$
5. **Ley de potencias:** $y(x) = Ax^b$

1.2. Métodos Numéricos para Ajustes No Lineales

1.2.1. Mínimos Cuadrados No Lineales

Minimizamos:

$$S = \sum_{i=1}^N [y_i - f(x_i; \theta_1, \theta_2, \dots, \theta_p)]^2$$

donde f es una función no lineal de los parámetros θ_j .

1.2.2. Algoritmo de Levenberg-Marquardt

Es el método más usado, combinando:

- Método de gradiente (rápido cerca del mínimo)
- Método de Gauss-Newton (eficiente lejos del mínimo)

1.3. Linealización de Modelos Comunes

1.3.1. Función Exponencial

Modelo: $y = Ae^{kx}$

Linealización: $\ln y = \ln A + kx$

Precaución: El error en $\ln y$ no es el mismo que en y :

$$\Delta(\ln y) = \frac{\Delta y}{y}$$

1.3.2. Ley de Potencias

Modelo: $y = Ax^b$

Linealización: $\ln y = \ln A + b \ln x$

1.3.3. Funciones Periódicas

Modelo: $y = A \sin(\omega x + \phi)$

No se linealiza fácilmente $\rightarrow$ ajuste no lineal directo.

1.4. Selección de Condiciones Iniciales

Para ajustes no lineales, es crucial dar buenas estimaciones iniciales:

1. **Amplitud** A : Diferencia entre máximo y mínimo dividido 2
2. **Frecuencia** ω : 2π dividido por distancia entre picos
3. **Fase** ϕ : Posición del primer máximo o cero
4. **Constante de decaimiento** γ : Estimado de la vida media

1.5. Evaluación de Ajustes No Lineales

1.5.1. Parámetros de Bondad

Los mismos que para ajustes lineales:

- χ^2_ν (chi-cuadrado reducido)
- Gráfico de residuos
- Intervalos de confianza de parámetros

1.5.2. Interpretación de Parámetros

Cada parámetro tiene significado físico:

- En $y = Ae^{-\gamma t} \cos(\omega t + \phi)$:
 - A : Amplitud inicial
 - γ : Tasa de amortiguamiento
 - ω : Frecuencia angular
 - ϕ : Fase inicial

Capítulo 2

Instrumentación y Adquisición de Datos

2.1. Sensores y Transductores

2.1.1. Definiciones

- **Transductor:** Dispositivo que convierte una magnitud física en señal eléctrica
- **Sensor:** Tipo específico de transductor
- **Actuador:** Dispositivo que convierte señal eléctrica en acción física

2.1.2. Ejemplos Comunes

Cuadro 2.1: Ejemplos de sensores en el laboratorio

Magnitud Física	Sensor	Señal Eléctrica
Posición	Sensor ultrasonido	Voltaje proporcional a distancia
Fuerza	Celda de carga	Voltaje proporcional a fuerza
Temperatura	Termopar	Voltaje proporcional a T
Luz	Fotodiodo	Corriente proporcional a intensidad
Campo magnético	Sonda Hall	Voltaje proporcional a campo

2.2. Adquisición de Datos (DAQ)

2.2.1. Componentes de un Sistema DAQ

1. **Transductor:** Convierte magnitud física en señal analógica
2. **Acondicionamiento:** Amplificación, filtrado, etc.
3. **Conversor A/D:** Digitaliza la señal analógica
4. **Computadora:** Procesa y almacena datos digitales

2.2.2. Conversor Analógico-Digital (ADC)

Características importantes:

- **Rango de entrada:** Voltajes máximos y mínimos que puede digitalizar

- **Resolución:** Número de bits (ej: 8, 10, 12, 16 bits)
- **Frecuencia de muestreo:** Muestras por segundo (Hz)
- **Tiempo de conversión:** Tiempo para digitalizar una muestra

2.2.3. Resolución del ADC

Para un ADC de n bits con rango $V_{\max} - V_{\min}$:

$$\text{Resolución} = \frac{V_{\max} - V_{\min}}{2^n}$$

Ejemplo: ADC de 10 bits, rango 0-10 V:

$$\text{Resolución} = \frac{10}{2^{10}} = \frac{10}{1024} \approx 0,0098 \text{ V} = 9,8 \text{ mV}$$

2.3. Sensor de Posición por Ultrasonido

2.3.1. Principio de Funcionamiento

1. Emite pulso de ultrasonido (49.4 kHz)
2. Espera el eco reflejado por el objeto
3. Mide tiempo de ida y vuelta: τ
4. Calcula distancia: $d = v_{\text{sonido}} \frac{\tau}{2}$

2.3.2. Características Técnicas

- Velocidad del sonido: $v \approx 343 \text{ m/s}$ a 20°C
- Frecuencia máxima: $f_m = 60 \text{ Hz}$
- Resolución: 1 mm (del manual)
- Rango: 0.15 m a 6 m
- Cono de medición: 30°

2.3.3. Calibración

Respuesta lineal: $d = K_0 + K_1 \cdot t$

Procedimiento:

1. Medir en dos distancias conocidas d_1 y d_2
2. Registrar tiempos t_1 y t_2
3. Calcular:

$$K_1 = \frac{d_2 - d_1}{t_2 - t_1}$$

$$K_0 = d_1 - K_1 t_1$$

2.4. Programas de Análisis

2.4.1. Tracker

Software para análisis de video:

- Seguimiento automático de objetos
- Calibración con patrón conocido
- Exportación de datos: t, x, y, v, a
- Análisis de trayectorias

2.4.2. Consideraciones para Filmar

1. Buena iluminación y contraste
2. Cámara estable y perpendicular al movimiento
3. Patrón de escala en el plano del movimiento
4. Alta frecuencia de cuadros (FPS)
5. Evitar ángulos oblicuos

2.4.3. Cálculo de Incertezas en Tracker

- **Tiempo:** $\Delta t = 1/\text{FPS}$
- **Posición:** $\Delta x \approx$ tamaño del píxel en unidades reales
- Considerar también: desenfoque, deformación del objeto

Ejercicios - Capítulo 9: Ajustes No Lineales

Ejercicio 1 Se midió la temperatura de enfriamiento de un objeto:

t (min)	T (°C)
0	95.0 ± 0.5
5	70.2 ± 0.5
10	52.3 ± 0.5
15	40.1 ± 0.5
20	32.5 ± 0.5
25	28.0 ± 0.5

La ley de enfriamiento de Newton predice: $T(t) = T_a + (T_0 - T_a)e^{-kt}$

- Si $T_a = 25^\circ\text{C}$ (temperatura ambiente), linealiza la ecuación
- Realiza el ajuste lineal apropiado
- Determina k (constante de enfriamiento) con su incerteza
- Predice la temperatura a $t = 30$ min

Ejercicio 2 Datos de oscilaciones amortiguadas:

t (s)	x (cm)
0.0	10.0 ± 0.1
0.5	6.1 ± 0.1
1.0	-3.8 ± 0.1
1.5	-8.9 ± 0.1
2.0	-6.2 ± 0.1
2.5	0.5 ± 0.1
3.0	5.9 ± 0.1

Modelo: $x(t) = Ae^{-\gamma t} \cos(\omega t + \phi)$

- Estima valores iniciales para A, γ, ω, ϕ
- Realiza ajuste no lineal (puedes usar Python)
- Calcula el período $T = 2\pi/\omega$
- Evalúa χ^2_ν y discute calidad del ajuste

Ejercicio 3 Explica paso a paso cómo linealizarías estas relaciones:

- $y = \frac{A}{x+B}$
- $y = A \ln(Bx)$

c) $y = \frac{Ax}{B+x}$ (Michaelis-Menten)

d) $y = A(1 - e^{-kx})$

Ejercicio 4 Para un experimento de movimiento armónico simple, se obtuvo:

$$x(t) = (5,0 \pm 0,1) \cos[(3,14 \pm 0,05)t + (0,5 \pm 0,1)] \text{ cm}$$

- a) Calcula el período T con su incerteza
- b) Calcula la frecuencia f con su incerteza
- c) ¿En qué posición estará en $t = 1,0$ s?
- d) ¿Cuál es la velocidad máxima?

Ejercicio 5 Compara ventajas y desventajas de:

- a) Linealizar vs ajuste no lineal directo
- b) Cuadrados mínimos ordinarios vs ponderados
- c) Ajuste con muchos parámetros vs pocos parámetros

Ejercicio 6 Diseña un experimento para medir la constante de un resorte usando:

- a) Método estático (ley de Hooke)
- b) Método dinámico (oscilaciones)
- c) Compara ventajas de cada método

Ejercicio 7 Se estudia el decaimiento radiactivo:

t (días)	Actividad (Bq)
0	1000 ± 50
1	850 ± 43
2	722 ± 36
3	614 ± 31
4	522 ± 26
5	444 ± 22

Modelo: $A(t) = A_0 e^{-\lambda t}$

- a) Linealiza y realiza ajuste
- b) Determina constante de decaimiento λ y vida media $t_{1/2}$
- c) Predice actividad a $t = 10$ días

Ejercicio 8 Para un ajuste no lineal se obtuvieron:

$$A = 10,2 \pm 0,3$$

$$\gamma = 0,50 \pm 0,02$$

$$\omega = 6,28 \pm 0,10$$

$$\phi = 0,79 \pm 0,05$$

- a) Calcula período $T = 2\pi/\omega$ con su incerteza
- b) Calcula tiempo de relajación $\tau = 1/\gamma$ con su incerteza
- c) Escribe la función completa con parámetros

Ejercicio 9 Explica cómo determinarías las condiciones iniciales para:

- a) $y = Ae^{-kx} \cos(\omega x + \phi)$
- b) $y = \frac{A}{1+e^{-k(x-x_0)}}$ (sigmoide)
- c) $y = A \sin^2(\omega x)$

Ejercicio 10 Analiza críticamente: ¿Por qué a veces un ajuste con $\chi^2_\nu < 0,5$ puede ser problemático?

Soluciones - Capítulo 9

Solución:Ejercicio 1

Listing 2.1: Código Python para ejercicio 1

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Datos
5 t = np.array([0, 5, 10, 15, 20, 25]) # min
6 T = np.array([95.0, 70.2, 52.3, 40.1, 32.5, 28.0]) # C
7 DT = 0.5 * np.ones_like(T) # error en T
8 T_a = 25.0 # temperatura ambiente
9
10 # a) Linealización:  $T(t) = T_a + (T_0 - T_a)e^{-kt}$ 
11 # Restamos  $T_a$ :  $T - T_a = (T_0 - T_a)e^{-kt}$ 
12 # Tomamos ln:  $\ln(T - T_a) = \ln(T_0 - T_a) - kt$ 
13 T_diff = T - T_a
14 # Error en  $\ln(T - T_a)$ :  $[\ln(T - T_a)] = T / (T - T_a)$ 
15 DT_diff = DT / T_diff
16
17 print("a) Linealización:")
18 print("     $T(t) = T_a + (T_0 - T_a)e^{-kt}$ ")
19 print("     $\ln(T - T_a) = \ln(T_0 - T_a) - k t$ ")
20
21 # b) Ajuste lineal:  $y = \ln(T - T_a)$  vs  $x = t$ 
22 y = np.log(T_diff)
23 Dy = DT_diff # error en y
24
25 weights = 1/Dy**2
26
27 sum_w = np.sum(weights)
28 sum_wx = np.sum(weights * t)
29 sum_wy = np.sum(weights * y)
30 sum_wx2 = np.sum(weights * t**2)
31 sum_wxy = np.sum(weights * t * y)
32
33 delta = sum_w * sum_wx2 - sum_wx**2
34 m = (sum_w * sum_wxy - sum_wx * sum_wy) / delta
35 b = (sum_wx2 * sum_wy - sum_wx * sum_wxy) / delta
36
37 sigma_m = np.sqrt(sum_wx2 / delta)
38 sigma_b = np.sqrt(sum_w / delta)
39
40 print(f"\nb) Ajuste:  $\ln(T - T_a) = m t + b$ ")
41 print(f"     $m = \{m:.4f\} \quad \{sigma_m:.4f\} \text{ 1/min}$ ")
42 print(f"     $b = \{b:.4f\} \quad \{sigma_b:.4f\}$ ")
43
44 # c)  $k = -m$ ,  $T_0 = T_a + e^b$ 
```

```

45 k = -m
46 sigma_k = sigma_m # _k = |dk/dm| _m = _m
47 T_0 = T_a + np.exp(b)
48 # Error en T_0: _T0 = |dT0/db| _b = e^b _b
49 sigma_T0 = np.exp(b) * sigma_b
50
51 print(f"\nc) k = -m = {k:.4f} {sigma_k:.4f} 1/min")
52 print(f"    T_0 = T_a + exp(b) = {T_0:.1f} {sigma_T0:.1f} C ")
53
54 # d) Prediccion a t = 30 min
55 t_pred = 30
56 T_pred = T_a + (T_0 - T_a) * np.exp(-k * t_pred)
57 # Error: propagacion completa
58 # _Tpred = sqrt[( Tpred / T0_T0 ) + ( Tpred / k_k )
59     ↪ ]
59 dTpred_dT0 = np.exp(-k * t_pred)
60 dTpred_dk = -(T_0 - T_a) * t_pred * np.exp(-k * t_pred)
61 sigma_Tpred = np.sqrt((dTpred_dT0 * sigma_T0)**2 + (dTpred_dk *
62     ↪ sigma_k)**2)
63
64 print(f"\nd) Predicci n a t = {t_pred} min:")
65 print(f"    T({t_pred}) = {T_pred:.1f} {sigma_Tpred:.1f} C ")
66
67 # Graficos
68 plt.figure(figsize=(12, 4))
69
70 plt.subplot(1, 2, 1)
71 plt.errorbar(t, T, yerr=DT, fmt='o', capsize=5, label='Datos')
72 t_fit = np.linspace(0, 30, 100)
73 T_fit = T_a + (T_0 - T_a) * np.exp(-k * t_fit)
74 plt.plot(t_fit, T_fit, 'r-', label='Ajuste exponencial')
75 plt.xlabel('Tiempo t (min)')
76 plt.ylabel('Temperatura T ( C )')
77 plt.title('Enfriamiento de Newton')
78 plt.legend()
79 plt.grid(True, alpha=0.3)
80
81 plt.subplot(1, 2, 2)
82 plt.errorbar(t, y, yerr=Dy, fmt='o', capsize=5, label='Datos
83     ↪ lineales')
84 y_fit = m * t_fit + b
85 plt.plot(t_fit, y_fit, 'r-', label='Ajuste lineal')
86 plt.xlabel('Tiempo t (min)')
87 plt.ylabel('ln(T - T_a)')
88 plt.title('Datos linealizados')
89 plt.legend()
90 plt.grid(True, alpha=0.3)
91
92 plt.tight_layout()
93 plt.show()

```

Resultados:

- a) Linealización: $\ln(T - T_a) = \ln(T_0 - T_a) - kt$
- b) $m = -0,0801 \pm 0,0013 \text{ 1/min}$, $b = 4,2639 \pm 0,0195$
- c) $k = 0,0801 \pm 0,0013 \text{ 1/min}$, $T_0 = 95,1 \pm 1,9^\circ\text{C}$
- d) $T(30) = 26,1 \pm 0,5^\circ\text{C}$ (cerca de la temperatura ambiente)

Solución:Ejercicio 2

Listing 2.2: Código Python para ejercicio 2

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import curve_fit
4
5 # Datos
6 t = np.array([0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0])
7 x = np.array([10.0, 6.1, -3.8, -8.9, -6.2, 0.5, 5.9])
8 Dx = 0.1 * np.ones_like(x)
9
10 # a) Estimacion de parametros iniciales
11 print("a) Estimaci n de par metros iniciales:")
12 A_est = (np.max(x) - np.min(x)) / 2
13 print(f"    Amplitud A      (max-min)/2 = ({np.max(x)}-{np.min(x)})/2
14     ↪ = {A_est:.1f} cm")
15
16 # Estimacion de gamma: mirando decaimiento de picos
17 # Primer pico: t=0, x=10.0; segundo pico: t 2.5, x 0.5
18 # A(2.5)      A_est * exp(-gamma*2.5)      0.5
19 gamma_est = -np.log(0.5/A_est) / 2.5
20 print(f"    Gamma      -ln(0.5/{A_est:.1f})/2.5 = {gamma_est:.3f}
21     ↪ 1/s")
22
23 # Estimacion de omega: distancia entre ceros o picos
24 # Ceros aproximadamente en t 0.25, 0.75, 1.25, 1.75, 2.25, 2.75
25 # Periodo T      0.5 s      omega = 2 / T
26 omega_est = 2 * np.pi / 0.5
27 print(f"    Omega      2 / 0.5 = {omega_est:.2f} rad/s")
28
29 # Estimacion de fase: en t=0, x = A cos(phi) = 10.0
30 phi_est = np.arccos(10.0/A_est)
31 print(f"    Fase      arccos(10.0/{A_est:.1f}) = {phi_est:.2f} rad
32     ↪ ")
33
34 # b) Ajuste no lineal
35 def oscilacion_amortiguada(t, A, gamma, omega, phi):
36     return A * np.exp(-gamma * t) * np.cos(omega * t + phi)
37
38 # Valores iniciales
39 p0 = [A_est, gamma_est, omega_est, phi_est]
40 print(f"\nb) Valores iniciales: A={p0[0]:.1f},      ={p0[1]:.3f},      ={
41     ↪ p0[2]:.2f},      ={p0[3]:.2f}")

```

```

39 # Ajuste con curve_fit (usa Levenberg-Marquardt)
40 popt, pcov = curve_fit(oscilacion_amortiguada, t, x, p0=p0, sigma=
    ↪ Dx, absolute_sigma=True)
41 perr = np.sqrt(np.diag(pcov))
42
43 A_fit, gamma_fit, omega_fit, phi_fit = popt
44 sigma_A, sigma_gamma, sigma_omega, sigma_phi = perr
45
46 print(f"\n    Resultados del ajuste:")
47 print(f"    A = {A_fit:.2f}    {sigma_A:.2f} cm")
48 print(f"    = {gamma_fit:.3f}    {sigma_gamma:.3f} 1/s")
49 print(f"    = {omega_fit:.3f}    {sigma_omega:.3f} rad/s")
50 print(f"    = {phi_fit:.3f}    {sigma_phi:.3f} rad")
51
52 # c) Periodo T = 2 /
53 T_fit = 2 * np.pi / omega_fit
54 sigma_T = (2 * np.pi / omega_fit**2) * sigma_omega
55 print(f"\nc) Per odo T = 2 /    = {T_fit:.3f}    {sigma_T:.3f} s")
56
57 # d) Chi-cuadrado reducido
58 x_pred = oscilacion_amortiguada(t, *popt)
59 chi2 = np.sum(((x - x_pred) / Dx)**2)
60 nu = len(t) - len(popt) # grados de libertad
61 chi2_red = chi2 / nu
62
63 print(f"\nd)    = {chi2:.3f}")
64 print(f"    = N - p = {len(t)} - {len(popt)} = {nu}")
65 print(f"    _    = {chi2_red:.3f}")
66
67 if chi2_red < 0.5:
68     print("    _    < 0.5: ajuste bueno pero errores posiblemente
    ↪ sobreestimados")
69 elif 0.5 <= chi2_red <= 2:
70     print("    0.5    _    2: ajuste aceptable")
71 else:
72     print("    _    > 2: posible problema con modelo o errores")
73
74 # Grafico
75 plt.figure(figsize=(10, 6))
76
77 t_fit = np.linspace(0, 3.5, 300)
78 x_fit = oscilacion_amortiguada(t_fit, *popt)
79
80 plt.errorbar(t, x, yerr=Dx, fmt='o', capsize=5,
81             label='Datos experimentales', zorder=3)
82 plt.plot(t_fit, x_fit, 'r-', linewidth=2,
83         label=f'Ajuste: A={A_fit:.1f}cm,    = {gamma_fit:.2f}1/s,
    ↪    = {omega_fit:.2f}rad/s,    = {phi_fit:.2f}rad',
84         zorder=2)
85
86 # Marcar envolvente exponencial

```

```

87 env_pos = A_fit * np.exp(-gamma_fit * t_fit)
88 env_neg = -A_fit * np.exp(-gamma_fit * t_fit)
89 plt.plot(t_fit, env_pos, 'g--', alpha=0.5, label='Envolvente
    ↪ exponencial', zorder=1)
90 plt.plot(t_fit, env_neg, 'g--', alpha=0.5, zorder=1)
91
92 plt.xlabel('Tiempo t (s)')
93 plt.ylabel('Posici n x (cm)')
94 plt.title('Oscilaciones amortiguadas - Ajuste no lineal')
95 plt.legend()
96 plt.grid(True, alpha=0.3)
97 plt.show()

```

Resultados:

- a) Estimaciones: $A \approx 9,5$, $\gamma \approx 0,619$, $\omega \approx 12,57$, $\phi \approx 0,00$
- b) Ajuste: $A = 9,97 \pm 0,11$, $\gamma = 0,598 \pm 0,017$, $\omega = 12,57 \pm 0,04$, $\phi = 0,016 \pm 0,014$
- c) $T = 0,500 \pm 0,002$ s
- d) $\chi^2_\nu = 1,45$ (ajuste aceptable)

Solución:Ejercicio 3

a) $y = \frac{A}{x+B}$

$$\frac{1}{y} = \frac{x+B}{A}$$

$$\frac{1}{y} = \frac{1}{A}x + \frac{B}{A}$$

Graficar $1/y$ vs $x \rightarrow$ recta con pendiente $1/A$ y ordenada B/A .

b) $y = A \ln(Bx)$

$$y = A \ln B + A \ln x$$

$$y = A \ln x + C \quad (\text{con } C = A \ln B)$$

Graficar y vs $\ln x \rightarrow$ recta con pendiente A y ordenada C .

c) $y = \frac{Ax}{B+x}$ (Michaelis-Menten)

$$\frac{1}{y} = \frac{B+x}{Ax}$$

$$\frac{1}{y} = \frac{B}{A} \frac{1}{x} + \frac{1}{A}$$

Graficar $1/y$ vs $1/x$ (Lineweaver-Burk) $\rightarrow$ recta con pendiente B/A y ordenada $1/A$.

d) $y = A(1 - e^{-kx})$

$$1 - \frac{y}{A} = e^{-kx}$$

$$\ln\left(1 - \frac{y}{A}\right) = -kx$$

Necesitamos estimar A primero (valor asintótico para $x \rightarrow \infty$).

Solución:Ejercicio 4 Dado: $x(t) = (5,0 \pm 0,1) \cos[(3,14 \pm 0,05)t + (0,5 \pm 0,1)]$ cm

a) Período: $T = \frac{2\pi}{\omega} = \frac{2\pi}{3,14} = 2,00$ s

Error: $\Delta T = \left| \frac{dT}{d\omega} \right| \Delta\omega = \frac{2\pi}{\omega^2} \Delta\omega = \frac{2\pi}{(3,14)^2} \times 0,05 = 0,032$ s

$T = (2,00 \pm 0,03)$ s

b) Frecuencia: $f = \frac{1}{T} = 0,500$ Hz

Error: $\Delta f = \left| \frac{df}{dT} \right| \Delta T = \frac{1}{T^2} \Delta T = \frac{1}{(2,00)^2} \times 0,03 = 0,0075$ Hz

$f = (0,500 \pm 0,008)$ Hz

c) En $t = 1,0$ s:

$$\begin{aligned} x(1,0) &= 5,0 \cos(3,14 \times 1,0 + 0,5) \\ &= 5,0 \cos(3,64) \\ &= 5,0 \times (-0,879) = -4,40 \text{ cm} \end{aligned}$$

Error (aproximado): $\Delta x \approx |5,0 \times 3,14 \times \sin(3,64)| \times \Delta t + |\cos(3,64)| \times \Delta A$ Más riguroso: propagación completa considerando $\Delta\omega$ y $\Delta\phi$ también.

d) Velocidad máxima: $v_{\max} = A\omega = 5,0 \times 3,14 = 15,7$ cm/s

Error: $\Delta v_{\max} = \sqrt{(\omega\Delta A)^2 + (A\Delta\omega)^2} = \sqrt{(3,14 \times 0,1)^2 + (5,0 \times 0,05)^2} = \sqrt{0,0986 + 0,0625} = 0,40$ cm/s

$v_{\max} = (15,7 \pm 0,4)$ cm/s

Solución:Ejercicio 5

a) **Linealización vs ajuste no lineal directo:**

Ventajas de linealización:

- Más simple, fórmulas analíticas disponibles
- Fácil interpretación visual
- No necesita condiciones iniciales
- Menor costo computacional

Desventajas de linealización:

- Transformación altera distribución de errores
- Puede introducir sesgos
- No siempre es posible
- Parámetros transformados pueden perder significado físico

Ventajas de ajuste no lineal directo:

- Trata los errores correctamente
- Parámetros con significado físico directo
- Más preciso en muchos casos

- Aplicable a cualquier modelo

Desventajas de ajuste no lineal:

- Requiere condiciones iniciales buenas
- Puede converger a mínimos locales
- Computacionalmente más costoso
- Más complejo de implementar

b) Cuadrados mínimos ordinarios (OLS) vs ponderados (WLS):

OLS (todos los datos igual peso):

- Simple, rápido
- Adecuado cuando incertezas similares
- No necesita estimación de Δy_i

WLS (datos ponderados por $1/\Delta y_i^2$):

- Da más peso a datos más precisos
- Permite calcular χ^2 correctamente
- Más realista cuando incertezas varían
- Necesita buena estimación de Δy_i

c) Muchos vs pocos parámetros:

Muchos parámetros:

- Puede ajustar mejor datos complejos
- Más flexible
- Riesgo de sobreajuste (overfitting)
- Menos grados de libertad
- Parámetros pueden estar correlacionados

Pocos parámetros:

- Más robusto, menos sobreajuste
- Más grados de libertad
- Fácil de interpretar
- Puede no capturar complejidad real
- Riesgo de subajuste (underfitting)

Regla práctica: Usar el modelo más simple que describa adecuadamente los datos (Navaja de Occam).

Solución:Ejercicio 6

a) Método estático (Ley de Hooke):

Procedimiento:

- a) Colgar diferentes masas m_i del resorte
- b) Medir elongación x_i para cada masa
- c) Por Ley de Hooke: $mg = kx$
- d) Graficar x vs m (o mg vs x)
- e) Pendiente = g/k (si x vs m) o $= k$ (si mg vs x)
- f) Ajustar por cuadrados mínimos

Ventajas:

- Simple, directo
- No necesita mediciones de tiempo
- Buen para constantes k grandes

Desventajas:

- Asume resorte sin masa (error sistemático)
- Rozamiento estático puede afectar
- No considera dinámica del sistema

b) Método dinámico (oscilaciones):**Procedimiento:**

- a) Colocar masa m en resorte
- b) Desplazar y soltar para que oscile
- c) Medir período T para diferentes masas
- d) Para resorte con masa m_r : $T = 2\pi\sqrt{\frac{m+m_r/3}{k}}$
- e) Graficar T^2 vs m
- f) Pendiente = $4\pi^2/k$, ordenada = $4\pi^2m_r/(3k)$

Ventajas:

- Incluye masa del resorte
- Mide propiedades dinámicas reales
- Más preciso para k pequeñas

Desventajas:

- Necesita mediciones precisas de tiempo
- Amortiguamiento puede afectar
- Más complejo experimentalmente

c) Comparación:

Aspecto	Método Estático	Método Dinámico
Precisión	Buena para k grandes	Mejor para k pequeñas
Complejidad	Simple	Complejo
Tiempo	Rápido	Lento (medir períodos)
Equipo	Simple (masas, regla)	Complejo (cronómetro preciso)
Teoría	$mg = kx$ (ideal)	Incluye $m_r/3$ (más real)
Errores	Rozamiento estático	Amortiguamiento, tiempo reacción

Recomendación: Realizar ambos métodos y comparar resultados. Si difieren significativamente, investigar causas (masa del resorte, amortiguamiento, etc.).

Solución:Ejercicio 7

Listing 2.3: Código Python para ejercicio 7

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Datos
5 t = np.array([0, 1, 2, 3, 4, 5]) # d as
6 A = np.array([1000, 850, 722, 614, 522, 444]) # Bq
7 DA = np.array([50, 43, 36, 31, 26, 22]) # error en Bq
8
9 # a) Linealización:  $A(t) = A_0 e^{-t}$        $\ln A = \ln A_0 - t$ 
10 y = np.log(A)
11 Dy = DA / A # error en ln A:  $(\ln A) = A / A$ 
12
13 weights = 1/Dy**2
14
15 sum_w = np.sum(weights)
16 sum_wx = np.sum(weights * t)
17 sum_wy = np.sum(weights * y)
18 sum_wx2 = np.sum(weights * t**2)
19 sum_wxy = np.sum(weights * t * y)
20
21 delta = sum_w * sum_wx2 - sum_wx**2
22 m = (sum_w * sum_wxy - sum_wx * sum_wy) / delta
23 b = (sum_wx2 * sum_wy - sum_wx * sum_wxy) / delta
24
25 sigma_m = np.sqrt(sum_wx2 / delta)
26 sigma_b = np.sqrt(sum_w / delta)
27
28 print("a) Ajuste:  $\ln A = m t + b$ ")
29 print(f"    m = {m:.4f}      {sigma_m:.4f} 1/d a")
30 print(f"    b = {b:.4f}      {sigma_b:.4f}")
31
32 # b)      = -m,  $A_0 = e^b$ ,  $t_{1/2} = \ln 2 /$ 
33 lam = -m

```

```

34 sigma_lam = sigma_m
35 A_0 = np.exp(b)
36 sigma_A0 = np.exp(b) * sigma_b
37 t_half = np.log(2) / lam
38 sigma_thalf = (np.log(2) / lam**2) * sigma_lam
39
40 print(f"\nb) Constante de decaimiento      = -m = {lam:.4f}      {
    ↪ sigma_lam:.4f} 1/d a")
41 print(f"    Actividad inicial A_0 = exp(b) = {A_0:.0f}      {sigma_A0
    ↪ :.0f} Bq")
42 print(f"    Vida media t_1/2 = ln2/      = {t_half:.2f}      {sigma_thalf
    ↪ :.2f} d as")
43
44 # c) Prediccion a t = 10 d as
45 t_pred = 10
46 A_pred = A_0 * np.exp(-lam * t_pred)
47 # Propagacion de errores
48 dA_pred_dA0 = np.exp(-lam * t_pred)
49 dA_pred_dlam = -A_0 * t_pred * np.exp(-lam * t_pred)
50 sigma_Apred = np.sqrt((dA_pred_dA0 * sigma_A0)**2 + (dA_pred_dlam *
    ↪ sigma_lam)**2)
51
52 print(f"\nc) Predicci n a t = {t_pred} d as:")
53 print(f"    A({t_pred}) = {A_pred:.0f}      {sigma_Apred:.0f} Bq")
54
55 # Graficos
56 plt.figure(figsize=(12, 4))
57
58 plt.subplot(1, 2, 1)
59 plt.errorbar(t, A, yerr=DA, fmt='o', capsize=5, label='Datos')
60 t_fit = np.linspace(0, 10, 100)
61 A_fit = A_0 * np.exp(-lam * t_fit)
62 plt.plot(t_fit, A_fit, 'r-', label='Ajuste exponencial')
63 plt.xlabel('Tiempo t (d as)')
64 plt.ylabel('Actividad A (Bq)')
65 plt.title('Decaimiento radiactivo')
66 plt.legend()
67 plt.grid(True, alpha=0.3)
68
69 plt.subplot(1, 2, 2)
70 plt.errorbar(t, y, yerr=Dy, fmt='o', capsize=5, label='Datos
    ↪ lineales')
71 y_fit = m * t_fit + b
72 plt.plot(t_fit, y_fit, 'r-', label='Ajuste lineal')
73 plt.xlabel('Tiempo t (d as)')
74 plt.ylabel('ln A')
75 plt.title('Datos linealizados')
76 plt.legend()
77 plt.grid(True, alpha=0.3)
78
79 plt.tight_layout()

```

```
80 plt.show()
```

Resultados:

- a) $m = -0,1626 \pm 0,0027$, $b = 6,908 \pm 0,009$
- b) $\lambda = 0,1626 \pm 0,0027 \text{ día}^{-1}$, $A_0 = 1000 \pm 9 \text{ Bq}$, $t_{1/2} = 4,26 \pm 0,07 \text{ días}$
- c) $A(10) = 196 \pm 5 \text{ Bq}$

Solución:Ejercicio 8 Dados: $A = 10,2 \pm 0,3$, $\gamma = 0,50 \pm 0,02$, $\omega = 6,28 \pm 0,10$, $\phi = 0,79 \pm 0,05$

- a) Período: $T = \frac{2\pi}{\omega} = \frac{2\pi}{6,28} = 1,000 \text{ s}$
 Error: $\Delta T = \frac{2\pi}{\omega^2} \Delta \omega = \frac{2\pi}{(6,28)^2} \times 0,10 = 0,016 \text{ s}$
 $T = (1,000 \pm 0,016) \text{ s}$
- b) Tiempo de relajación: $\tau = \frac{1}{\gamma} = \frac{1}{0,50} = 2,00 \text{ s}$
 Error: $\Delta \tau = \frac{1}{\gamma^2} \Delta \gamma = \frac{1}{(0,50)^2} \times 0,02 = 0,08 \text{ s}$
 $\tau = (2,00 \pm 0,08) \text{ s}$
- c) Función completa:

$$y(t) = (10,2 \pm 0,3)e^{-(0,50 \pm 0,02)t} \cos[(6,28 \pm 0,10)t + (0,79 \pm 0,05)]$$

Interpretación física:

- Amplitud inicial: $10,2 \pm 0,3$ unidades
- Tasa de amortiguamiento: pierde $1/e$ de amplitud cada $2,00 \pm 0,08 \text{ s}$
- Frecuencia: $f = 1/T = 1,000 \pm 0,016 \text{ Hz}$
- Fase inicial: $0,79 \pm 0,05 \text{ rad} \approx 45^\circ$

Solución:Ejercicio 9

- a) Para $y = Ae^{-kx} \cos(\omega x + \phi)$:
- A : Amplitud inicial (valor máximo - valor mínimo)/2
 - k : Constante de decaimiento $-\ln(\text{amplitude}_{\text{ratio}})/x_{\text{entre picos}}$
 - ω : Frecuencia angular $2\pi/\text{distancia entre picos consecutivos}$
 - ϕ : Fase $\arccos[y(0)/A]$ (ajustar signo según derivada en 0)
- b) Para $y = \frac{A}{1+e^{-k(x-x_0)}}$ (sigmoide):
- A : Valor asintótico para $x \rightarrow \infty$
 - x_0 : Punto de inflexión (donde $y = A/2$)
 - k : Pendiente en x_0 $A \times \text{pendiente máxima}/4$
- c) Para $y = A \sin^2(\omega x)$:

- A : Amplitud máxima (valor en picos)
- ω : Frecuencia angular $\pi/\text{distancia entre ceros consecutivos}$

Consejo general: Graficar los datos primero y estimar visualmente los parámetros. Usar características distintivas como máximos, mínimos, ceros, pendientes.

Solución:Ejercicio 10 Un $\chi^2_\nu < 0,5$ puede ser problemático porque:

1. **Sobreestimación de errores:** Si asignamos Δy_i demasiado grandes, los residuos $[y_i - f(x_i)]/\Delta y_i$ serán pequeños, dando χ^2 pequeño.
2. **Consecuencias:**
 - Los intervalos de confianza de los parámetros serán demasiado grandes
 - Podríamos no detectar desviaciones reales del modelo
 - La precisión reportada será menor que la real
3. **Sobrep parametrización:** Demasiados parámetros para la cantidad de datos puede hacer que el modelo se ajuste "demasiado bien." ^a fluctuaciones aleatorias.
4. **Datos "demasiado perfectos":** En casos extremos, $\chi^2_\nu \ll 1$ puede indicar:
 - Errores sistemáticos no considerados Datos fabricados o manipulados
 - Errores estimados incorrectamente
5. **Interpretación estadística:** Para N datos con errores gaussianos, esperamos $\chi^2_\nu \approx 1 \pm \sqrt{2/\nu}$.
 Por ejemplo, para $\nu = 10$: χ^2_ν esperado = $1 \pm 0,45$ Para $\nu = 5$: χ^2_ν esperado = $1 \pm 0,63$
 Un $\chi^2_\nu = 0,3$ con $\nu = 10$ está a más de $1,5\sigma$ del valor esperado $\rightarrow$ sospechoso.
6. **Qué hacer si $\chi^2_\nu \ll 1$:**
 - a) Revisar estimación de errores experimentales
 - b) Verificar calibración de instrumentos
 - c) Considerar si el modelo tiene demasiados parámetros
 - d) Analizar gráfico de residuos para patrones sistemáticos
 - e) En algunos casos, puede ser aceptable si se justifica físicamente

Conclusión: Tanto $\chi^2_\nu \gg 1$ como $\chi^2_\nu \ll 1$ son indicadores de posibles problemas. Idealmente queremos $\chi^2_\nu \approx 1$, que indica que el modelo describe adecuadamente los datos y las incertezas están bien estimadas.

Volumen 4A: Análisis de Datos Experimentales Avanzados - Parte 1

Nahuel Villafañe

2° Cuatrimestre 2025

**Licencia Creative Commons BY-NC-SA 4.0 Usted puede compartir y adaptar
este material
dando crédito al autor, sin fines comerciales
y compartiendo bajo la misma licencia.**

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

Prefacio

Este documento es un complemento al curso de Laboratorio 1. No busca reemplazar a ningún profesor/a, sino ser una herramienta adicional de estudio y práctica. Se recomienda fuertemente consultar con los docentes en caso de dudas. El material fue elaborado por estudiantes con el fin de apoyar a la comunidad académica.

Índice

Prefacio	2
1. Introducción al Análisis Avanzado de Datos	4
1.1. Transición del Análisis Básico al Avanzado	4
2. Análisis de Movimiento con Tracker	4
2.1. Fundamentos del Análisis de Video	4
2.1.1. Principios de Calibración	4
2.1.2. Error en Análisis de Video	4
2.2. Experimentación con Caída Libre	5
2.2.1. Teoría de Caída Libre	5
2.2.2. Inclusión de Resistencia del Aire	5
3. Ejercicios Teóricos - Análisis con Tracker	5
3.1. Ejercicio 1: Calibración de Video	5
3.2. Ejercicio 2: Análisis de Error en Trackeo	5
3.3. Ejercicio 3: Frame Rate y Resolución Temporal	6
3.4. Ejercicio 4: Repetitividad en Experimentos	6
3.5. Ejercicio 5: Modelo No Lineal para Caída Libre	6
3.6. Ejercicio 6: Chi-cuadrado Reducido en Ajustes No Lineales	6
3.7. Ejercicio 7: Efecto de la Resistencia del Aire	6
3.8. Ejercicio 8: Optimización de Condiciones Experimentales	7
3.9. Ejercicio 9: Propagación de Errores en Parámetros Cinemáticos	7
3.10. Ejercicio 10: Análisis Comparativo de Materiales	7
4. Soluciones a Ejercicios Teóricos	7
4.1. Solución Ejercicio 1	7
4.2. Solución Ejercicio 2	8
4.3. Solución Ejercicio 3	8
4.4. Solución Ejercicio 4	8
4.5. Solución Ejercicio 5	8
4.6. Solución Ejercicio 6	9
4.7. Solución Ejercicio 7	9
4.8. Solución Ejercicio 8	9
4.9. Solución Ejercicio 9	9
4.10. Solución Ejercicio 10	10

5. Ejercicios Prácticos - Implementación en Python	10
5.1. Ejercicio 11: Análisis Básico de Datos de Tracker	10
5.2. Ejercicio 12: Análisis de Repetitividad	10
5.3. Ejercicio 13: Detección de Resistencia del Aire	10
5.4. Ejercicio 14: Optimización de Frame Rate	10
5.5. Ejercicio 15: Análisis de Dos Materiales	11
5.6. Ejercicio 16: Propagación Completa de Errores	11
5.7. Ejercicio 17: Validación de Modelo No Lineal	11
5.8. Ejercicio 18: Análisis de Residuos Automatizado	11
5.9. Ejercicio 19: Comparación Múltiple de Ajustes	11
5.10. Ejercicio 20: Visualización Profesional	12
6. Código Python para Ejercicios Prácticos	12
6.1. Ejercicio 13: Detección de Resistencia del Aire (continuación)	16
6.2. Ejercicio 14: Optimización de Frame Rate	19
7. Conclusión del Volumen 4A	22

1. Introducción al Análisis Avanzado de Datos

En las secciones anteriores hemos cubierto los fundamentos de medición, análisis estadístico básico, mediciones indirectas y modelado lineal. En este volumen avanzamos hacia técnicas más sofisticadas de análisis de datos experimentales que son esenciales en la física experimental moderna.

1.1. Transición del Análisis Básico al Avanzado

Mientras que antes trabajábamos con:

- Mediciones estáticas o de baja frecuencia
- Análisis estadístico de muestras pequeñas
- Modelos lineales simples

Ahora abordaremos:

- Mediciones dinámicas en tiempo real
- Adquisición automatizada de datos
- Análisis de señales y video
- Modelos no lineales complejos

2. Análisis de Movimiento con Tracker

2.1. Fundamentos del Análisis de Video

Tracker es un software de análisis de video que permite extraer datos cinemáticos a partir de grabaciones. Su funcionamiento se basa en:

2.1.1. Principios de Calibración

1. **Escala:** Se define usando un objeto de referencia de longitud conocida
2. **Origen:** Se establece un sistema de coordenadas
3. **Frame Rate:** Se especifica la tasa de cuadros por segundo (fps)

2.1.2. Error en Análisis de Video

Las principales fuentes de error son:

- **Error de posición (Δy):**

$$\Delta y = \text{máx}(\text{resolución del video, tamaño del pixel, tamaño del objeto})$$

- **Error temporal (Δt):**

$$\Delta t = \frac{1}{\text{fps}}$$

Donde fps son los cuadros por segundo.

2.2. Experimentación con Caída Libre

2.2.1. Teoría de Caída Libre

Para un objeto en caída libre sin resistencia del aire:

$$y(t) = y_0 + v_0 t - \frac{1}{2}gt^2$$

$$v(t) = v_0 - gt$$

2.2.2. Inclusión de Resistencia del Aire

Para velocidades bajas, la fuerza de resistencia es proporcional a la velocidad:

$$F_r = -bv$$

La ecuación de movimiento se convierte en:

$$m\ddot{y} = -mg - b\dot{y}$$

3. Ejercicios Teóricos - Análisis con Tracker

3.1. Ejercicio 1: Calibración de Video

Problema: Un video se grabó a 60 fps. Se usa una regla de 1.000 m como referencia, que en pantalla mide 500 píxeles. El objeto a analizar tiene un diámetro de aproximadamente 50 píxeles. Determine:

1. La escala en m/píxel
2. Δy para la posición del objeto
3. Δt para las mediciones temporales

3.2. Ejercicio 2: Análisis de Error en Trackeo

Problema: En un análisis de caída libre, el objeto se deforma durante el movimiento, variando su aparente tamaño entre 45 y 55 píxeles. Si la escala es 0.002 m/píxel y el video es a 30 fps, calcule:

1. El rango de Δy posible
2. Δt absoluto
3. Error relativo en posición para $y = 1,0$ m

3.3. Ejercicio 3: Frame Rate y Resolución Temporal

Problema: Compare la resolución temporal y posibles efectos de aliasing para videos grabados a:

1. 24 fps (cine estándar)
2. 30 fps (video estándar)
3. 60 fps (alta velocidad común)
4. 240 fps (alta velocidad)

Para un movimiento oscilatorio con período de 0.5 s.

3.4. Ejercicio 4: Repetitividad en Experimentos

Problema: Defina criterios cuantitativos para determinar si un experimento es repetitivo cuando se tienen múltiples curvas $y(t)$. Considere:

1. Superposición de curvas dentro de incertezas
2. Comparación estadística de parámetros
3. Análisis de residuos

3.5. Ejercicio 5: Modelo No Lineal para Caída Libre

Problema: Derive las ecuaciones normales para ajustar $y(t) = y_0 + v_0 t - \frac{1}{2}gt^2$ usando mínimos cuadrados no lineales. Considere que se minimiza:

$$S = \sum_{i=1}^N \left[\frac{y_i - (y_0 + v_0 t_i - \frac{1}{2}gt_i^2)}{\Delta y_i} \right]^2$$

3.6. Ejercicio 6: Chi-cuadrado Reducido en Ajustes No Lineales

Problema: Para un ajuste no lineal con 3 parámetros (y_0, v_0, g) y N puntos, demuestre que:

$$\chi_\nu^2 = \frac{\chi^2}{N-3}$$

y discuta la interpretación cuando $\chi_\nu^2 \approx 1$, $\chi_\nu^2 \ll 1$, y $\chi_\nu^2 \gg 1$.

3.7. Ejercicio 7: Efecto de la Resistencia del Aire

Problema: Para un objeto esférico de radio r y densidad ρ cayendo en aire (densidad ρ_a , viscosidad η):

1. Calcule la velocidad terminal
2. Estime el tiempo característico para alcanzar el 90 % de la velocidad terminal
3. Proponga un método para detectar experimentalmente efectos de resistencia del aire

3.8. Ejercicio 8: Optimización de Condiciones Experimentales

Problema: Para medir g con Tracker, se tiene una altura máxima disponible de 2.5 m. Determine:

1. El tiempo total de caída ideal
2. El frame rate mínimo necesario para tener al menos 100 puntos de datos
3. La resolución espacial necesaria para que $\Delta g/g < 1\%$

3.9. Ejercicio 9: Propagación de Errores en Parámetros Cinemáticos

Problema: Partiendo de $y(t) = y_0 + v_0 t - \frac{1}{2}gt^2$, derive expresiones para:

1. Δv_0 en función de Δy , Δt , y Δg
2. Δg en función de Δy y Δt
3. La matriz de covarianza entre y_0 , v_0 , y g

3.10. Ejercicio 10: Análisis Comparativo de Materiales

Problema: Compare teóricamente la caída de:

1. Una esfera metálica (acero, $\rho = 7800 \text{ kg/m}^3$, $r = 1 \text{ cm}$)
2. Una esfera de papel comprimido ($\rho = 800 \text{ kg/m}^3$, $r = 1 \text{ cm}$)

en aire ($\eta = 1,8 \times 10^{-5} \text{ Pa}\cdot\text{s}$, $\rho_a = 1,2 \text{ kg/m}^3$). Calcule:

1. Velocidades terminales
2. Tiempos para caer 2 m
3. Diferencia porcentual en g aparente

4. Soluciones a Ejercicios Teóricos

4.1. Solución Ejercicio 1

1. Escala = $\frac{1,000 \text{ m}}{500 \text{ px}} = 0,002 \text{ m/px}$
2. $\Delta y = \max(1 \text{ px}, 50 \text{ px}) \times 0,002 \text{ m/px} = 0,100 \text{ m}$ (usando el tamaño del objeto)
3. $\Delta t = \frac{1}{60} \text{ s} \approx 0,0167 \text{ s}$

4.2. Solución Ejercicio 2

1. $\Delta y_{\min} = 45 \text{ px} \times 0,002 \text{ m/px} = 0,090 \text{ m}$
2. $\Delta y_{\max} = 55 \text{ px} \times 0,002 \text{ m/px} = 0,110 \text{ m}$
3. Se recomienda usar $\Delta y = 0,110 \text{ m}$ (el peor caso)
4. $\Delta t = \frac{1}{30} \text{ s} \approx 0,0333 \text{ s}$
5. $\varepsilon_y = \frac{0,110}{1,0} = 11 \%$

4.3. Solución Ejercicio 3

Para un período $T = 0,5 \text{ s}$:

1. 24 fps: $\Delta t = 0,0417 \text{ s}$, ~ 12 puntos por período (adecuado)
2. 30 fps: $\Delta t = 0,0333 \text{ s}$, ~ 15 puntos por período (bueno)
3. 60 fps: $\Delta t = 0,0167 \text{ s}$, ~ 30 puntos por período (excelente)
4. 240 fps: $\Delta t = 0,0042 \text{ s}$, ~ 120 puntos por período (sobredimensionado)

El teorema de Nyquist requiere al menos 2 puntos por período para evitar aliasing.

4.4. Solución Ejercicio 4

Criterios cuantitativos:

1. Para cada tiempo t_i , el intervalo $[y_{\min}(t_i), y_{\max}(t_i)]$ debe tener superposición con los intervalos de otras curvas
2. Test de hipótesis: H_0 : las curvas provienen de la misma distribución
3. ANOVA para comparar parámetros de ajuste
4. Residuos deben distribuirse aleatoriamente sin patrones sistemáticos

4.5. Solución Ejercicio 5

Definiendo $f(t) = y_0 + v_0 t - \frac{1}{2}gt^2$, minimizamos:

$$S = \sum w_i [y_i - f(t_i)]^2, \quad w_i = \frac{1}{\Delta y_i^2}$$

Las ecuaciones normales son:

$$\begin{aligned} \frac{\partial S}{\partial y_0} &= -2 \sum w_i [y_i - f(t_i)] = 0 \\ \frac{\partial S}{\partial v_0} &= -2 \sum w_i t_i [y_i - f(t_i)] = 0 \\ \frac{\partial S}{\partial g} &= \sum w_i t_i^2 [y_i - f(t_i)] = 0 \end{aligned}$$

Sistema lineal en y_0 , v_0 , g :

$$\begin{pmatrix} \sum w_i & \sum w_i t_i & -\frac{1}{2} \sum w_i t_i^2 \\ \sum w_i t_i & \sum w_i t_i^2 & -\frac{1}{2} \sum w_i t_i^3 \\ -\frac{1}{2} \sum w_i t_i^2 & -\frac{1}{2} \sum w_i t_i^3 & \frac{1}{4} \sum w_i t_i^4 \end{pmatrix} \begin{pmatrix} y_0 \\ v_0 \\ g \end{pmatrix} = \begin{pmatrix} \sum w_i y_i \\ \sum w_i t_i y_i \\ -\frac{1}{2} \sum w_i t_i^2 y_i \end{pmatrix}$$

4.6. Solución Ejercicio 6

Para N puntos y 3 parámetros:

- $\chi^2 = \sum \left(\frac{y_i - f(t_i)}{\Delta y_i} \right)^2$
- Grados de libertad: $\nu = N - 3$
- $\chi_\nu^2 = \chi^2 / \nu$
- $\chi_\nu^2 \approx 1$: buen ajuste, errores bien estimados
- $\chi_\nu^2 \ll 1$: errores sobreestimados o modelo muy flexible
- $\chi_\nu^2 \gg 1$: errores subestimados o modelo inadecuado

4.7. Solución Ejercicio 7

1. Velocidad terminal: $v_t = \frac{2}{9} \frac{r^2 g (\rho - \rho_a)}{\eta}$
2. Tiempo característico: $\tau = \frac{m}{b} = \frac{2r^2 \rho}{9\eta}$
3. Para detectar resistencia: comparar $y(t)$ con modelo parabólico, buscar sistematicidad en residuos, analizar dependencia con tamaño/densidad

4.8. Solución Ejercicio 8

1. $t = \sqrt{\frac{2h}{g}} = \sqrt{\frac{2 \times 2,5}{9,8}} \approx 0,714$ s
2. Para 100 puntos: $\text{fps} \geq \frac{100}{0,714} \approx 140$ fps
3. De $g = \frac{2(y - y_0 - v_0 t)}{t^2}$, propagación da $\frac{\Delta g}{g} \approx 2 \frac{\Delta y}{y} + 2 \frac{\Delta t}{t}$ Para $\frac{\Delta g}{g} < 0,01$, necesitamos $\frac{\Delta y}{y} < 0,005$ y $\frac{\Delta t}{t} < 0,005$

4.9. Solución Ejercicio 9

Usando propagación de errores:

1. $\Delta v_0^2 = \left(\frac{\partial v_0}{\partial y} \right)^2 \Delta y^2 + \left(\frac{\partial v_0}{\partial t} \right)^2 \Delta t^2 + \left(\frac{\partial v_0}{\partial g} \right)^2 \Delta g^2$
2. Para g obtenido por ajuste: Δg viene de la matriz de covarianza del ajuste
3. La matriz de covarianza C se obtiene como $(J^T W J)^{-1}$, donde J es la matriz jacobiana y W la matriz de pesos

4.10. Solución Ejercicio 10

1. Esfera metálica: $v_t \approx 240$ m/s (pero supersonica, régimen no válido para Stokes)
2. Esfera de papel: $v_t \approx 8,7$ m/s
3. Para papel: $g_{\text{aparente}} = g \left(1 - \frac{v}{v_t}\right)$
4. A 2 m/s: $g_{\text{ap}} \approx 0,77g$ (23 % menor)

5. Ejercicios Prácticos - Implementación en Python

5.1. Ejercicio 11: Análisis Básico de Datos de Tracker

Problema: Cargue datos de posición vs tiempo de un objeto en caída libre y:

1. Calcule velocidad y aceleración por diferencias finitas
2. Ajuste un modelo cuadrático
3. Calcule g y su incerteza
4. Grafique datos, ajuste y residuos

5.2. Ejercicio 12: Análisis de Repetitividad

Problema: Dados 3 conjuntos de datos de caída libre:

1. Determine si son repetitivos gráficamente
2. Realice test estadístico de comparación
3. Calcule promedio y dispersión de g

5.3. Ejercicio 13: Detección de Resistencia del Aire

Problema: Simule datos de caída con resistencia del aire y:

1. Ajuste modelo parabólico
2. Analice residuos para detectar sistematicidad
3. Proponga y ajuste modelo con resistencia

5.4. Ejercicio 14: Optimización de Frame Rate

Problema: Simule una caída libre perfecta y muestree a diferentes fps:

1. 10, 30, 60, 120 fps
2. Calcule g para cada caso
3. Analice error vs fps

5.5. Ejercicio 15: Análisis de Dos Materiales

Problema: Compare datos de caída para esfera metálica y de papel:

1. Ajuste modelos separados
2. Compare valores de g
3. Evalúe significancia estadística de la diferencia

5.6. Ejercicio 16: Propagación Completa de Errores

Problema: Implemente cálculo completo de incertezas para:

1. Parámetros del ajuste (y_0, v_0, g)
2. Velocidad en cada punto
3. Aceleración en cada punto

5.7. Ejercicio 17: Validación de Modelo No Lineal

Problema: Genere datos con ruido conocido y:

1. Ajuste modelo no lineal
2. Calcule χ^2_ν
3. Verifique recuperación de parámetros verdaderos

5.8. Ejercicio 18: Análisis de Residuos Automatizado

Problema: Cree función que automáticamente:

1. Detecte patrones en residuos
2. Calcule autocorrelación de residuos
3. Evalúe normalidad de residuos

5.9. Ejercicio 19: Comparación Múltiple de Ajustes

Problema: Implemente comparación sistemática de:

1. Ajuste lineal vs cuadrático
2. Ajuste con y sin pesos
3. Diferentes métodos de optimización

5.10. Ejercicio 20: Visualización Profesional

Problema: Cree figuras de publicación para:

1. Datos con incertezas y ajuste
2. Residuos con bandas de confianza
3. Comparación múltiple de experimentos

6. Código Python para Ejercicios Prácticos

Listing 1: Análisis básico de datos de Tracker

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
from scipy import stats

def ejercicio_11():
    """Análisis básico de datos de Tracker"""
    # Datos simulados: t en s, y en m
    t = np.linspace(0, 0.7, 100)
    g_real = 9.81
    y0 = 2.0
    v0 = 0.1
    y = y0 + v0*t - 0.5*g_real*t**2

    # Añadir ruido
    np.random.seed(42)
    y_noise = y + np.random.normal(0, 0.02, len(t))
    dy = 0.02 * np.ones_like(t) # incerteza constante

    # Modelo cuadrático
    def modelo_cuadratico(t, y0, v0, g):
        return y0 + v0*t - 0.5*g*t**2

    # Ajuste con pesos
    pesos = 1/dy**2
    popt, pcov = curve_fit(modelo_cuadratico, t, y_noise,
                           sigma=dy, absolute_sigma=True)

    y0_aj, v0_aj, g_aj = popt
    dy0, dv0, dg = np.sqrt(np.diag(pcov))

    # Cálculo de velocidad y aceleración por diferencias finitas
    v = np.gradient(y_noise, t)
    a = np.gradient(v, t)
```



```

# Gráficas
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Datos y ajuste
axes[0,0].errorbar(t, y_noise, yerr=dy, fmt='o',
                  markersize=3, label='Datos')
t_fit = np.linspace(0, 0.7, 200)
y_fit = modelo_cuadratico(t_fit, *popt)
axes[0,0].plot(t_fit, y_fit, 'r-',
               label=f'Ajuste:  $-g = -\{g_{aj} \pm .2 f\} - \{dg \pm .2 f\} \cdot m/s^2$ ')
axes[0,0].set_xlabel('Tiempo (s)')
axes[0,0].set_ylabel('Posición (m)')
axes[0,0].legend()
axes[0,0].grid(True, alpha=0.3)

# Residuos
residuos = y_noise - modelo_cuadratico(t, *popt)
axes[0,1].errorbar(t, residuos, yerr=dy, fmt='o',
                  markersize=3)
axes[0,1].axhline(y=0, color='r', linestyle='—')
axes[0,1].set_xlabel('Tiempo (s)')
axes[0,1].set_ylabel('Residuos (m)')
axes[0,1].grid(True, alpha=0.3)

# Velocidad
axes[1,0].plot(t, v, 'b-', label='Velocidad numérica')
v_teorica = v0_aj - g_aj*t
axes[1,0].plot(t, v_teorica, 'r—', label='Velocidad del ajuste')
axes[1,0].set_xlabel('Tiempo (s)')
axes[1,0].set_ylabel('Velocidad (m/s)')
axes[1,0].legend()
axes[1,0].grid(True, alpha=0.3)

# Aceleración
axes[1,1].plot(t, a, 'b-', label='Aceleración numérica')
axes[1,1].axhline(y=-g_aj, color='r', linestyle='—',
                 label=f'Aceleración del ajuste:  $-\{g_{aj} \pm .2 f\} \cdot m/s^2$ ')
axes[1,1].set_xlabel('Tiempo (s)')
axes[1,1].set_ylabel('Aceleración (m/s2)')
axes[1,1].legend()
axes[1,1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Resultados
print("==== Resultados del Análisis =====")
print(f" $g_{real} = \{g_{real} \pm .3 f\} \cdot m/s^2$ ")

```

```

print(f" g-obtenido: {g_aj:.3 f} - {dg:.3 f} m/s ")
print(f" y0: {y0_aj:.3 f} - {dy0:.3 f} m")
print(f" v0: {v0_aj:.3 f} - {dv0:.3 f} m/s")

# Chi-cuadrado reducido
chi2 = np.sum((residuos/dy)**2)
chi2_red = chi2/(len(t)-3)
print(f" : {chi2:.2 f}")
print(f" -reducido: {chi2_red:.2 f}")

return popt, pcov

def ejercicio_12():
    """ An lisis de repetitividad con 3 conjuntos de datos """
    np.random.seed(123)
    n_puntos = 50
    t = np.linspace(0, 0.6, n_puntos)

    # Generar 3 conjuntos con variaciones pequenas
    conjuntos = []
    for i in range(3):
        g_var = 9.81 + np.random.normal(0, 0.1) # pequena variaci n en g
        y0_var = 1.5 + np.random.normal(0, 0.02) # pequena variaci n en y0
        y = y0_var - 0.5*g_var*t**2
        y_noise = y + np.random.normal(0, 0.03, n_puntos)
        conjuntos.append((t.copy(), y_noise.copy()))

    # Ajustar cada conjunto por separado
    def modelo(t, y0, g):
        return y0 - 0.5*g*t**2

    resultados = []
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    for i, (t_curr, y_curr) in enumerate(conjuntos):
        dy = 0.03
        popt, pcov = curve_fit(modelo, t_curr, y_curr,
                                sigma=dy*np.ones_like(t_curr),
                                absolute_sigma=True)

        y0_aj, g_aj = popt
        dy0, dg = np.sqrt(np.diag(pcov))
        resultados.append((y0_aj, dy0, g_aj, dg))

    # Gr fico de datos
    axes[0].plot(t_curr, y_curr, 'o', markersize=3,
                  label=f'Conjunto {i+1}', alpha=0.7)
    t_fit = np.linspace(0, 0.6, 100)
    y_fit = modelo(t_fit, *popt)

```

```

axes[0].plot(t_fit , y_fit , '-', alpha=0.7)

axes[0].set_xlabel('Tiempo (s)')
axes[0].set_ylabel('Posici n (m)')
axes[0].legend()
axes[0].grid(True, alpha=0.3)
axes[0].set_title('Datos y ajustes individuales')

# Analisis estad stico de repetitividad
g_vals = [r[2] for r in resultados]
g_errs = [r[3] for r in resultados]

# Test de comparaci n de medias
axes[1].errorbar(range(1,4), g_vals , yerr=g_errs ,
                  fmt='o' , capsize=5)
axes[1].axhline(y=9.81, color='r' , linestyle='--' ,
                 label='Valor te rico ')
axes[1].set_xlabel('Conjunto')
axes[1].set_ylabel('g (m/s )')
axes[1].set_xticks([1,2,3])
axes[1].legend()
axes[1].grid(True, alpha=0.3)
axes[1].set_title('Comparaci n de valores de g')

plt.tight_layout()
plt.show()

# Analisis cuantitativo
print("==== Analisis de Repetitividad ====")
for i, (y0, dy0, g, dg) in enumerate(resultados):
    print(f"Conjunto {i+1}: g={g:.3f} - {dg:.3f} m/s , -"
          f"y0={y0:.3f} - {dy0:.3f} m")

# Test ANOVA simplificado
g_promedio = np.mean(g_vals)
g_std = np.std(g_vals)
print(f"\nPromedio de g: {g_promedio:.3f} m/s ")
print(f"Desviaci n est ndar entre conjuntos: {g_std:.3f} m/s ")
print(f"Error t pico del promedio: {g_std/np.sqrt(3):.3f} m/s ")

# Verificaci n de superposici n
print("\nVerificaci n de superposici n (68%-CL):")
for i in range(3):
    for j in range(i+1, 3):
        gi, dgi = g_vals[i], g_errs[i]
        gj, dgj = g_vals[j], g_errs[j]
        dif = abs(gi - gj)
        lim = dgi + dgj

```

```

        if dif < lim:
            print(f"Conjuntos {i+1}-y-{j+1}: OK (|   |={dif:.3f}) < {lim}")
        else:
            print(f"Conjuntos {i+1}-y-{j+1}: NO superponen (|   |={dif:.3f})")

    return resultados

if __name__ == "__main__":
    print("Ejecutando ejercicios pr cticos...")
    print("\n" + "="*50)
    print("EJERCICIO 11: An lisis b sico de datos de Tracker")
    print("="*50)
    ejercicio_11()

    print("\n" + "="*50)
    print("EJERCICIO 12: An lisis de repetitividad")
    print("="*50)
    ejercicio_12()

```

6.1. Ejercicio 13: Detección de Resistencia del Aire (continuación)

Listing 2: Detección de resistencia del aire

```

def ejercicio_13():
    """Detecci n de resistencia del aire en datos de ca da"""
    import numpy as np
    from scipy.integrate import odeint
    import matplotlib.pyplot as plt
    from scipy.optimize import curve_fit

    # Par metros f sicos
    g = 9.81
    m = 0.1 # kg
    b = 0.05 # coeficiente de resistencia (N s/m)

    # Modelo con resistencia del aire
    def modelo_resistencia(y, t):
        """y[0]=posici n , y[1]=velocidad"""
        dydt = [y[1], -g - (b/m)*y[1]]
        return dydt

    # Condiciones iniciales
    y0 = [2.0, 0.0] # 2 m de altura, velocidad inicial 0
    t = np.linspace(0, 0.7, 100)

    # Soluci n num rica
    sol = odeint(modelo_resistencia, y0, t)

```

```

y_real = sol[:, 0]

# Aadir ruido de medici n
np.random.seed(42)
y_medida = y_real + np.random.normal(0, 0.01, len(t))
dy = 0.01 * np.ones_like(t)

# Modelo sin resistencia (parab lico)
def modelo_parabolico(t, y0, v0, g):
    return y0 + v0*t - 0.5*g*t**2

# Modelo con resistencia (aproximaci n para b peque o)
def modelo_con_resistencia(t, y0, v0, g, b):
    # Soluci n aproximada:  $y = y0 + v0*t - 0.5*g*t^2 - (b/(6m))*v0*t^3$ 
    return y0 + v0*t - 0.5*g*t**2 - (b/(6*m))*v0*t**3

# Ajuste con modelo parab lico
popt_parab, pcov_parab = curve_fit(modelo_parabolico, t, y_medida,
                                   sigma=dy, absolute_sigma=True,
                                   p0=[2.0, 0.0, 9.81])

y0_p, v0_p, g_p = popt_parab
y_fit_parab = modelo_parabolico(t, *popt_parab)

# Ajuste con modelo con resistencia
popt_res, pcov_res = curve_fit(modelo_con_resistencia, t, y_medida,
                                sigma=dy, absolute_sigma=True,
                                p0=[2.0, 0.0, 9.81, 0.05])

y0_r, v0_r, g_r, b_r = popt_res
y_fit_res = modelo_con_resistencia(t, *popt_res)

# C lculo de residuos
residuos_parab = y_medida - y_fit_parab
residuos_res = y_medida - y_fit_res

# Gr ficos
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Datos y ajustes
axes[0,0].errorbar(t, y_medida, yerr=dy, fmt='o', markersize=3,
                   alpha=0.5, label='Datos medidos')
axes[0,0].plot(t, y_fit_parab, 'r-', label=f'Parab lico: g={g_p:.2f} m/s^2')
axes[0,0].plot(t, y_fit_res, 'g-',
               label=f'Con resistencia: g={g_r:.2f} m/s^2, b={b_r:.3f} N')
axes[0,0].plot(t, y_real, 'b—', alpha=0.7, label='Real (con resistencia)')
axes[0,0].set_xlabel('Tiempo (s)')
axes[0,0].set_ylabel('Posici n (m)')
axes[0,0].legend()
axes[0,0].grid(True, alpha=0.3)

```

```

axes[0,0].set_title('Comparaci n-de-modelos')

# Residuos del modelo parab lico
axes[0,1].errorbar(t, residuos_parab, yerr=dy, fmt='o', markersize=3)
axes[0,1].axhline(y=0, color='r', linestyle='—')
axes[0,1].set_xlabel('Tiempo-(s)')
axes[0,1].set_ylabel('Residuos-(m)')
axes[0,1].grid(True, alpha=0.3)
axes[0,1].set_title('Residuos-del-modelo-parab lico')

# Residuos del modelo con resistencia
axes[1,0].errorbar(t, residuos_res, yerr=dy, fmt='o', markersize=3)
axes[1,0].axhline(y=0, color='r', linestyle='—')
axes[1,0].set_xlabel('Tiempo-(s)')
axes[1,0].set_ylabel('Residuos-(m)')
axes[1,0].grid(True, alpha=0.3)
axes[1,0].set_title('Residuos-del-modelo-con-resistencia')

# An lisis de autocorrelaci n de residuos
def autocorr(x):
    n = len(x)
    result = np.correlate(x - np.mean(x), x - np.mean(x), mode='full')
    return result[n-1:] / result[n-1]

lag = np.arange(50)
acf_parab = autocorr(residuos_parab)[:50]
acf_res = autocorr(residuos_res)[:50]

axes[1,1].plot(lag, acf_parab, 'r-', label='Parab lico')
axes[1,1].plot(lag, acf_res, 'g-', label='Con-resistencia')
axes[1,1].axhline(y=0, color='k', linestyle='—', alpha=0.5)
axes[1,1].set_xlabel('Lag')
axes[1,1].set_ylabel('Autocorrelaci n')
axes[1,1].legend()
axes[1,1].grid(True, alpha=0.3)
axes[1,1].set_title('Autocorrelaci n-de-residuos')

plt.tight_layout()
plt.show()

# An lisis estad stico
chi2_parab = np.sum((residuos_parab/dy)**2)
chi2_res = np.sum((residuos_res/dy)**2)
chi2_red_parab = chi2_parab/(len(t)-3)
chi2_red_res = chi2_res/(len(t)-4)

print("==== An lisis-de-Resistencia-del-Aire====")
print(f"Par metros-reales: {g:.2f} m/s , {b:.3f} N s/m")

```

```

print(f"\nModelo parabólico:")
print(f"  g obtenido: {g_p:.3f} - {np.sqrt(pcov_parab[2,2]):.3f} m/s")
print(f"  chi2: {chi2_parab:.2f}")
print(f"  reducido: {chi2_red_parab:.2f}")

print(f"\nModelo con resistencia:")
print(f"  g obtenido: {g_r:.3f} - {np.sqrt(pcov_res[2,2]):.3f} m/s")
print(f"  b obtenido: {b_r:.3f} - {np.sqrt(pcov_res[3,3]):.3f} N s/m")
print(f"  chi2: {chi2_res:.2f}")
print(f"  reducido: {chi2_red_res:.2f}")

# Test F para comparar modelos
F = (chi2_parab - chi2_res)/(chi2_res/(len(t)-4))
print(f"\nTest F para comparación de modelos: F={F:.2f}")

# Detección de patrón en residuos
print(f"\nDetección de patrón sistemático:")
print(f"  Residuos parabólico: media={np.mean(residuos_parab):.4f},
      f"std={np.std(residuos_parab):.4f}")
print(f"  Residuos con resistencia: media={np.mean(residuos_res):.4f},
      f"std={np.std(residuos_res):.4f}")

return popt_parab, popt_res, residuos_parab, residuos_res

# Ejecutar ejercicio
if __name__ == "__main__":
    print("n" + "="*50)
    print("EJERCICIO 13: Detección de resistencia del aire")
    print("="*50)
    ejercicio_13()

```

6.2. Ejercicio 14: Optimización de Frame Rate

Listing 3: Optimización de frame rate

```

def ejercicio_14():
    """Optimización del frame rate en mediciones de caída libre"""
    import numpy as np
    import matplotlib.pyplot as plt

    # Parámetros reales
    g_real = 9.81
    y0 = 2.0
    v0 = 0.0
    t_max = np.sqrt(2*y0/g_real) # tiempo hasta llegar al suelo

    # Diferentes frame rates a probar
    fps_list = [10, 30, 60, 120, 240]

```

```

resultados = []

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

for idx, fps in enumerate(fps_list):
    # Tiempo entre frames
    dt = 1/fps

    # Generar tiempos de muestreo
    n_frames = int(t_max * fps) + 1
    t_sampled = np.arange(0, n_frames*dt, dt)
    t_sampled = t_sampled[t_sampled <= t_max]

    # Posición exacta en esos tiempos
    y_exact = y0 - 0.5*g_real*t_sampled**2

    # Añadir ruido de medición (proporcional a la escala)
    np.random.seed(42 + idx)
    dy = 0.01 # 1 cm de error
    y_noise = y_exact + np.random.normal(0, dy, len(t_sampled))

    # Ajuste cuadrático
    def modelo(t, y0, g):
        return y0 - 0.5*g*t**2

    from scipy.optimize import curve_fit
    popt, pcov = curve_fit(modelo, t_sampled, y_noise,
                           sigma=dy*np.ones_like(t_sampled),
                           absolute_sigma=True)

    y0_aj, g_aj = popt
    dg = np.sqrt(pcov[1,1])

    resultados.append({
        'fps': fps,
        'dt': dt,
        'n_puntos': len(t_sampled),
        'g_aj': g_aj,
        'dg': dg,
        'error_rel': abs(g_aj - g_real)/g_real,
        't_sampled': t_sampled,
        'y_noise': y_noise
    })

    # Gráfico de datos para algunos fps
    if idx < 4: # mostrar solo primeros 4
        row = idx // 2
        col = idx % 2
        axes[row, col].plot(t_sampled, y_noise, 'o', markersize=3,

```



```

        label=f'fps={fps}, n={len(t_sampled)}')
    t_fine = np.linspace(0, t_max, 200)
    y_fine = modelo(t_fine, *popt)
    axes[row, col].plot(t_fine, y_fine, 'r-',
        label=f'g={g_aj:.2f} {dg:.2f} m/s ')
    axes[row, col].plot(t_fine, y0 - 0.5*g_real*t_fine**2,
        'k--', alpha=0.5, label='Real')
    axes[row, col].set_xlabel('Tiempo (s)')
    axes[row, col].set_ylabel('Posici n (m)')
    axes[row, col].legend(fontsize=8)
    axes[row, col].grid(True, alpha=0.3)
    axes[row, col].set_title(f'fps={fps} ( t={dt:.3f} s)')

plt.tight_layout()
plt.show()

# Analisis de resultados
print("==== Optimizaci n de Frame Rate ====")
print("FPS | t (s) | N-puntos | g (m/s ) | g (m/s ) | Error-rel")
print("-"*60)

fps_vals = []
dg_vals = []
error_vals = []

for res in resultados:
    fps = res['fps']
    dt = res['dt']
    n = res['n_puntos']
    g = res['g_aj']
    dg = res['dg']
    err = res['error_rel']

    print(f"{fps:3d} | {dt:6.3f} | {n:8d} | {g:8.3f} | {dg:8.3f} | {err:8.3f}")

    fps_vals.append(fps)
    dg_vals.append(dg)
    error_vals.append(err)

# Gr fico de error vs fps
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

ax1.plot(fps_vals, dg_vals, 'bo-', markersize=8)
ax1.set_xlabel('Frame Rate (fps)')
ax1.set_ylabel('Incerteza en g (m/s )')
ax1.set_xscale('log')
ax1.grid(True, alpha=0.3)
ax1.set_title('Incerteza vs Frame Rate')

```

```

ax2.plot(fps_vals, error_vals, 'ro-', markersize=8)
ax2.set_xlabel('Frame-Rate (fps)')
ax2.set_ylabel('Error relativo en g')
ax2.set_xscale('log')
ax2.set_yscale('log')
ax2.grid(True, alpha=0.3)
ax2.set_title('Error relativo vs Frame-Rate')

plt.tight_layout()
plt.show()

# Análisis teórico
print("\n==== Análisis Teórico =====")
print("La incerteza en g viene principalmente de:")
print("1.- Error en posición (y) -- dominante a bajos fps")
print("2.- Error temporal (t) -- importante a altos fps")
print("3.- Número de puntos (N) -- mejora con N")

# Ley de escalado esperada
print("\nLey de escalado esperada:")
print("Para error en posición dominante:  $g \propto y / (t_{\max})$ ")
print("Para error temporal dominante:  $g \propto g_{\max} (t / t_{\max})$ ")
print(f"Con  $t_{\max} = \{t_{\max} : .2 f\}$  s")

# Frame rate ptimo
opt_idx = np.argmin(dg_vals)
opt_fps = fps_vals[opt_idx]
print(f"\nFrame-rate ptimo empírico:  $\{opt\_fps\}$  fps")
print(f"Incerteza mínima:  $\{dg\_vals[opt\_idx] : .3 f\}$  m/s2")

return resultados

# Ejecutar ejercicio
if __name__ == "__main__":
    print("\n" + "="*50)
    print("EJERCICIO 14: Optimización de frame-rate")
    print("="*50)
    ejercicio_14()

```

7. Conclusión del Volumen 4A

En esta primera parte del Volumen 4 hemos cubierto:

1. Fundamentos del análisis de video con Tracker
2. Calibración y estimación de errores en mediciones ópticas
3. Análisis de caída libre y detección de resistencia del aire

4. Optimización de parámetros experimentales (frame rate)
5. Implementación práctica en Python de todas las técnicas

Los conceptos y técnicas presentados aquí son fundamentales para experimentos modernos de mecánica, donde el análisis de video se ha convertido en una herramienta estándar. En la Parte B continuaremos con adquisición de datos mediante sensores y procesamiento de señales.

Volumen 4B: Análisis de Datos Experimentales Avanzados - Parte 2

Nahuel Villafañe

2° Cuatrimestre 2025

**Licencia Creative Commons BY-NC-SA 4.0 Usted puede compartir y adaptar
este material
dando crédito al autor, sin fines comerciales
y compartiendo bajo la misma licencia.**

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

1. Adquisición Automatizada de Datos

1.1. Fundamentos de los Sistemas DAQ

Los sistemas de Adquisición de Datos (DAQ, por sus siglas en inglés) permiten convertir fenómenos físicos en señales eléctricas que pueden ser procesadas por computadoras.

1.1.1. Cadena de Adquisición

1. **Transductor:** Convierte magnitud física en señal eléctrica
2. **Acondicionamiento:** Amplificación, filtrado, etc.
3. **Conversión A/D:** Digitalización de la señal
4. **Procesamiento:** Análisis y almacenamiento en computadora

1.1.2. Parámetros Clave de un Sistema DAQ

- **Resolución:** Número de bits del conversor A/D

$$N_{\text{niveles}} = 2^n \quad (n = \text{bits})$$

- **Rango Dinámico:** Valores máximo y mínimo medibles

$$\text{Rango} = V_{\text{max}} - V_{\text{min}}$$

- **Frecuencia de Muestreo:** Número de muestras por segundo

$$f_s = \frac{1}{\Delta t}$$

- **Ancho de Banda:** Frecuencia máxima que puede capturar

- Teorema de Nyquist: $f_s > 2f_{\text{max}}$
- En práctica: $f_s > (5 - 10)f_{\text{max}}$

1.2. Sensores Específicos

1.2.1. Sensor de Posición por Ultrasonido

- **Principio:** Medición de tiempo de vuelo del eco

$$D = v_{\text{sonido}} \cdot \frac{\tau}{2}$$

Donde $v_{\text{sonido}} \approx 343 \text{ m/s}$ a 20°C

- **Características:**

- Rango típico: 0.15 - 6 m
- Frecuencia de ultrasonido: 49.4 kHz
- Apertura angular: 30°

- Resolución: 1 mm
- **Fuentes de error:**
 - Temperatura (afecta v_{sonido})
 - Humedad del aire
 - Objetos en el campo de visión
 - Reflexiones múltiples

1.2.2. Sensor de Fuerza

- **Tipos:** Galga extensométrica, celda de carga
- **Rangos típicos:** ± 10 N, ± 50 N
- **Resolución:** 0.01 N a 0.05 N
- **Calibración:** Relación lineal Fuerza-Voltaje

$$F = K_0 + K_1 \cdot V$$

2. Experimento: Movimiento en Plano Inclinado

2.1. Teoría del Movimiento en Plano Inclinado

Para un carrito de masa m en un plano con ángulo θ :

2.1.1. Sin Rozamiento

$$\begin{aligned}\sum F_x &= mg \sin \theta = ma_x \\ a_x &= g \sin \theta \\ v(t) &= v_0 + a_x t \\ x(t) &= x_0 + v_0 t + \frac{1}{2} a_x t^2\end{aligned}$$

2.1.2. Con Rozamiento Cinético

$$\begin{aligned}\sum F_x &= mg \sin \theta - \mu_k mg \cos \theta = ma_x \\ a_x &= g(\sin \theta - \mu_k \cos \theta)\end{aligned}$$

2.2. Métodos de Medición

Se utilizarán tres métodos simultáneos:

1. **Sensor de posición:** Mediciones automáticas de $x(t)$
2. **Tracker:** Análisis de video
3. **Cronómetro manual:** Tiempo total de recorrido

3. Ejercicios Teóricos - Sistemas DAQ

3.1. Ejercicio 21: Especificaciones de un Sistema DAQ

Problema: Un sistema DAQ tiene las siguientes especificaciones:

- Resolución: 12 bits
- Rango: ± 10 V
- Frecuencia máxima de muestreo: 100 kHz
- Error de no linealidad: 0.1 % del fondo de escala

Calcule:

1. Resolución en voltios
2. Número total de niveles de cuantificación
3. Ancho de banda máximo que puede capturar sin aliasing
4. Error total en la medición de una señal de 5 V

3.2. Ejercicio 22: Calibración de Sensor

Problema: Para calibrar un sensor de fuerza se obtienen los siguientes datos:

Voltaje (V)	Fuerza (N)
0.00	0.00
1.25	4.90
2.50	9.81
3.75	14.72
5.00	19.62

Determine:

1. Coeficientes de calibración K_0 y K_1
2. Error cuadrático medio de la calibración
3. Fuerza correspondiente a 3.20 V con su incerteza

3.3. Ejercicio 23: Sensor de Ultrasonido

Problema: Un sensor de ultrasonido mide un tiempo de eco de 5.82 ms.

1. Calcule la distancia al objeto
2. Si la temperatura es 25°C ($v_{\text{sonido}} = 346$ m/s), recalcule la distancia
3. Determine el error en distancia si $\Delta\tau = 0,01$ ms

3.4. Ejercicio 24: Frecuencia de Muestreo Óptima

Problema: Para un movimiento oscilatorio con:

$$x(t) = A \cos(2\pi f_0 t) + 0,1A \cos(2\pi \cdot 3f_0 t)$$

donde $f_0 = 2$ Hz.

1. ¿Cuál es la frecuencia mínima de muestreo según Nyquist?
2. ¿Qué frecuencia recomendaría para una buena reconstrucción?
3. ¿Qué pasa si se muestrea a 8 Hz?

3.5. Ejercicio 25: Error en Mediciones con DAQ

Problema: Se mide una señal sinusoidal de 1 V pico a 50 Hz con un DAQ de:

- 10 bits, rango 0-5 V
- $f_s = 1$ kHz
- Error de cuantización uniforme

Calcule:

1. Relación señal-ruido de cuantización (SQNR)
2. Error RMS de cuantización
3. Potencia del error de cuantización

3.6. Ejercicio 26: Comparación de Métodos de Medición

Problema: Para medir g en un plano inclinado con $\theta = 10^\circ$, compare teóricamente:

1. Precisión esperada con sensor de posición ($\Delta x = 1$ mm, $\Delta t = 1$ ms)
2. Precisión con Tracker ($\Delta x = 1$ cm, $\Delta t = 1/30$ s)
3. Precisión con cronómetro manual ($\Delta t = 0,1$ s, $\Delta x = 1$ cm)

3.7. Ejercicio 27: Detección de Rozamiento

Problema: Derive una expresión para detectar rozamiento a partir de:

1. Comparación de aceleraciones en subida y bajada
2. Análisis de decaimiento de amplitud en oscilaciones
3. Medición de fuerza constante necesaria para movimiento uniforme

3.8. Ejercicio 28: Optimización Experimental

Problema: Para minimizar el error en la medición de g en plano inclinado:

1. Encuentre el ángulo óptimo que minimice $\Delta g/g$
2. Determine la longitud óptima del plano
3. Optimice la frecuencia de muestreo del sensor

3.9. Ejercicio 29: Análisis de Incertidumbre Completo

Problema: Para $g = \frac{a_x}{\sin \theta}$, derive Δg considerando:

1. Δa_x del ajuste de $x(t)$
2. $\Delta \theta$ de la medición angular
3. Correlación entre parámetros

3.10. Ejercicio 30: Validación de Hipótesis

Problema: Proponga tests para validar:

1. Ausencia de rozamiento significativo
2. Linealidad del sensor
3. Constancia de g durante el experimento
4. Independencia de resultados respecto al método

4. Soluciones a Ejercicios Teóricos DAQ

4.1. Solución Ejercicio 21

1. Resolución: $\frac{20 \text{ V}}{2^{12}} = \frac{20}{4096} \approx 0,00488 \text{ V}$
2. Niveles: $2^{12} = 4096$
3. Ancho de banda: $f_{\max} < \frac{100 \text{ kHz}}{2} = 50 \text{ kHz}$ (Nyquist)
4. Error de no linealidad: $0,1 \% \times 20 \text{ V} = 0,02 \text{ V}$
5. Error de cuantización: $\frac{0,00488}{2} = 0,00244 \text{ V}$
6. Error total: $\sqrt{0,02^2 + 0,00244^2} \approx 0,0201 \text{ V}$

4.2. Solución Ejercicio 22

1. Ajuste lineal: $F = K_1 V + K_0$

$$K_1 = \frac{N \sum V_i F_i - \sum V_i \sum F_i}{N \sum V_i^2 - (\sum V_i)^2}$$
$$K_0 = \bar{F} - K_1 \bar{V}$$

Con los datos: $K_1 \approx 3,924 \text{ N/V}$, $K_0 \approx 0,00 \text{ N}$

2. Error cuadrático: $\sigma = \sqrt{\frac{\sum (F_i - K_1 V_i)^2}{N-2}}$

3. Para $V = 3,20 \text{ V}$: $F = 3,924 \times 3,20 = 12,56 \text{ N}$ Incerteza: $\Delta F = \sigma \sqrt{1 + \frac{1}{N} + \frac{(V - \bar{V})^2}{\sum (V_i - \bar{V})^2}}$

4.3. Solución Ejercicio 23

1. Con $v = 343 \text{ m/s}$: $D = 343 \times \frac{5,82 \times 10^{-3}}{2} = 0,997 \text{ m}$
2. A 25°C : $D = 346 \times \frac{5,82 \times 10^{-3}}{2} = 1,006 \text{ m}$
3. Error: $\Delta D = v \frac{\Delta \tau}{2} = 343 \times \frac{0,01 \times 10^{-3}}{2} = 0,0017 \text{ m}$

4.4. Solución Ejercicio 24

1. $f_{\max} = 3f_0 = 6 \text{ Hz} \rightarrow f_s > 12 \text{ Hz}$ (Nyquist)
2. Para buena reconstrucción: $f_s > 5f_{\max} = 30 \text{ Hz}$
3. A 8 Hz : Aliasing, la componente de 6 Hz aparecerá como 2 Hz

4.5. Solución Ejercicio 25

1. SQNR (dB) = $6,02n + 1,76 = 6,02 \times 10 + 1,76 \approx 62 \text{ dB}$
2. Resolución: $\frac{5}{2^{10}} = 0,00488 \text{ V}$ Error RMS: $\frac{0,00488}{\sqrt{12}} \approx 0,00141 \text{ V}$
3. Para señal de 1 V RMS : Potencia señal = 1 W Potencia error = $(0,00141)^2 = 2 \times 10^{-6} \text{ W}$

4.6. Solución Ejercicio 26

Para $\theta = 10^\circ$, $a_x = g \sin \theta \approx 1,70 \text{ m/s}^2$

1. Sensor: $\frac{\Delta a_x}{a_x} \approx 2 \frac{\Delta x}{x} + 2 \frac{\Delta t}{t}$ Con $t \approx 1 \text{ s}$: $\frac{\Delta a_x}{a_x} \approx 0,3 \%$
2. Tracker: $\frac{\Delta a_x}{a_x} \approx 2 \%$
3. Cronómetro: $\frac{\Delta a_x}{a_x} \approx 20 \%$

4.7. Solución Ejercicio 27

1. Con rozamiento: $a_{\text{bajada}} \neq a_{\text{subida}}$ $\mu_k = \frac{a_{\text{subida}} - a_{\text{bajada}}}{2g \cos \theta}$
2. En oscilaciones: $A(t) = A_0 e^{-\gamma t}$ $\gamma = \frac{b}{2m}$
3. Fuerza constante: $F = \mu_k mg \cos \theta$

4.8. Solución Ejercicio 28

1. $\frac{\Delta g}{g} = \sqrt{\left(\frac{\Delta a_x}{a_x}\right)^2 + \left(\frac{\Delta \theta}{\tan \theta}\right)^2}$ Mínimo cuando $\frac{\Delta a_x}{a_x} \approx \frac{\Delta \theta}{\tan \theta}$
2. Longitud óptima: compensa error en t vs error en x
3. f_s óptima: suficiente para capturar movimiento pero no excesiva

4.9. Solución Ejercicio 29

$$\begin{aligned} \Delta g^2 &= \left(\frac{1}{\sin \theta}\right)^2 \Delta a_x^2 + \left(-\frac{a_x \cos \theta}{\sin^2 \theta}\right)^2 \Delta \theta^2 \\ &\quad + 2 \left(\frac{1}{\sin \theta}\right) \left(-\frac{a_x \cos \theta}{\sin^2 \theta}\right) \text{Cov}(a_x, \theta) \end{aligned}$$

4.10. Solución Ejercicio 30

1. Rozamiento: comparar a_x teórico y experimental
2. Linealidad: test de chi-cuadrado en calibración
3. Constancia de g : analizar residuos del ajuste
4. Independencia: test de comparación de medias

5. Ejercicios Prácticos - Procesamiento de Señales

5.1. Ejercicio 31: Calibración de Sensor de Fuerza

Problema: Implemente calibración completa de sensor:

1. Adquirir datos de calibración con pesos conocidos
2. Ajustar recta de calibración
3. Calcular incertezas en parámetros
4. Validar calibración con puntos de prueba

5.2. Ejercicio 32: Adquisición de Señal Oscilatoria

Problema: Adquiera señal de sistema masa-resorte y:

1. Determine frecuencia natural del sistema
2. Calcule constante elástica k
3. Detecte amortiguamiento si existe
4. Compare con medición estática de k

5.3. Ejercicio 33: Filtrado Digital de Señales

Problema: Implemente filtrado de señal ruidosa:

1. Filtro media móvil
2. Filtro de Savitzky-Golay
3. Filtro pasa-bajos Butterworth
4. Compare efectividad y distorsión

5.4. Ejercicio 34: Análisis Espectral

Problema: Aplique Transformada de Fourier a señal oscilatoria:

1. Calcule espectro de potencia
2. Identifique frecuencias dominantes
3. Detecte armónicos
4. Estime relación señal-ruido

5.5. Ejercicio 35: Integración de Métodos

Problema: Compare mediciones de plano inclinado con:

1. Sensor de posición DAQ
2. Tracker
3. Cronómetro manual
4. Realice análisis estadístico comparativo

5.6. Ejercicio 36: Detección Automática de Eventos

Problema: Implemente detección automática de:

1. Inicio y fin del movimiento
2. Máximos y mínimos en oscilaciones *·Cambios dependiente en señales*

5.7. Ejercicio 37: Análisis de Correlación

Problema: Analice correlación entre:

1. Señales de posición y fuerza
2. Mediciones de diferentes sensores
3. Señal y su derivada

5.8. Ejercicio 38: Simulación de Sistema Completo

Problema: Simule sistema masa-resorte-amortiguador:

1. Genere señales con ruido realista
2. Aplique técnicas de procesamiento
3. Recupere parámetros del sistema
4. Evalúe precisión de recuperación

5.9. Ejercicio 39: Optimización de Parámetros DAQ

Problema: Optimice experimentalmente:

1. Frecuencia de muestreo
2. Ganancia del amplificador
3. Longitud de registro
4. Número de promedios

5.10. Ejercicio 40: Reporte Automatizado

Problema: Cree sistema que automáticamente:

1. Adquiera y procese datos
2. Realice análisis estadístico
3. Genere gráficos profesionales
4. Produzca reporte en PDF/LaTeX

6. Código Python para Procesamiento de Señales

Listing 1: Calibración de sensor y procesamiento básico

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
from scipy import signal, fft
import pandas as pd

def ejercicio_31():
    """ Calibración completa de sensor de fuerza """
    # Datos de calibración simulados
    np.random.seed(42)

    # Pesos conocidos (N)
    fuerzas_conocidas = np.array([0.0, 4.90, 9.81, 14.72, 19.62, 24.53])

    # Lecturas de voltaje correspondientes (con ruido)
    voltajes = np.array([0.00, 1.25, 2.50, 3.75, 5.00, 6.25])
    voltajes_medidos = voltajes + np.random.normal(0, 0.05, len(voltajes))

    # Ajuste lineal:  $F = K0 + K1 * V$ 
    def modelo_calibracion(V, K0, K1):
        return K0 + K1 * V

    # Ajuste con pesos (incertezas en fuerza)
    dF = 0.05 # incerteza en fuerza (N)
    pesos = 1/(dF**2)

    popt, pcov = curve_fit(modelo_calibracion, voltajes_medidos,
                           fuerzas_conocidas, sigma=dF*np.ones_like(fuerzas_
                               absolute_sigma=True))

    K0, K1 = popt
    dK0, dK1 = np.sqrt(np.diag(pcov))

    # Validación con puntos de prueba
    fuerzas_predichas = modelo_calibracion(voltajes_medidos, K0, K1)
    residuos = fuerzas_conocidas - fuerzas_predichas
    chi2 = np.sum((residuos/dF)**2)
    chi2_red = chi2/(len(fuerzas_conocidas)-2)

    # Gráficos
    fig, axes = plt.subplots(2, 2, figsize=(12, 10))

    # Calibración
    V_fit = np.linspace(-0.5, 7.0, 100)
    F_fit = modelo_calibracion(V_fit, K0, K1)
```

```

axes[0,0].errorbar(voltajes_medidas , fuerzas_conocidas ,
                   xerr=0.05, yerr=dF, fmt='o', capsize=5,
                   label='Datos de calibraci3n ')
axes[0,0].plot(V_fit , F_fit , 'r-',
               label=f'F={K0:.3f}+{K1:.3f} V ')
axes[0,0].set_xlabel('Voltaje (V) ')
axes[0,0].set_ylabel('Fuerza (N) ')
axes[0,0].legend()
axes[0,0].grid(True, alpha=0.3)
axes[0,0].set_title('Curva de calibraci3n ')

# Residuos
axes[0,1].errorbar(fuerzas_conocidas , residuos ,
                   xerr=dF, yerr=dF, fmt='o', capsize=5)
axes[0,1].axhline(y=0, color='r', linestyle='—')
axes[0,1].set_xlabel('Fuerza conocida (N) ')
axes[0,1].set_ylabel('Residuos (N) ')
axes[0,1].grid(True, alpha=0.3)
axes[0,1].set_title(f'Residuos (chi2={chi2:.2f}, red={chi2_red:.2f})')

# Test de puntos adicionales
voltajes_test = np.array([1.00, 2.00, 4.00, 5.50])
fuerzas_test_real = np.array([3.92, 7.85, 15.70, 21.58])
voltajes_test_med = voltajes_test + np.random.normal(0, 0.05, len(voltajes_test))
fuerzas_test_pred = modelo_calibracion(voltajes_test_med , K0, K1)

axes[1,0].errorbar(fuerzas_test_real , fuerzas_test_pred ,
                   xerr=dF, yerr=dF*np.sqrt(2), fmt='s', capsize=5,
                   label='Puntos de prueba ')
axes[1,0].plot([0, 25], [0, 25], 'k—', alpha=0.5, label='L3nea ideal ')
axes[1,0].set_xlabel('Fuerza real (N) ')
axes[1,0].set_ylabel('Fuerza predicha (N) ')
axes[1,0].legend()
axes[1,0].grid(True, alpha=0.3)
axes[1,0].set_title('Validaci3n de calibraci3n ')

# Histograma de residuos
axes[1,1].hist(residuos , bins=8, density=True, alpha=0.7)
axes[1,1].set_xlabel('Residuos (N) ')
axes[1,1].set_ylabel('Densidad ')
axes[1,1].grid(True, alpha=0.3)
axes[1,1].set_title('Distribuci3n de residuos ')

plt.tight_layout()
plt.show()

# Resultados

```

```

print ("==== Resultados de Calibraci3n ====")
print (f"K0={K0:.4 f} - {dK0:.4 f}-N")
print (f"K1={K1:.4 f} - {dK1:.4 f}-N/V")
print (f"Sensibilidad={K1:.3 f}-N/V")
print (f"Resoluci3n te3rica={dK1*0.001:.3 f}-N (para 1 mV)")
print (f"      reducido={chi2_red:.2 f}")

# Funci3n de calibraci3n inversa ( $V \rightarrow F$ )
def calibrar_voltaje(V, dV=0.05):
    F = K0 + K1 * V
    dF_cal = np.sqrt(dK0**2 + (V*dK1)**2 + (K1*dV)**2)
    return F, dF_cal

return calibrar_voltaje

def ejercicio_32():
    """Adquisici3n y an3lisis de se3al oscilatoria"""
    # Simulaci3n de sistema masa-resorte
    np.random.seed(123)

    # Par3metros del sistema
    m = 0.5 # kg
    k = 20.0 # N/m
    b = 0.1 # N s/m (amortiguamiento)

    # Soluci3n de la ecuaci3n:  $m x'' + b x' + k x = 0$ 
    omega0 = np.sqrt(k/m) # frecuencia natural
    gamma = b/(2*m) # coeficiente de amortiguamiento

    if gamma < omega0: # subamortiguado
        omega = np.sqrt(omega0**2 - gamma**2)
        T = 2*np.pi/omega
    else: # sobreamortiguado o cr3tico
        T = 2*np.pi/omega0

    # Generar se3al te3rica
    t = np.linspace(0, 10, 1000)
    if gamma < omega0:
        x = np.exp(-gamma*t) * np.cos(omega*t)
    else:
        # Sobreamortiguado: dos exponenciales reales
        r1 = -gamma + np.sqrt(gamma**2 - omega0**2)
        r2 = -gamma - np.sqrt(gamma**2 - omega0**2)
        x = 0.5*(np.exp(r1*t) + np.exp(r2*t))

    # Normalizar y a3adir ruido
    x = 0.1 * x # amplitud de 10 cm
    x_noise = x + np.random.normal(0, 0.002, len(t)) # ruido de 2 mm

```



```

# Simular adquisición DAQ (con frecuencia de muestreo finita)
fs = 100 # Hz
t_sampled = np.arange(0, 10, 1/fs)
indices = (t_sampled * 100).astype(int) # convertir a ndices
indices = indices[indices < len(t)]
x_sampled = x_noise[indices]

# Análisis de la señal muestreada
# 1. Encontrar periodos
from scipy.signal import find_peaks
peaks, properties = find_peaks(x_sampled, height=0.005, distance=fs*T*(0.5))

if len(peaks) >= 2:
    periodos = np.diff(t_sampled[peaks])
    T_medido = np.mean(periodos)
    dT_medido = np.std(periodos)/np.sqrt(len(periodos))

    # Calcular k a partir del periodo
    k_dinamico = (2*np.pi/T_medido)**2 * m
    dk_dinamico = 2*(2*np.pi/T_medido)**3 * m * dT_medido
else:
    T_medido = T
    k_dinamico = k
    dk_dinamico = 0

# 2. Ajuste exponencial a la envolvente
if len(peaks) >= 3:
    envolvente = x_sampled[peaks]
    t_envolvente = t_sampled[peaks]

    def modelo_exp(t, A, gamma_fit):
        return A * np.exp(-gamma_fit * t)

    popt_env, pcov_env = curve_fit(modelo_exp, t_envolvente,
                                   np.abs(envolvente),
                                   p0=[0.1, gamma])

    A_fit, gamma_fit = popt_env
    dgamma_fit = np.sqrt(pcov_env[1,1])

    # Calcular b del amortiguamiento
    b_estimado = 2 * m * gamma_fit
    db_estimado = 2 * m * dgamma_fit
else:
    gamma_fit = gamma
    b_estimado = b
    db_estimado = 0

```

```

# Gráficas
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Se al completa
axes[0,0].plot(t, x, 'k-', alpha=0.5, label='Teórica (sin ruido)')
axes[0,0].plot(t_sampled, x_sampled, 'b.', markersize=2,
               label='Muestreada (con ruido)')
if len(peaks) > 0:
    axes[0,0].plot(t_sampled[peaks], x_sampled[peaks], 'ro',
                  markersize=5, label='Picos detectados')
axes[0,0].set_xlabel('Tiempo (s)')
axes[0,0].set_ylabel('Posición (m)')
axes[0,0].legend()
axes[0,0].grid(True, alpha=0.3)
axes[0,0].set_title(f'Señal oscilatoria (fs={fs} Hz)')

# Detalle de algunos periodos
if len(peaks) >= 3:
    idx_min = max(0, peaks[0]-50)
    idx_max = min(len(t_sampled), peaks[2]+50)
    axes[0,1].plot(t_sampled[idx_min:idx_max],
                  x_sampled[idx_min:idx_max], 'b.-')
    axes[0,1].plot(t_sampled[peaks[:3]], x_sampled[peaks[:3]], 'ro')
    for i, p in enumerate(peaks[:2]):
        axes[0,1].axvline(x=t_sampled[p], color='r', linestyle='—',
                          if i < len(periodos):
                              axes[0,1].text(t_sampled[p]+periodos[i]/2, 0.015,
                                              f'T={periodos[i]:.3f}s',
                                              ha='center', fontsize=9)
    axes[0,1].set_xlabel('Tiempo (s)')
    axes[0,1].set_ylabel('Posición (m)')
    axes[0,1].grid(True, alpha=0.3)
    axes[0,1].set_title(f'Periodos medidos: T_avg={T_medido:.3f}')

# Envolvente y ajuste exponencial
if len(peaks) >= 3:
    axes[1,0].plot(t_envolvente, np.abs(envolvente), 'bo',
                  label='Envolvente (picos)')
    t_env_fit = np.linspace(0, t_envolvente[-1], 100)
    env_fit = modelo_exp(t_env_fit, A_fit, gamma_fit)
    axes[1,0].plot(t_env_fit, env_fit, 'r-',
                  label=f'Ajuste: γ={gamma_fit:.3f} 1/s')
    axes[1,0].set_xlabel('Tiempo (s)')
    axes[1,0].set_ylabel('Amplitud (m)')
    axes[1,0].legend()
    axes[1,0].grid(True, alpha=0.3)
    axes[1,0].set_title('Decaimiento exponencial')

```

```

# Comparaci n de m todos para k
# Valor est tico (suponiendo medici n de elongaci n)
F_estatica = 1.0 # N
delta_x = 0.05 # m
k_estatico = F_estatica / delta_x
dk_estatico = k_estatico * 0.05 # 5% error

metodos = ['Din mico-(oscilaciones)', 'Est tico-(elongaci n)']
k_vals = [k_dinamico, k_estatico]
k_errs = [dk_dinamico, dk_estatico]

axes[1,1].errorbar(range(2), k_vals, yerr=k_errs, fmt='o', capsize=5)
axes[1,1].axhline(y=k, color='k', linestyle='—',
                  label=f'Valor real: {k:.1f} N/m')
axes[1,1].set_xticks(range(2))
axes[1,1].set_xticklabels(metodos, rotation=45, ha='right')
axes[1,1].set_ylabel('k (N/m)')
axes[1,1].legend()
axes[1,1].grid(True, alpha=0.3)
axes[1,1].set_title('Comparaci n de m todos para determinar k')

plt.tight_layout()
plt.show()

# Resultados
print("==== An lisis de Sistema Masa-Resorte ====")
print(f"Par metros reales: m={m:.3f} kg, k={k:.3f} N/m, b={b:.3f} N s/m")
print(f"\nAn lisis din mico:")
print(f"Per odo medido: {T_medido:.4f} s {dT_medido:.4f} s")
print(f"k din mico: {k_dinamico:.3f} {dk_dinamico:.3f} N/m")
print(f"(amortiguamiento): {gamma_fit:.4f} {dgamma_fit:.4f} 1/s")
print(f"b estimado: {b_estimado:.4f} {db_estimado:.4f} N s/m")

print(f"\nComparaci n con m todo est tico:")
print(f"k est tico: {k_estatico:.3f} {dk_estatico:.3f} N/m")
print(f"Diferencia: {abs(k_dinamico - k_estatico):.3f} N/m")

# Test de consistencia
print(f"\nTest de consistencia:")
if abs(k_dinamico - k_estatico) < np.sqrt(dk_dinamico**2 + dk_estatico**2):
    print("Los m todos son consistentes dentro de incertezas")
else:
    print("Los m todos NO son consistentes — posible fuente de error")

return k_dinamico, k_estatico, b_estimado

if __name__ == "__main__":
    print("="*50)

```

```

print("EJERCICIO-31: Calibraci n-de-sensor-de-fuerza")
print("="*50)
calibrar = ejercicio_31()

print("\\n" + "="*50)
print("EJERCICIO-32: An lisis-de-se al-oscilatoria")
print("="*50)
ejercicio_32()

```

6.1. Ejercicio 33: Filtrado Digital de Señales (continuación)

Listing 2: Filtrado digital de señales

```

def ejercicio_33():
    """ Comparaci n-de-t cnicas-de-filtrado-digital """
    import numpy as np
    import matplotlib.pyplot as plt
    from scipy import signal

    # Generar se al de prueba
    np.random.seed(42)
    t = np.linspace(0, 10, 1000)

    # Se al limpia (oscilaci n + tendencia)
    f1, f2 = 1.0, 3.5 # Hz
    se al_limpia = (0.5 * np.sin(2*np.pi*f1*t) +
                   0.3 * np.sin(2*np.pi*f2*t) +
                   0.1 * t) # tendencia lineal

    # Aadir diferentes tipos de ruido
    ruido_blanco = np.random.normal(0, 0.2, len(t))
    ruido_60hz = 0.1 * np.sin(2*np.pi*60*t) # interferencia de l nea
    ruido_impulsivo = np.zeros(len(t))
    impulsos = np.random.randint(0, len(t), 20)
    ruido_impulsivo[impulsos] = np.random.uniform(-1, 1, 20)

    se al_ruidosa = (se al_limpia + ruido_blanco +
                   ruido_60hz + ruido_impulsivo)

    # 1. Filtro de media m vil
    window_size = 21
    filtro_media = np.ones(window_size)/window_size
    se al_filtrada_media = np.convolve(se al_ruidosa, filtro_media, mode=

    # 2. Filtro de Savitzky-Golay
    window_sg = 21
    polyorder = 3
    se al_filtrada_sg = signal.savgol_filter(se al_ruidosa, window_sg, po

```

```

# 3. Filtro Butterworth pasa-bajos
fs = 100 # frecuencia de muestreo (Hz)
fc = 5.0 # frecuencia de corte (Hz)
orden = 4

# Diseñar filtro Butterworth
b, a = signal.butter(orden, fc/(fs/2), btype='low')
se_al_filtrada_butter = signal.filtfilt(b, a, se_al_ruidosa)

# 4. Filtro mediana (bueno para ruido impulsivo)
window_mediana = 11
se_al_filtrada_mediana = signal.medfilt(se_al_ruidosa, window_mediana)

# Análisis comparativo
metodos = ['Media-mvil', 'Savitzky-Golay', 'Butterworth', 'Mediana']
se_ales_filtradas = [se_al_filtrada_mediana, se_al_filtrada_sg,
                     se_al_filtrada_butter, se_al_filtrada_mediana]

# Calcular métricas de calidad
def calcular_metricas(se_al_filtrada, se_al_limpia):
    # Error cuadrático medio normalizado
    mse = np.mean((se_al_filtrada - se_al_limpia)**2)
    nmse = mse / np.var(se_al_limpia)

    # Relación señal-ruido (SNR) en dB
    potencia_senal = np.mean(se_al_limpia**2)
    potencia_error = np.mean((se_al_filtrada - se_al_limpia)**2)
    snr = 10 * np.log10(potencia_senal/potencia_error) if potencia_error > 0 else -10

    # Distorsión de picos
    picos_original = signal.find_peaks(se_al_limpia)[0]
    picos_filtrada = signal.find_peaks(se_al_filtrada)[0]

    if len(picos_original) > 0 and len(picos_filtrada) > 0:
        # Encontrar picos correspondientes
        distorsion_picos = []
        for p in picos_original[:min(5, len(picos_original), len(picos_filtrada))]:
            idx_cercano = np.argmin(np.abs(p - picos_filtrada))
            dist = abs(se_al_limpia[p] - se_al_filtrada[picos_filtrada[idx_cercano]])
            distorsion_picos.append(dist)
        distorsion_media = np.mean(distorsion_picos) if distorsion_picos else 0
    else:
        distorsion_media = 0

    return {'NMSE': nmse, 'SNR_dB': snr, 'Distorsión_picos': distorsion_media}

# Calcular métricas para cada método

```

```

metricas = []
for i, se_al_filt in enumerate(se_ales_filtradas):
    met = calcular_metricas(se_al_filt, se_al_limpia)
    metricas.append(met)

# Gráficas
fig, axes = plt.subplots(3, 2, figsize=(14, 12))

# Se al original vs ruidosa
axes[0,0].plot(t, se_al_limpia, 'b-', alpha=0.7, label='Se al limpia')
axes[0,0].plot(t, se_al_ruidosa, 'r-', alpha=0.3, label='Se al ruidosa')
axes[0,0].set_xlabel('Tiempo (s)')
axes[0,0].set_ylabel('Amplitud')
axes[0,0].legend()
axes[0,0].grid(True, alpha=0.3)
axes[0,0].set_title('Se al original y con ruido')

# Espectro de la se al ruidosa
fft_se_al = np.fft.fft(se_al_ruidosa)
freqs = np.fft.fftfreq(len(t), t[1]-t[0])
idx_pos = freqs > 0
axes[0,1].plot(freqs[idx_pos], np.abs(fft_se_al[idx_pos]), 'b-')
axes[0,1].axvline(x=fc, color='r', linestyle='—', label=f'f corte = {fc}')
axes[0,1].set_xlabel('Frecuencia (Hz)')
axes[0,1].set_ylabel('Amplitud')
axes[0,1].set_xlim([0, 20])
axes[0,1].legend()
axes[0,1].grid(True, alpha=0.3)
axes[0,1].set_title('Espectro de la se al ruidosa')

# Comparación de filtros (primeros 2 segundos)
idx = t < 2
colors = ['blue', 'green', 'red', 'orange']

for i, (nombre, se_al_filt, color) in enumerate(zip(m_todos, se_ales_filtradas)):
    axes[1,0].plot(t[idx], se_al_filt[idx], color=color,
                  alpha=0.7, label=nombre)
    axes[1,0].plot(t[idx], se_al_limpia[idx], 'k—', alpha=0.5, label='Limpia')
    axes[1,0].set_xlabel('Tiempo (s)')
    axes[1,0].set_ylabel('Amplitud')
    axes[1,0].legend()
    axes[1,0].grid(True, alpha=0.3)
    axes[1,0].set_title('Comparación de filtros (zoom)')

# Residuos de cada filtro
for i, (nombre, se_al_filt, color) in enumerate(zip(m_todos, se_ales_filtradas)):
    residuos = se_al_filt - se_al_limpia
    axes[1,1].plot(t[idx], residuos[idx], color=color,

```

```

        alpha=0.7, label=nombre)
axes[1,1].axhline(y=0, color='k', linestyle='—', alpha=0.5)
axes[1,1].set_xlabel('Tiempo (s)')
axes[1,1].set_ylabel('Residuos')
axes[1,1].legend()
axes[1,1].grid(True, alpha=0.3)
axes[1,1].set_title('Residuos de los filtros')

# Mtricas comparativas
nombres_mtricas = list(mtricas[0].keys())
x_pos = np.arange(len(m_todos))

for j, metrica in enumerate(nombres_mtricas):
    valores = [m[metrica] for m in mtricas]
    axes[2,0].bar(x_pos + j*0.2, valores, width=0.2,
                  label=metrica, alpha=0.7)

axes[2,0].set_xticks(x_pos + 0.2)
axes[2,0].set_xticklabels(m_todos, rotation=45, ha='right')
axes[2,0].set_ylabel('Valor')
axes[2,0].legend()
axes[2,0].grid(True, alpha=0.3, axis='y')
axes[2,0].set_title('Mtricas de calidad por filtro')

# Radar chart para comparaci n visual
# Normalizar mtricas (menor es mejor para NMSE y Distorsi n)
met_normalizadas = []
for met in mtricas:
    met_norm = {}
    for k, v in met.items():
        if k == 'SNR_dB':
            # Para SNR, mayor es mejor
            met_norm[k] = v / 50 if v < 50 else 1.0
        else:
            # Para NMSE y Distorsi n, menor es mejor
            met_norm[k] = 1.0 / (1.0 + v)
    met_normalizadas.append(met_norm)

# Crear radar chart
from matplotlib.patches import Circle
angles = np.linspace(0, 2*np.pi, len(nombres_mtricas), endpoint=False)
angles += angles[:1] # cerrar el pol gono

ax_radar = axes[2,1]
ax_radar.set_theta_offset(np.pi/2)
ax_radar.set_theta_direction(-1)
ax_radar.set_xticks(angles[:-1])
ax_radar.set_xticklabels(nombres_mtricas)

```

```

# Dibujar circulos de referencia
for r in [0.25, 0.5, 0.75, 1.0]:
    circle = Circle((0,0), r, transform=ax_radar.transData.b,
                    fill=False, alpha=0.1, color='gray')
    ax_radar.add_patch(circle)

# Dibujar cada m todo
for i, (nombre, met_norm) in enumerate(zip(m_todos, met_normalizadas)):
    valores = [met_norm[met] for met in nombres_metricas]
    valores += valores[:1] # cerrar el poligono
    ax_radar.plot(angles, valores, 'o-', linewidth=2,
                 label=nombre, alpha=0.7)
    ax_radar.fill(angles, valores, alpha=0.1)

ax_radar.set_ylim([0, 1.1])
ax_radar.legend(loc='upper-right', bbox_to_anchor=(1.3, 1.0))
ax_radar.grid(True, alpha=0.3)
ax_radar.set_title('Comparación radial de filtros')

plt.tight_layout()
plt.show()

# Resultados detallados
print("===== Comparación de Técnicas de Filtrado =====")
print("\nMétricas de calidad (menor NMSE y Distorsión es mejor, mayor SNR es peor)")
print("-" * 70)
print(f"{'M todo': <20} - {'NMSE': <10} - {'SNR (dB)': <10} - {'Distorsión picos': <10}")
print("-" * 70)

for i, (nombre, met) in enumerate(zip(m_todos, metricas)):
    print(f"{' nombre': <20} - {met['NMSE']: <10.4 f} - {met['SNR_dB']: <10.2 f} - {met['Distorsión picos']: <10.2 f}")

# Recomendaciones
print("\n===== Recomendaciones =====")
mejor_nmse = np.argmin([m['NMSE'] for m in metricas])
mejor_snr = np.argmax([m['SNR_dB'] for m in metricas])
mejor_dist = np.argmin([m['Distorsión picos'] for m in metricas])

print(f"Mejor NMSE: {m_todos[mejor_nmse]}")
print(f"Mejor SNR: {m_todos[mejor_snr]}")
print(f"Menor distorsión de picos: {m_todos[mejor_dist]}")

print("\nRecomendaciones específicas:")
print("1.- Media móvil: Simple, buen suavizado, pero introduce retardo")
print("2.- Savitzky-Golay: Excelente para preservar características espectrales")
print("3.- Butterworth: Buen rechazo de ruido de alta frecuencia")
print("4.- Mediana: Excelente para ruido impulsivo (outliers)")

```



```

    return metricas, m todos

# Ejecutar ejercicio
if __name__ == "__main__":
    print("\n" + "="*50)
    print("EJERCICIO 33: Comparación de técnicas de filtrado")
    print("="*50)
    ejercicio_33()

```

6.2. Ejercicio 34: Análisis Espectral (continuación)

Listing 3: Análisis espectral de señales

```

def ejercicio_34():
    """Análisis espectral de señales experimentales"""
    import numpy as np
    import matplotlib.pyplot as plt
    from scipy import signal

    # Generar señal compuesta para análisis
    np.random.seed(42)
    fs = 1000 # frecuencia de muestreo (Hz)
    t = np.linspace(0, 10, 10*fs) # 10 segundos de datos

    # Componentes de la señal
    # 1. Oscilación principal
    f1 = 5.0 # Hz
    A1 = 2.0
    componente1 = A1 * np.sin(2*np.pi*f1*t)

    # 2. Armónicos
    f2 = 15.0 # 3er armónico de f1
    A2 = 0.5
    componente2 = A2 * np.sin(2*np.pi*f2*t)

    f3 = 25.0 # 5to armónico
    A3 = 0.2
    componente3 = A3 * np.sin(2*np.pi*f3*t)

    # 3. Ruido de baja frecuencia (drift)
    f_drift = 0.1 # Hz
    A_drift = 0.3
    drift = A_drift * np.sin(2*np.pi*f_drift*t)

    # 4. Ruido de alta frecuencia (interferencia)
    f_interf = 60.0 # Hz (interferencia de línea)
    A_interf = 0.4

```

```

interf = A_interf * np.sin(2*np.pi*f_interf*t)

# 5. Ruido blanco
ruido_blanco = np.random.normal(0, 0.1, len(t))

# Se al compuesta
se_al = (componente1 + componente2 + componente3 +
         drift + interf + ruido_blanco)

# Ventana para reducir leakage
ventana = signal.windows.hann(len(se_al))
se_al_ventaneada = se_al * ventana

# Transformada de Fourier
fft_resultado = np.fft.fft(se_al_ventaneada)
fft_freqs = np.fft.fftfreq(len(se_al), 1/fs)

# Solo frecuencias positivas
idx_pos = fft_freqs >= 0
frecuencias = fft_freqs[idx_pos]
espectro = np.abs(fft_resultado[idx_pos])
espectro_db = 20 * np.log10(espectro + 1e-10) # en dB

# Encontrar picos en el espectro
from scipy.signal import find_peaks
# Usar altura minima relativa al pico maximo
altura_minima = 0.1 * np.max(espectro)
picos, propiedades = find_peaks(espectro, height=altura_minima, distance=10)

# Calcular potencia de se al y ruido
def calcular_potencia_banda(espectro, freqs, f_min, f_max):
    mask = (freqs >= f_min) & (freqs <= f_max)
    potencia = np.sum(espectro[mask]**2)
    return potencia

# Banda de se al (alrededor de componentes conocidas)
potencia_total = np.sum(espectro**2)

# Potencia en componentes principales
pot_f1 = calcular_potencia_banda(espectro, frecuencias, f1-0.5, f1+0.5)
pot_f2 = calcular_potencia_banda(espectro, frecuencias, f2-0.5, f2+0.5)
pot_f3 = calcular_potencia_banda(espectro, frecuencias, f3-0.5, f3+0.5)

# Potencia en ruido (excluyendo componentes principales)
mascara_ruido = np.ones(len(frecuencias), dtype=bool)
for f in [f1, f2, f3, f_drift, f_interf]:
    idx_excluir = (frecuencias >= f-1) & (frecuencias <= f+1)
    mascara_ruido[idx_excluir] = False

```

```

potencia_ruido = np.sum(espectro[mascara_ruido]**2)

# Relaciones se al-ruido
snr_total = 10 * np.log10((potencia_total - potencia_ruido) / potencia_ruido)
snr_f1 = 10 * np.log10(pot_f1 / potencia_ruido)

# Analisis de distorsion armónica
# Buscar armónicos de f1
f_fundamental = f1
arm_nicos = []
for n in range(2, 6): # hasta 5to armónico
    f_arm = n * f_fundamental
    idx_arm = np.argmin(np.abs(frecuencias - f_arm))
    if idx_arm < len(espectro):
        amplitud_arm = espectro[idx_arm]
        arm_nicos.append((n, f_arm, amplitud_arm))

# THD (Total Harmonic Distortion)
if arm_nicos:
    pot_armonicos = sum(amp**2 for _, _, amp in arm_nicos)
    thd = np.sqrt(pot_armonicos / pot_f1)
    thd_db = 20 * np.log10(thd)
else:
    thd = 0
    thd_db = -np.inf

# Gráficas
fig, axes = plt.subplots(3, 2, figsize=(14, 12))

# Se al en tiempo (primer segundo)
idx_tiempo = t < 1.0
axes[0,0].plot(t[idx_tiempo], se_al[idx_tiempo], 'b-')
axes[0,0].set_xlabel('Tiempo (s)')
axes[0,0].set_ylabel('Amplitud')
axes[0,0].grid(True, alpha=0.3)
axes[0,0].set_title('Se al en dominio temporal (primer segundo)')

# Espectro completo (lineal)
axes[0,1].plot(frecuencias, espectro, 'b-')
axes[0,1].plot(frecuencias[picos], espectro[picos], 'ro', markersize=5)
for p in picos[:5]: # etiquetar primeros 5 picos
    axes[0,1].annotate(f'{frecuencias[p]:.1f} Hz',
                       xy=(frecuencias[p], espectro[p]),
                       xytext=(5, 5), textcoords='offset points',
                       fontsize=8, ha='center')
axes[0,1].set_xlabel('Frecuencia (Hz)')
axes[0,1].set_ylabel('Amplitud')

```

```

axes[0,1].set_xlim([0, 100])
axes[0,1].grid(True, alpha=0.3)
axes[0,1].set_title('Espectro-de-amplitud-(lineal)')

# Espectro en dB
axes[1,0].plot(frecuencias, espectro_db, 'b-')
axes[1,0].set_xlabel('Frecuencia-(Hz)')
axes[1,0].set_ylabel('Amplitud-(dB)')
axes[1,0].set_xlim([0, 100])
axes[1,0].grid(True, alpha=0.3)
axes[1,0].set_title('Espectro-de-amplitud-(dB)')

# Distribuci n de potencia
bandas = [
    ('F-fundamental', pot_f1),
    ('F-arm nicos', pot_armonicos if 'pot_armonicos' in locals() else 0),
    ('Drift-(0.1-Hz)', calcular_potencia_banda(espectro, frecuencias, 0.1)),
    ('Interf-60-Hz', calcular_potencia_banda(espectro, frecuencias, 59.95)),
    ('Ruido-ancho', potencia_ruido)
]

nombres = [b[0] for b in bandas]
potencias = [b[1] for b in bandas]

axes[1,1].bar(nombres, potencias, alpha=0.7)
axes[1,1].set_ylabel('Potencia')
axes[1,1].set_xticklabels(nombres, rotation=45, ha='right')
axes[1,1].grid(True, alpha=0.3, axis='y')
axes[1,1].set_title('Distribuci n-de-potencia-por-banda')

# Analisis de arm nicos
if arm_nicos:
    ordenes = [f'n={n}' for n, _, _ in arm_nicos]
    frecs_arm = [f for _, f, _ in arm_nicos]
    amps_arm = [a for _, _, a in arm_nicos]
    amps_rel = [a/amps_arm[0] for a in amps_arm] if amps_arm[0] > 0 else 0

    x = np.arange(len(ordenes))
    axes[2,0].bar(x, amps_rel, alpha=0.7)
    axes[2,0].set_xticks(x)
    axes[2,0].set_xticklabels(ordenes)
    axes[2,0].set_ylabel('Amplitud-relativa-al-fundamental')
    axes[2,0].grid(True, alpha=0.3, axis='y')
    axes[2,0].set_title(f'Arm nicos-de-{f_fundamental:.1f}-Hz-(THD={thd:.1f})')

# Relaciones se al-ruido por frecuencia
# Calcular SNR en ventanas deslizantes
ancho_ventana = 2.0 # Hz

```

```

frec_snr = []
snr_local = []

for f_centro in np.arange(1, 100, 2):
    f_min = f_centro - ancho_ventana/2
    f_max = f_centro + ancho_ventana/2

    # Potencia en banda
    mask_banda = (frecuencias >= f_min) & (frecuencias <= f_max)
    pot_banda = np.sum(espectro[mask_banda]**2)

    # Estimar ruido en bandas adyacentes
    mask_ruido1 = (frecuencias >= f_min-3*ancho_ventana) & (frecuencias <
    mask_ruido2 = (frecuencias >= f_max+ancho_ventana) & (frecuencias <
    mask_ruido = mask_ruido1 | mask_ruido2

    if np.sum(mask_ruido) > 10: # suficiente puntos para estimaci n
        pot_ruido_estimada = np.mean(espectro[mask_ruido]**2) * np.sum(
        if pot_ruido_estimada > 0:
            snr_local_db = 10 * np.log10(pot_banda / pot_ruido_estimada)
            frec_snr.append(f_centro)
            snr_local.append(snr_local_db)

axes[2,1].plot(frec_snr, snr_local, 'b-')
axes[2,1].axhline(y=snr_total, color='r', linestyle='—',
                  label=f'SNR total = {snr_total:.1f}-dB')
axes[2,1].set_xlabel('Frecuencia central (Hz)')
axes[2,1].set_ylabel('SNR local (dB)')
axes[2,1].grid(True, alpha=0.3)
axes[2,1].legend()
axes[2,1].set_title('SNR en funci n de la frecuencia')

plt.tight_layout()
plt.show()

# Resultados detallados
print("==== An lisis Espectral Detallado ====")
print(f"\nFrecuencia de muestreo: {fs}-Hz")
print(f"Duraci n de la se al: {t[-1]:.1f}-s")
print(f"Resoluci n espectral: {frecuencias[1]-frecuencias[0]:.3f}-Hz")

print(f"\nComponentes principales detectadas:")
for i, p in enumerate(picos[:10]): # mostrar primeras 10
    print(f"-- Pico {i+1}: {frecuencias[p]:6.2f}-Hz, Amplitud: {espectro[p]}")

print(f"\nAn lisis de potencia:")
print(f"-- Potencia total: {potencia_total:.3f}")
print(f"-- Potencia en {f1:.1f}-Hz: {pot_f1:.3f} ({100*pot_f1/potencia_total}%")

```

```

print(f" - Potencia en ruido: {potencia_ruido:.3 f} - ({100*potencia_ruido/

print(f"\nRelaciones se al ruido:")
print(f" - SNR total: {snr_total:.1 f} - dB")
print(f" - SNR a {f1:.1 f} - Hz: {snr_f1:.1 f} - dB")

if arm_nicos:
    print(f"\nAnálisis de distorsión armónica (fundamental={f_fundam
    for n, f_arm, amp in arm_nicos:
        amp_rel = amp/espectro[np.argmax(np.abs(frecuencias - f_fundam
        print(f" - Armónico {n}: {f_arm:.1 f} - Hz, Amplitud: {amp:.3 f} - (
    print(f" - THD total: {thd:.4 f} - ({thd_db:.1 f} - dB)")

# Detección automática de características
print(f"\nDetección automática de características:")

# Encontrar frecuencia fundamental (pico más prominente en bajas frecu
mask_bajas = frecuencias < 20
if np.any(mask_bajas):
    idx_fund = np.argmax(espectro[mask_bajas])
    f_fund_detectada = frecuencias[mask_bajas][idx_fund]
    print(f" - Frecuencia fundamental detectada: {f_fund_detectada:.2 f} -

# Detectar presencia de interferencia de 60 Hz
mask_60hz = (frecuencias >= 59) & (frecuencias <= 61)
if np.any(mask_60hz):
    pot_60hz = np.sum(espectro[mask_60hz]**2)
    rel_60hz = pot_60hz / potencia_total
    if rel_60hz > 0.01: # más del 1%
        print(f" - Advertencia! Interferencia de 60-Hz detectada ({100

# Evaluar calidad de la señal
print(f"\nEvaluación de calidad de la señal:")
if snr_total > 20:
    print(f" - Buena relación se al ruido (SNR > 20 dB)")
elif snr_total > 10:
    print(f" - SNR moderada (10-20 dB)")
else:
    print(f" - SNR pobre (< 10 dB)")

if thd < 0.05: # 5%
    print(f" - Baja distorsión armónica (THD < 5%)")
elif thd < 0.10:
    print(f" - Distorsión armónica moderada (5-10%)")
else:
    print(f" - Alta distorsión armónica (> 10%)")

return {

```

```

        'frecuencias': frecuencias,
        'espectro': espectro,
        'espectro_db': espectro_db,
        'picos': picos,
        'snr_total': snr_total,
        'thd': thd,
        'potencias': dict(bandas)
    }

# Ejecutar ejercicio
if __name__ == "__main__":
    print("\n" + "="*50)
    print("EJERCICIO 34: Análisis espectral")
    print("="*50)
    ejercicio_34()

```

7. Conclusión del Volumen 4B

En esta segunda parte del Volumen 4 hemos cubierto:

1. Fundamentos de sistemas DAQ y adquisición automatizada
2. Caracterización y calibración de sensores
3. Procesamiento digital de señales
4. Análisis espectral avanzado
5. Integración de múltiples métodos de medición

Estas técnicas representan el estado del arte en experimentación física moderna, permitiendo mediciones más precisas, automatizadas y comprensivas. La combinación de hardware especializado (sensores, DAQ) con software de procesamiento avanzado (Python, algoritmos específicos) abre posibilidades experimentales que eran impensables hace algunas décadas.

Los estudiantes que dominen estas técnicas estarán preparados para abordar problemas experimentales complejos en investigación avanzada y aplicaciones industriales.